



National University of Computer and Emerging Sciences, Lahore



StockSense

Muhammad Usman 23L-0693 BS(CS)

Muhammad Rizwan Arshad 23L-0755 BS(CS)

Hayyan Haider 23L-0727 BS(CS)

Supervisor: Mr. Saifullah Tanvir

Final Year Project

November 13, 2026

Anti-Plagiarism Declaration

This is to declare that the above publication was produced under the:

Title: StockSense

is the sole contribution of the author(s), and no part hereof has been reproduced as it is the basis (cut and paste) that can be considered Plagiarism. All referenced parts have been used to argue the idea and cited properly. I/We will be responsible and liable for any consequence if a violation of this declaration is determined.

Date: November 13, 2026

Name: Muhammad Usman

Signature: .....

Name: Muhammad Rizwan Arshad

Signature: .....

Name: Hayyan Haider

Signature: .....

Author's Declaration

This states Authors' declaration that the work presented in the report is their own, and has not been submitted/presented previously to any other institution or organization.

Abstract

Retailers lose money every day when a product is out of stock in one place while unsold units of the same product sit on the shelves of another. The problem is common in businesses that sell physical products in several variants, sizes or store locations, such as clothing and footwear. Large chains handle it with enterprise forecasting systems such as SAP and Blue Yonder, but these systems are too expensive and too complex for small businesses to buy or maintain. This project develops StockSense, a multi-tenant software as a service (SaaS) application built around a general-purpose forecasting engine that can serve any business that keeps stock. The platform offers a free point of sale (POS) module for recording sales and tracking inventory, while demand forecasting at the variant and location level, a markdown optimizer and stock redistribution recommendations are offered as paid subscription features. StockSense differs from current tools because it treats redistribution between branches as its first recommendation and warns the owner early enough to act before an item turns into dead stock, rather than after it has already stopped selling. Businesses that already use another POS or spreadsheets can join by uploading their past sales reports, so the system can learn from the data they already have.

Executive Summary

Retailers face two opposite problems every day, and both cost them revenue. Dead stock stays on the shelves for months and ties up capital and space, while stockouts lose sales because the item a customer wants is not available, even as less popular items keep piling up. This affects any retailer that manages physical inventory across variants, sizes or store locations, from clothing and footwear shops to many other kinds of stores. The problem is most serious in smaller businesses, where restocking is usually decided by instinct. Owners have no systematic view of which items sell at which locations, so a manager simply reorders whatever seemed to sell well. Large retailers solve this with enterprise demand forecasting platforms such as SAP and Blue Yonder, but these are expensive and designed for companies that already have a data team and an enterprise resource planning (ERP) system. Small retailers that run a basic POS or keep their records in spreadsheets have no equivalent. Even the lighter inventory tools made for small businesses usually forecast demand per product rather than per size or variant, alert the owner only when stock is already low, list slow movers after they have stopped selling, and treat each store's inventory separately.

StockSense is a multi-tenant SaaS platform designed to fill this gap. It is built on a general-purpose forecasting engine that works for any business that keeps stock, not only for one product category. Small retailers get a free POS that records sales and tracks stock as part of their daily work, and this brings their data onto the platform. On top of that data, a paid subscription unlocks demand forecasts at the location and variant level, an automated markdown optimizer and stock redistribution recommendations. Businesses that already use another POS or spreadsheets can join without changing anything by uploading their past sales in comma-separated values (CSV) or Excel format. In both cases, owners and managers see their past sales, current forecasts and restocking suggestions on a simple web dashboard that does not require any technical knowledge. StockSense differs from existing solutions in three ways. First, it puts redistribution first. Existing tools make recommendations for each store separately and mention transfers only as one option among many, yet for a business with two or more branches it is almost always cheaper and faster to move unsold stock to a branch where it sells than to discount it or place a new order.

Second, StockSense acts early instead of reacting late. Existing tools mark stock as dead once it is already too old to sell and then suggest a write-off or a discount, while the StockSense markdown optimizer detects an item as soon as its sales start to slow down and warns the manager while there is still time to act. Third, every recommendation is explained. Each suggestion comes with a short reason and an estimate of its financial impact, so the owner can understand it and trust it instead of following a black box. The preliminary work for the project included a review of the literature on demand forecasting

and small-business inventory management, a survey of enterprise and small-business platforms, and the selection of public retail sales datasets for building and validating the first models. We did not find any platform that combines a free POS with a paid layer for reducing dead stock in small retailers, which makes this freemium model unusual in the market. Forecast accuracy will be measured with standard error metrics, and the possible reduction in dead stock will be estimated by replaying historical sales. In short, StockSense helps small businesses use the sales data they already collect but rarely analyze, and gives them the kind of forecasting and inventory optimization that has so far been available only to large retailers.

Table of Contents

List of Figures **xii**

List of Tables **xiii**

1 Introduction **1**

1.1 Purpose of this Document 1

1.2 Intended Audience 1

1.3 Definitions, Acronyms, and Abbreviations 2

1.4 Conclusion 3

2 Project Vision **4**

2.1 Problem Domain Overview 4

2.2 Problem Statement 4

2.3 Problem Elaboration 5

2.4 Goals and Objectives 5

2.5 Project Scope 6

2.6 Sustainable Development Goal (SDG) 7

2.6.1 SDG-8 (Decent Work and Economic Growth) 7

2.6.2 SDG-9 (Industry, Innovation, and Infrastructure) 7

2.7 Constraints 7

2.8 Business Opportunity 8

2.9 Stakeholders Description / User Characteristics 8

2.9.1 Key High-Level Goals and Problems of Stakeholders 9

2.10 Conclusion 9

3 Literature Review / Related Work **10**

3.1 Definitions 10

3.2 Detailed Literature Review 10

3.2.1 Exploring the Use of Deep Neural Networks for Sales Forecasting in Fashion Retail . . . 11
3.2.2 Clearance Pricing Optimization for a Fast-Fashion Retailer . . . 12
3.2.3 Inventory Models with Lateral Transshipments: A Review . . . 13
3.2.4 Forecasting Across Time Series Databases Using Recurrent Neural Networks on Groups of Similar Series . . . 14
3.2.5 M5 Accuracy Competition: Results, Findings, and Conclusions . . . 15
3.2.6 Demand Forecasting for Improved Inventory Management in Small and Medium-Sized Businesses . . . 16
3.2.7 Retail Forecasting: Research and Practice . . . 17
3.2.8 Principles and Algorithms for Forecasting Groups of Time Series: Locality and Globality . . . 18
3.2.9 Reducing the Cost of Demand Uncertainty Through Accurate Response to Early Sales . . . 19
3.2.10 Sales Forecasts in Clothing Industry: The Key Success Factor of the Supply Chain Management . . . 20
3.2.11 Predictive Big Data Analytics for Supply Chain Demand Forecasting . . . 21
3.2.12 Forecasting: Theory and Practice . . . 22
3.2.13 From Predictive to Prescriptive Analytics . . . 23
3.2.14 Stop Explaining Black Box Machine Learning Models for High Stakes Decisions 24
3.2.15 Desperately Seeking Shelf Availability . . . 25
3.3 Literature Review Summary Table . . . 26
3.4 Related Work . . . 29
3.4.1 SAP Integrated Business Planning . . . 30
3.4.2 Blue Yonder . . . 30
3.4.3 Nextail . . . 30
3.4.4 Shopify POS (with Stocky) . . . 31
3.4.5 Lightspeed Retail . . . 31
3.4.6 Odoo . . . 31
3.5 Related Work Summary Table . . . 32
3.6 Conclusion . . . 32
3.7 Selected Model . . . 33

4 Software Requirement Specifications 34

4.1 List of Features . . . 34

4.2 Functional Requirements . . . 35
4.2.1 Functional Requirements of the Business Owner . . . 35
4.2.2 Functional Requirements of the Store Manager . . . 35
4.2.3 Functional Requirements of the System . . . 36
4.3 Quality Attributes . . . 36
4.3.1 Reliability . . . 37
4.3.2 Maintainability . . . 37
4.3.3 Usability . . . 37
4.3.4 Correctness . . . 38
4.3.5 Efficiency . . . 38
4.3.6 Flexibility . . . 38
4.4 Non-Functional Requirements . . . 39
4.4.1 Performance . . . 39
4.4.2 Reliability . . . 39
4.4.3 Security . . . 39
4.4.4 Compatibility . . . 40
4.5 Assumptions . . . 40
4.6 Use Cases . . . 40
4.7 Hardware and Software Requirements . . . 46
4.7.1 Hardware Requirements . . . 46
4.7.2 Software Requirements . . . 46
4.8 Graphical User Interface . . . 46
4.8.1 Login Page . . . 47
4.8.2 Signup Page . . . 47
4.8.3 Dashboard . . . 48
4.8.4 POS Page . . . 48
4.8.5 Upload Sales Report . . . 48
4.8.6 Inventory . . . 48
4.8.7 Recommendations . . . 51
4.9 Database Design . . . 51
4.9.1 ER Diagram . . . 51
4.9.2 Data Dictionary . . . 52
4.10 Risk Analysis . . . 54
4.11 Conclusion . . . 54

5 Proposed Approach and Methodology **55**

5.1 Overall Pipeline . . . 55
5.2 Data Ingestion . . . 55
5.3 Data Validation, Cleaning, and Normalization . . . 56
5.4 Demand Series Construction . . . 56
5.5 Feature Engineering . . . 57
5.6 Demand Forecasting Model . . . 57
5.6.1 Global Gradient-Boosted Model . . . 58
5.6.2 Loss Function and Forecast Horizon . . . 58
5.6.3 Training and Tuning . . . 58
5.6.4 Statistical Baselines . . . 59
5.6.5 Initial Training Period . . . 59
5.6.6 Forecast Uncertainty . . . 59
5.7 Recommendation Engine and Stock Redistribution . . . 60
5.8 Markdown Optimization . . . 61
5.9 Explainable Recommendations . . . 61
5.10 Forecast and Recommendation Evaluation . . . 62
5.10.1 Data . . . 62
5.10.2 Protocol . . . 62
5.10.3 Metrics . . . 63
5.10.4 Experiments . . . 63
5.10.5 Recommendation Backtest . . . 63
5.11 Iterative Development . . . 64
5.12 Conclusion . . . 64

6 High-Level and Low-Level Design **65**

6.1 System Overview . . . 65
6.1.1 Tenant and User Management . . . 65
6.1.2 POS and Inventory Management . . . 65
6.1.3 Data Ingestion and Cleaning . . . 66
6.1.4 Demand Forecasting . . . 66
6.1.5 Stock Redistribution and Reorder Recommendations . . . 66
6.1.6 Markdown Optimization . . . 67
6.1.7 Explainable Recommendations . . . 67
6.1.8 Dashboard and Reporting . . . 67

6.2 Design Considerations . . . 67
6.2.1 Assumptions and Dependencies . . . 68
6.2.2 General Constraints . . . 69
6.2.3 Goals and Guidelines . . . 71
6.2.4 Development Methods . . . 72
6.3 System Architecture . . . 73
6.3.1 Subsystem Architecture . . . 74
6.4 Architectural Strategies . . . 74
6.4.1 Multi-Tenant Architecture . . . 74
6.4.2 Modular Architecture . . . 74
6.4.3 Programming Languages and Frameworks . . . 75
6.4.4 Database . . . 75
6.4.5 Background Processing . . . 75
6.4.6 Error Detection and Recovery . . . 76
6.5 Domain Model/Class Diagram . . . 76
6.6 Sequence Diagrams . . . 77
6.7 Policies and Tactics . . . 78
6.7.1 Tools to be Used . . . 78
6.7.2 Coding Guidelines and Conventions . . . 86
6.7.3 User Interface . . . 86
6.7.4 Extensibility . . . 86
6.7.5 Tenant Data Isolation and Security . . . 87
6.7.6 Data Validation . . . 87
6.7.7 Explainable Recommendations . . . 87
6.7.8 System Testing . . . 88
6.7.9 Software Maintenance . . . 88
6.7.10 Algorithms . . . 88
6.8 Conclusion . . . 89

7 Implementation and Test Cases 90

7.1 Implementation . . . 90
7.1.1 Development Environment . . . 90
7.1.2 Tenant and User Management . . . 91
7.1.3 Store, Product and Inventory Management . . . 91
7.1.4 POS Module . . . 92

7.1.5 Data Ingestion and Cleaning . . . 92
7.1.6 Demand Forecasting . . . 92
7.1.7 Recommendation Engine . . . 93
7.1.8 Markdown Optimizer . . . 93
7.1.9 Explainable Recommendations . . . 93
7.1.10 Dashboard, Alerts and Subscription . . . 94
7.2 Test Case Design and Description . . . 94
7.3 Test Metrics . . . 102
7.4 Conclusion . . . 102

9 Experimental Results and Discussion 103
9.1 Objectives of the Experiments . . . 103
9.2 Experimental Setup . . . 103
9.2.1 Datasets . . . 103
9.2.2 Data Preparation . . . 104
9.2.3 Evaluation Protocol . . . 104
9.3 Exploratory Analysis of the Demand Data . . . 104
9.4 Experiment 1 – LightGBM Against Statistical Baselines . . . 105
9.5 Experiment 2 – Global Model Against Separate Models . . . 105
9.6 Experiment 3 – Amount of Training History . . . 106
9.7 Experiment 4 – Loss Function Comparison . . . 106
9.8 Experiment 5 – Recommendation Backtest . . . 107
9.9 Discussion . . . 107
9.9.1 Threats to Validity . . . 108
9.10 Conclusion . . . 108

10 Conclusion and Future Work 109
10.1 Conclusion . . . 109
10.2 Future Work . . . 110

List of Figures

4.1 Login Page . . . 47
4.2 Signup Page . . . 48
4.3 Dashboard . . . 49
4.4 POS Page . . . 49
4.5 Upload Sales Report . . . 50
4.6 Inventory . . . 50
4.7 Recommendations . . . 51
4.8 ER Diagram . . . 52

6.1 System Architecture Diagram . . . 73
6.2 Domain Model / Class Diagram . . . 77
6.3 Sequence Diagram of UC-01 Register Business . . . 78
6.4 Sequence Diagram of UC-02 Login . . . 79
6.5 Sequence Diagram of UC-03 Manage Stores . . . 79
6.6 Sequence Diagram of UC-04 Manage Store Managers . . . 80
6.7 Sequence Diagram of UC-05 Manage Products and Variants . . . 80
6.8 Sequence Diagram of UC-06 Adjust Inventory . . . 81
6.9 Sequence Diagram of UC-07 Record Sale (POS) . . . 81
6.10 Sequence Diagram of UC-08 Upload Historical Sales Data . . . 82
6.11 Sequence Diagram of UC-09 View Dashboard and Alerts . . . 82
6.12 Sequence Diagram of UC-10 View Demand Forecast . . . 83
6.13 Sequence Diagram of UC-11 View and Act on Redistribution Recommendation . . . 83
6.14 Sequence Diagram of UC-12 View Markdown Recommendation . . . 84
6.15 Sequence Diagram of UC-13 View Reorder Recommendation . . . 84
6.16 Sequence Diagram of UC-14 Manage Subscription . . . 85
6.17 Sequence Diagram of UC-15 Manage Tenants . . . 85

List of Tables

2.1 Stakeholders of StockSense 9

3.1 Definitions of Terms Used in the Literature Review 10
3.2 Summary of Literature Review 26
3.3 Summary of Related Work 32

4.1 Functional Requirements of the Business Owner 35
4.2 Functional Requirements of the Store Manager 36
4.3 Functional Requirements of the System 36
4.4 UC-01 Register Business 41
4.5 UC-02 Login 41
4.6 UC-03 Manage Stores 41
4.7 UC-04 Manage Store Managers 42
4.8 UC-05 Manage Products and Variants 42
4.9 UC-06 Adjust Inventory 42
4.10 UC-07 Record Sale (POS) 43
4.11 UC-08 Upload Historical Sales Data 43
4.12 UC-09 View Dashboard and Alerts 43
4.13 UC-10 View Demand Forecast 44
4.14 UC-11 View and Act on Redistribution Recommendation 44
4.15 UC-12 View Markdown Recommendation 44
4.16 UC-13 View Reorder Recommendation 45
4.17 UC-14 Manage Subscription 45
4.18 UC-15 Manage Tenants 45
4.19 Data Dictionary for Tenant Table 53
4.20 Data Dictionary for Store Table 53
4.21 Data Dictionary for Product Variant Table 53
4.22 Data Dictionary for Sale Table 53
4.23 Data Dictionary for Recommendation Table 53

7.1 Implementation Status of StockSense Modules . . . 91
7.2 Test Case 1 (Register Business) . . . 95
7.3 Test Case 2 (Login) . . . 95
7.4 Test Case 3 (Manage Stores) . . . 95
7.5 Test Case 4 (Manage Store Managers) . . . 96
7.6 Test Case 5 (Manage Products and Variants) . . . 96
7.7 Test Case 6 (Adjust Inventory) . . . 96
7.8 Test Case 7 (Record Sale (POS)) . . . 97
7.9 Test Case 8 (Upload Historical Sales Data) . . . 97
7.10 Test Case 9 (Upload of an Unsupported or Broken File) . . . 97
7.11 Test Case 10 (View Dashboard and Alerts) . . . 98
7.12 Test Case 11 (View Demand Forecast) . . . 98
7.13 Test Case 12 (Redistribution Recommendation) . . . 99
7.14 Test Case 13 (Markdown Recommendation) . . . 99
7.15 Test Case 14 (Reorder Recommendation) . . . 99
7.16 Test Case 15 (Manage Subscription) . . . 100
7.17 Test Case 16 (Manage Tenants) . . . 100
7.18 Test Case 17 (Tenant Data Isolation) . . . 100
7.19 Test Case 18 (POS Speed) . . . 101
7.20 Test Case 19 (Large File Upload) . . . 101
7.21 Test Case 20 (Password and Connection Security) . . . 101
7.22 Functional Test Case Metrics . . . 102
7.23 Non-Functional Test Case Metrics . . . 102

9.1 Distribution of Demand Classes . . . 105
9.2 Forecast Accuracy of LightGBM and Baselines (Weekly) . . . 105
9.3 Global Model Against Separate Models . . . 106
9.4 Forecast Accuracy Against Training History . . . 106
9.5 Comparison of LightGBM Objective Functions . . . 107
9.6 Recommendation Backtest Results . . . 107

Chapter 1 Introduction

Retailers need the right products in the right quantities at the right locations, but small and medium-sized businesses often rely on spreadsheets, basic POS systems or manual records and rarely analyze their sales data in a systematic way. This leads to stockouts and dead stock, especially when demand for the same product differs between sizes and between branches. Large retailers use enterprise forecasting and inventory optimization tools for this, but those tools are expensive and difficult to operate without specialist staff. StockSense is proposed as a multi-tenant SaaS solution that makes data-driven inventory management accessible to smaller businesses. It combines a free POS module with demand forecasting, stock redistribution between branches, markdown optimization and recommendations that explain their own reasoning. Because it works on sales data that retailers already collect, it can show inventory risks early and let the owner act before products turn into unsold stock or run out on the shelf. This chapter states the purpose of the document, its intended audience and the terms used throughout the report.

1.1 Purpose of this Document

This document describes the StockSense Final Year Project (FYP), which aims to develop a multi-tenant SaaS platform for retail inventory management and demand forecasting. The platform gives small and medium-sized retailers an easy way to turn POS records and historical sales data into practical inventory decisions, such as what to reorder, which stock to move between branches and which items to discount before they stop selling. The document records the work completed in the first phase of the project and explains the reasoning behind each design decision so that the evaluation panel can follow how the requirements lead to the proposed design. It covers the problem domain, objectives, scope, related research, requirements, methodology, system design, implementation plan, planned experiments and future work. Topics such as demand forecasting, inventory redistribution, markdown recommendations and explainable decision support are described in enough detail to guide the implementation in the second phase of the project.

1.2 Intended Audience

The main readers of this report are the FYP evaluation panel and the project supervisor, who will assess the problem definition, the proposed solution and the quality of the design. The report is also written for software developers who will continue the implementation, since it describes the requirements, data model, architecture and test cases in enough detail to build and verify the system. Researchers interested in demand forecasting and inventory optimization for small businesses may find the literature review

and the forecasting methodology useful, as they summarize current methods and explain how they are adapted to retailers with little historical data. Finally, the report addresses business stakeholders and retailers who want to understand how data-driven tools can help smaller stores plan their inventory, reduce the risk of excess stock and avoid stockouts. No background in data science is assumed, and technical terms are explained when they are first used or listed in the next section.

1.3 Definitions, Acronyms, and Abbreviations

The definitions, acronyms, and abbreviations used throughout this report are listed below. Each acronym is written in full the first time it appears in the text and in short form after that.

ADI: Average Inter-Demand Interval

AI: Artificial Intelligence

API: Application Programming Interface

CRUD: Create, Read, Update, Delete

CSV: Comma-Separated Values

ERP: Enterprise Resource Planning

FYP: Final Year Project

HTTPS: Hypertext Transfer Protocol Secure

MAE: Mean Absolute Error

MAPE: Mean Absolute Percentage Error

ML: Machine Learning

MVP: Minimum Viable Product

POS: Point of Sale

REST: Representational State Transfer

RMSE: Root Mean Squared Error

SaaS: Software as a Service

SDG: Sustainable Development Goal

SKU: Stock Keeping Unit

TLS: Transport Layer Security

UI: User Interface

UX: User Experience

WAPE: Weighted Absolute Percentage Error

1.4 Conclusion

StockSense is intended to help owners of small and medium-sized stores make better inventory decisions by combining day-to-day sales management with forecasting and practical recommendations. The platform focuses on two problems, stockouts and dead stock, which usually appear at the level of individual variants and store locations rather than whole products. It gives priority to moving existing stock between branches when this is possible and identifies slow-moving stock early enough to take preventive action. The project combines a free POS layer with subscription-based forecasting and inventory optimization so that businesses can use their existing sales data without adopting a complex enterprise system. Through forecasting, redistribution recommendations, markdown optimization and clear explanations, StockSense aims to bring inventory analytics within reach of smaller retailers. The following chapters present the project vision and requirements in detail, followed by the related work, the methodology, the system design, the implementation plan and the planned experiments.

Chapter 2 Project Vision

StockSense aims to give small and medium-sized retailers a low-cost way to understand their sales patterns and make data-driven decisions about stock. It targets any retailer that sells physical products in several variants or sizes, or through more than one store, such as clothing, footwear and similar goods, where demand changes a great deal between products, sizes and locations. In such businesses a single product can sell out in one size at one branch while the same product in another size, or at another branch, does not sell at all. The vision is a platform that a small business can start using for free as its everyday sales system and that, once enough sales history is available, turns this history into clear advice on what to reorder, what to move between branches and what to discount early. This chapter describes the problem domain, the problem statement, the goals and scope of the project, the related sustainable development goals, the constraints, the business opportunity and the stakeholders of the system.

2.1 Problem Domain Overview

Retail inventory management is the process of deciding which products to offer, in what quantities and at which locations. In small businesses these decisions are usually based on experience and intuition rather than on formal forecasting, so mistakes are noticed only when shelves are empty or when unsold stock has piled up. StockSense addresses this by collecting historical sales data and using it to forecast demand for each product, size and location, and then turning these forecasts into recommendations such as reordering stock, moving stock from one branch to another or discounting stock that is close to becoming unsellable. The system is designed as a multi-tenant SaaS platform in which each business is a separate tenant with its own products, stores, inventory, sales, forecasts and recommendations, and no tenant can see the data of another. Businesses that already use an external POS or keep their sales in spreadsheets can upload historical sales reports as CSV or Excel files, while other businesses can record their sales directly through the built-in POS module. In both cases the same data feeds the forecasting and recommendation features.

2.2 Problem Statement

Retailers find it difficult to turn past sales information into inventory decisions at the level of the individual product, size and location. Without this detail, owners order the same quantities for every branch and every size, which results in overstocking, dead stock, capital tied up in slow-moving items and lost sales when popular items run out. Existing enterprise solutions that solve this problem are priced and designed for large chains, while affordable tools for small businesses stop at simple stock tracking and

low-stock alerts. An integrated and affordable solution is therefore needed that records day-to-day sales, forecasts demand at a detailed level and recommends concrete inventory actions with a clear explanation of each recommendation, so that small retailers can manage their stock based on data rather than guesswork. The problem is made worse by the fact that small retailers rarely have the time or skills to build such an analysis themselves, so even businesses that keep good records do not use them to plan their stock.

2.3 Problem Elaboration

The problem stated above has several parts, and each of them shapes a requirement of the system. Many retailers simply do not have a tool that can analyze their previous sales, and the tools that can do so are designed for organizations with dedicated planning teams. Even when a forecast is available, it is often made at the product level, which hides the large differences in demand between stores and between sizes of the same product. Because each branch is managed separately, one store may hold excess units of a product that another branch urgently needs. Slow-moving items are usually noticed only after they have already turned into dead stock, when the only remaining option is a heavy discount. Finally, owners are unlikely to follow recommendations produced by a machine learning (ML) model if the reasoning behind them is hidden. The main problems that StockSense addresses are summarized below.

- Retailers lack the tools to analyze their previous sales data.
- Existing enterprise demand forecasting solutions are expensive and technically complex for small businesses.
- Demand differs between stores, so aggregate product-level forecasts are not enough.
- Demand also differs significantly between product sizes and variants.
- One store may have excess inventory while another branch needs the same product.
- Slow-moving inventory is often identified only after it has already become dead stock.
- Business owners find ML-based recommendations difficult to trust when the reasoning is not explained.

2.4 Goals and Objectives

The overall goal of StockSense is to give small retailers the inventory planning abilities of a large chain at a cost and level of complexity they can manage. To reach this goal, the project has to deliver a secure multi-tenant platform, a reliable way of bringing sales data into it, a forecasting model that works with

limited history, a recommendation engine that prefers redistribution over new orders, and an evaluation that shows whether the recommendations actually reduce dead stock. Every recommendation must also be explained so that it can be trusted. The business model is part of the objectives as well, because the free POS layer is what brings the sales data onto the platform in the first place. The specific objectives of the project are listed below. The objectives are also chosen so that each of them can be checked at the end of the project, either by a working feature or by a measured result.

- Design a multi-tenant platform that allows multiple small businesses to securely manage their sales and inventory data.
- Develop a data ingestion and cleaning pipeline for CSV and Excel sales reports that normalizes the data into a schema of product, size, store, date and quantity sold.
- Develop a demand forecasting model for product-size-location combinations.
- Develop a recommendation engine that determines what to reorder, how much to reorder and where existing inventory should be redistributed.
- Develop a markdown optimizer for slow-moving stock.
- Provide an explanation and an estimated financial impact for every recommendation.
- Evaluate forecast accuracy with standard error metrics and estimate the potential reduction in dead stock through historical simulation.
- Offer the platform under a freemium model with a free POS layer and paid forecasting and recommendation features.

2.5 Project Scope

StockSense targets businesses that sell clothing, footwear or any other physical products with variants and sizes, possibly through several stores, and that suffer from stockouts and dead stock. Several businesses can use the platform as separate tenants, and each tenant's stores, products, inventory and sales history are managed independently. Within this scope the system will provide multi-tenant business management, management of stores, products and variants, recording of sales and inventory through the POS, upload of historical data from CSV and Excel files, data cleaning and normalization, demand forecasting at the product, size and location level, redistribution and reorder recommendations, markdown recommendations, explanations with an estimated financial impact for every recommendation, and dashboard visualization. Forecasting will focus on weekly and monthly demand rather than long-term trends, because these horizons match the reordering and transfer decisions that small retailers make.

Fully automatic integration with external POS systems is outside the initial scope, so manual and batch upload of CSV and Excel files will serve as the main way of importing data from other systems.

2.6 Sustainable Development Goal (SDG)

StockSense contributes to two of the United Nations SDGs. Both contributions come from the same idea, which is that better use of existing sales data lets small businesses grow without wasting money and resources on stock that does not sell.

2.6.1 SDG-8 (Decent Work and Economic Growth)

Small enterprises are one of the main sources of jobs, yet they often operate on thin margins and cannot afford the analytics tools that help large retailers grow. Demand forecasting and inventory optimization have so far been mostly available to large chains with enterprise systems and data teams. StockSense makes these capabilities available to small retailers through a free POS and an affordable subscription, which helps them sell more of what customers want, reduce losses on unsold stock and invest their capital more productively. A shop that loses fewer sales to stockouts and spends less on discounts and write-offs is in a better position to keep its staff, open new branches and hire more people. In this way the project supports productive employment and sustained economic growth in the small business sector, which is the focus of SDG-8.

2.6.2 SDG-9 (Industry, Innovation, and Infrastructure)

StockSense brings data-driven innovation to a part of the retail industry that has had little access to it. By turning ordinary sales records into forecasts and recommendations, it helps retailers use their existing capital, store space and stock more efficiently. Moving unsold items to branches where they sell, and discounting slow movers before they become worthless, reduces the build-up of excess inventory that would otherwise be thrown away or sold at a loss. This means fewer goods are produced and transported only to remain unsold, which supports a more responsible use of resources. The cloud-based, multi-tenant design also lets many small businesses share the same infrastructure at low cost instead of each needing its own servers and specialist staff, which reflects the aim of SDG-9 to build resilient infrastructure and promote inclusive and sustainable industrialization.

2.7 Constraints

Several constraints affect the design and evaluation of StockSense. The most important is the limited availability and quality of historical sales data, since small retailers often have incomplete or inconsistent

records. A new business also has to record or upload enough sales history before reliable forecasts can be produced, and the accuracy of the forecasts will always depend on the quality and quantity of this data. Public retail datasets used for development may not represent the behaviour of small retailers accurately, so results obtained on them have to be interpreted with care. The system initially focuses on physical products such as clothing and footwear and on short- to medium-term forecasts, not on long-term planning. Finally, redistribution is not always cost-effective, because moving stock between branches involves transport and handling costs and transfer delays, and it creates a risk of stockouts at the branch that gives the stock away. The recommendation engine therefore has to compare these costs with the expected benefit before suggesting a transfer.

2.8 Business Opportunity

StockSense follows a freemium SaaS business model. The basic POS features are free, so small businesses can track their inventory and sales without paying a subscription fee, which removes the main barrier that stops small shops from adopting new software. A paid subscription unlocks advanced features such as demand forecasting, inventory redistribution, markdown optimization, benchmarking and recommendations with an estimated financial impact. Because the free layer already holds a business's sales history, moving to the paid tier requires no extra setup, and the value of the paid features can be shown with the business's own data. This model lets companies start using the platform with almost no upfront investment while giving the growing stores a smooth path to advanced analytics. It also gives StockSense a large base of users from which paying customers can be drawn, and the revenue from subscriptions covers the cost of running the forecasting infrastructure for all tenants.

2.9 Stakeholders Description / User Characteristics

StockSense has three main types of users, and their needs differ in scope rather than in kind. Business owners look at the performance of the whole business, including inventory, forecasts and recommendations across all branches, and they decide on subscriptions and store managers. Store managers look after a single branch, record sales through the POS and follow the sales, forecasts, stock movements and recommendations that concern their store. The system administrator manages the platform as a whole, including tenants, subscriptions and system configuration, but does not take part in the business decisions of any tenant. Owners and managers are assumed to have basic computer skills but no training in data analysis, so the interface must present forecasts and recommendations in plain business terms. Table 2.1 summarizes the responsibilities of each stakeholder. Each type of user therefore sees a different view of the same data, and the system controls access to that data according to the user's role.

Table 2.1: Stakeholders of StockSense

Stakeholder	Responsibilities
Business Owner	Monitor business performance, inventory, forecasts, and recommendations.
Store Manager	Monitor branch-level sales, inventory, forecasts, and stock movement.
System Administrator	Manage tenants, platform configuration, and system-level operations.

2.9.1 Key High-Level Goals and Problems of Stakeholders

Each stakeholder group needs something different from the system, but all of them share the same underlying problem. Business owners need a simple way to turn the sales data they already have into meaningful inventory decisions, such as how much to order for the next season and which branch should receive new stock. Store managers need to see what is selling in their own store, which items are at risk of running out and which items are not moving, so that they can act on transfers and markdowns before it is too late. The system administrator needs reliable ways to keep tenants separate, manage subscriptions and keep the platform running for all businesses. The common problem is that no simple and affordable system currently combines sales recording, forecasting and inventory recommendations in one place, so each of these users has to rely on separate tools, spreadsheets or intuition. StockSense brings these functions together in a single platform built around their needs.

2.10 Conclusion

This chapter explained the problem domain, the problem statement, the goals, the scope and the stakeholders of StockSense. The platform is meant to help small and medium-sized businesses use their existing sales data to make better inventory decisions without the cost and complexity of enterprise solutions. Its main ideas are forecasting at the level of individual variants and locations, giving priority to redistribution between branches, acting early on slow-moving stock and explaining every recommendation. The chapter also described how the project supports SDG-8 and SDG-9, the constraints that affect the design, such as limited sales history and the cost of transfers, and the freemium business model that brings sales data onto the platform through a free POS. These elements form the basis of the requirements presented in the following chapters. The next chapter reviews the research literature and the existing commercial products related to this project and identifies the gap that StockSense is intended to fill.

Chapter 3 Literature Review / Related Work

This chapter reviews the academic research and the commercial products that are related to StockSense. The literature review covers four groups of work that together define the technical basis of the project. The first group deals with demand forecasting for retail and fashion products, including the use of ML models and of global models trained on many related time series. The second group deals with inventory decisions, in particular redistribution of stock between locations and markdown pricing of slow-moving items. The third group covers the step from prediction to decision, that is, how forecasts can be turned into actions that a business can take. The fourth group concerns the explainability of model outputs and the practical cost of stockouts. After the research papers, the chapter reviews six commercial platforms that solve similar problems and compares them with StockSense. The chapter ends by identifying the gap that StockSense fills and by stating the modeling approach selected for the project on the basis of this review.

3.1 Definitions

The terms in Table 3.1 are used throughout the literature review and the rest of the report. The acronyms used in this chapter are listed in Section 1.3 in Chapter 1.

Table 3.1: Definitions of Terms Used in the Literature Review

Term	Definition
Demand forecasting	The process of estimating future customer demand for a product using historical sales data and other relevant information.
Lateral transshipment	The movement of inventory between locations at the same level of a supply chain, such as between two retail branches, rather than from a central warehouse.
Markdown optimization	The process of choosing the size and timing of a price discount so that slow-moving inventory is sold before it becomes unsellable.
Stock keeping unit (SKU)	A unique identifier for a specific product variant, for example a particular size and colour of an item.
Global forecasting model	A single model trained on many related time series at once, as opposed to a separate local model for each series.

3.2 Detailed Literature Review

The papers reviewed in this section were selected because each one informs a specific part of the StockSense design. Research on fashion retail shows that forecasting demand for new items is especially difficult because they have little or no sales history, and that product attributes can partly make up for the missing history [1]. Work on online retail has also shown that demand forecasting combined with

price optimization can increase revenue in practice [2], which supports the idea of linking forecasts directly to pricing decisions such as markdowns. The following subsections review fifteen papers in detail. For each paper, a summary is followed by a critical analysis and by an explanation of how the paper relates to StockSense, in the same format as the project template. Together the fifteen papers cover forecasting methods, inventory and pricing decisions, the link between prediction and action, and the trust that users place in model outputs.

3.2.1 Exploring the Use of Deep Neural Networks for Sales Forecasting in Fashion Retail

3.2.1.1 Summary of the Research Item

Loureiro, Miguéis and da Silva [3] study how well deep neural networks forecast the sales of new fashion products compared with shallower methods such as decision trees, random forests, support vector regression and conventional neural networks. The study uses a real dataset from a fashion retailer, in which each product is described by attributes related to its physical characteristics and its distribution, together with predictive variables defined by the retailer's domain experts. Because new products have no sales history of their own, the models learn the relationship between these attributes and the sales observed for earlier products. The authors find that the deep learning model gives good forecasting performance and compares favourably with the other methods, although some simpler ensemble methods such as random forests also perform well. The paper concludes that data-driven models can support the planning of new collections better than relying only on expert judgment.

3.2.1.2 Critical Analysis of the Research Item

The main strength of the paper is that it evaluates the models on the data of a real company rather than on a synthetic dataset, which makes the results relevant to practice. It also shows that combining product attributes with expert knowledge is a practical way to forecast demand when no history is available. However, the study focuses on the total sales of new products for a whole season and does not forecast demand per store or per size. The data comes from a single large company with an established data infrastructure and many products, so the findings are difficult to generalize to small businesses with only a few branches and a short sales history. Deep neural networks also need a large amount of training data and careful tuning, and the paper does not discuss the cost of maintaining such models in a small business setting.

3.2.1.3 Relationship to the Proposed Research Work

The paper supports the choice of a learning-based approach to demand forecasting over manual estimation, which is the starting point of StockSense. Its use of product attributes as predictive variables is

directly relevant, because StockSense also describes each product by its category, size and colour and uses these attributes as features. Since StockSense must forecast demand for each product, size and store combination, the idea of sharing information between related products through their attributes is especially useful for items with short histories. At the same time, the limitations of the paper show why StockSense prefers gradient-boosted trees, which perform well on tabular data with modest data volumes, over deep networks that need more data than a small retailer can provide. The paper also reminds us that expert knowledge remains valuable, which is why StockSense lets owners review and dismiss every recommendation instead of applying it automatically.

3.2.2 Clearance Pricing Optimization for a Fast-Fashion Retailer

3.2.2.1 Summary of the Research Item

Caro and Gallien [4] describe a forecast-and-optimize process that replaced the manual markdown decisions used by the fast-fashion retailer Zara at the end of each season. The process combines a demand forecasting model, which estimates how the sales of each article respond to price changes, with an optimization model that selects the price of each article for the coming weeks of the clearance period. The authors tested the new process in a controlled field experiment across all stores in Belgium and Ireland during a fall-winter clearance season. The experiment showed that the new process increased clearance revenue by approximately six percent compared with the manual process. After this result the company adopted the process across its worldwide network of stores. The process was designed to work within the practical rules of the company, such as a limited set of allowed price levels and the existing schedule of price changes. This shows that a formal model can be introduced into an established retail process without disrupting how the stores operate.

3.2.2.2 Critical Analysis of the Research Item

The paper is a rare example of a markdown optimization model that has been documented, tested in the field and adopted at scale with a measured effect on revenue. The field experiment provides stronger evidence than a simulation, and the paper explains clearly how the model was integrated into the existing decision process of the company. Its main limitation for StockSense is that it was designed for a very large and vertically integrated retailer with detailed historical pricing data for thousands of articles. Most small retailers do not have this pricing history, which makes it harder to estimate how demand responds to discounts. The process is also focused on the clearance period at the end of the season, when the goal is to sell remaining stock, rather than on preventing slow-moving stock from building up during the season.

3.2.2.3 Relationship to the Proposed Research Work

The paper confirms the main idea behind the StockSense markdown optimizer, namely that a formal process combining forecasting and pricing gives better results than manual markdown decisions based on instinct. StockSense follows the same forecast-then-optimize structure, in which demand is first forecast and the smallest discount that reaches a sales target is then selected. The difference is that StockSense applies the idea to small data sets and uses it as an early warning measure during the season rather than as a clearance event at the end of it. Because small retailers have little price history, StockSense uses a simple price elasticity model with a cautious category-level estimate when the tenant's own price data is not varied enough. The result of the Zara experiment also gives a realistic expectation of the size of the benefit, which helps in judging the outcome of the StockSense backtest.

3.2.3 Inventory Models with Lateral Transshipments: A Review

3.2.3.1 Summary of the Research Item

Paterson, Kiesmüller, Teunter and Glazebrook [5] review the academic literature on lateral transshipments, which are stock movements between locations at the same level of a supply chain, such as between retail stores. The review classifies the research according to the timing of the transfers. Reactive transshipments are made in response to a stockout or an expected stockout at one location, while proactive transshipments are planned in advance to rebalance stock between locations before shortages occur. The authors further classify the models by features such as the number of locations, the inventory review policy and the cost structure, and they summarize the main results reported for each class of model. The review closes by pointing out several open research questions in the area. For a retailer, the main lesson of the review is that the timing and the rules of transfers matter as much as the decision to transfer at all. Proactive policies require a forecast of where demand will arise, which links transshipment directly to demand forecasting.

3.2.3.2 Critical Analysis of the Research Item

As a review, the paper gives a broad and well-organized overview of transshipment models and makes it easy to see which combinations of assumptions have been studied and where gaps remain. Its distinction between reactive and proactive transfers is particularly useful for designing a recommendation system. However, the paper does not propose a specific algorithm that suits a small retailer with only two or three branches, and most of the models it reviews assume detailed knowledge of demand distributions and costs that small businesses do not have. The review also does not address how transfer decisions should be combined with other actions such as markdowns or new orders, which is an important practical

question when stock can be handled in several different ways. These gaps are understandable for a review of theoretical models, but they mean that a practical system has to combine the ideas of the review with simpler and more robust decision rules.

3.2.3.3 Relationship to the Proposed Research Work

This review provides the academic basis for treating transfers between branches as a separate decision from reordering or discounting. StockSense builds on the proactive framing described in the paper, because its redistribution recommendations are meant to prevent dead stock and stockouts before they happen rather than react to an empty shelf. The recommendation engine formulates each transfer decision as a small transportation problem that moves surplus units to branches with a forecast shortage only when the expected margin exceeds the transfer cost. StockSense also addresses the gap identified above by placing redistribution in a fixed order of priority, ahead of markdowns and reorders, so that existing stock is used before money is spent on discounts or new purchases. In this way the review shapes both the formulation and the position of redistribution in the overall recommendation process.

3.2.4 Forecasting Across Time Series Databases Using Recurrent Neural Networks on Groups of Similar Series

3.2.4.1 Summary of the Research Item

Bandara, Bergmeir and Smyl [6] propose a forecasting method for large collections of related time series. Instead of fitting one model to each series, they first group the series into clusters of similar series and then train a single recurrent neural network on all the series in each cluster. The idea is that series with similar patterns can share information, so that a short series benefits from the history of other series in the same cluster. The authors evaluate the method on benchmark datasets from forecasting competitions and show that training on clusters of similar series improves accuracy compared with training one model on all series together and compared with several standard univariate methods. The method thus combines two ideas, grouping similar series and training a shared model, which have since become common in large-scale forecasting. The authors also discuss how the grouping can be based on features of the series, such as their trend and seasonality.

3.2.4.2 Critical Analysis of the Research Item

The approach is well suited to situations with many short or sparse series, which is typical of retail sales at a detailed level, and the paper shows clearly that sharing information between series can improve accuracy. The clustering step also gives a systematic way to deal with very different groups of series in the same dataset. The main limitation is that the method was tested mostly on benchmark time series

from competitions rather than on retail sales at the product, size and store level, which contain many zero values and are affected by prices and promotions. Recurrent neural networks also need careful tuning and considerable training data, which may be difficult to provide for each tenant of a platform that serves small businesses. The extra clustering step also adds a design choice, namely the number of clusters, that has to be tuned for each new dataset.

3.2.4.3 Relationship to the Proposed Research Work

The paper supports the StockSense strategy of training one shared forecasting model on many product, size and store series instead of building a separate model for every combination. A small retailer has too few observations for each individual series to train a reliable local model, so sharing information across series is necessary. StockSense achieves a similar effect to clustering in a simpler way, by giving a gradient-boosted model the product, size, store and category as features, so that the model can learn which series behave alike. This keeps the advantages of a global model while avoiding the heavier training requirements of recurrent neural networks. If the experiments in Chapter 9 show that some groups of series behave very differently, the clustering idea of the paper could be added later by training separate global models for each group. The paper therefore provides both the justification for the current design and a possible direction for improving it.

3.2.5 M5 Accuracy Competition: Results, Findings, and Conclusions

3.2.5.1 Summary of the Research Item

Makridakis, Spiliotis and Assimakopoulos [7] report the results of the M5 forecasting competition, in which thousands of participants forecast the unit sales of Walmart products. The dataset contained 42,840 hierarchically organized series covering products, departments, categories, stores and states, together with prices, promotions and calendar information. The competition found that ML methods, and especially gradient-boosted trees such as LightGBM, were more accurate than the statistical benchmarks for this kind of data. Most of the winning solutions trained global models on many series at once and used external information such as prices and calendar events. The authors conclude that cross-learning between series and the use of explanatory variables were key reasons for the improved accuracy. The competition also included an uncertainty track, in which participants forecast quantiles of demand, which showed the value of measuring the spread of possible outcomes and not only their average.

3.2.5.2 Critical Analysis of the Research Item

The M5 competition is one of the largest empirical comparisons of forecasting methods on real retail data, so its conclusions are widely used to judge which model families work well for retail sales. Its

strength lies in the size and realism of the dataset and in the large number of independent participants, which makes the findings robust. However, the data comes from a very large retailer with years of sales history for thousands of products, and the competition does not consider the situation of small retailers with short histories. It also does not include sales by size or variant, which is central to StockSense. The winning solutions were often complex ensembles tuned for the competition, so the findings indicate a model family rather than a ready solution for a small business platform.

3.2.5.3 Relationship to the Proposed Research Work

The finding that gradient-boosted trees outperform statistical methods on hierarchical retail sales data directly shaped the choice of LightGBM as the main forecasting model of StockSense. The competition also supports two further design decisions, namely training a single global model on all series of a tenant and including price and calendar features. At the same time, its limitations explain why StockSense keeps simple statistical methods as baselines and plans experiments with short histories, since the advantage of gradient boosting on data from a large retailer must be confirmed on the smaller and sparser data of small businesses. The M5 dataset itself is also used in the preliminary experiments planned in Chapter 9. The uncertainty results of the competition also support the use of an upper quantile forecast to set safety stock in the recommendation engine. In this way the M5 findings influence the model, the features and the treatment of uncertainty in StockSense.

3.2.6 Demand Forecasting for Improved Inventory Management in Small and Medium-Sized Businesses

3.2.6.1 Summary of the Research Item

Purnamasari, Permadi, Saepudin and Agusdin [8] study demand forecasting for small leathercraft businesses. They first group products by similarity using K-means clustering and then compare ML approaches, including an ensemble gradient boosting model, with classical statistical projection methods for forecasting demand. The study uses the sales data of a small business and evaluates the methods with standard error measures. The authors report that the ML methods, and the gradient boosting ensemble in particular, give more accurate forecasts than the classical approaches. They conclude that such methods can help small businesses plan their inventory better and avoid both shortages and excess stock. The clustering step allows products with similar demand to share information, which is useful when each product alone has only a short history. The study therefore shows that the combination of grouping and gradient boosting is practical even for a business with very limited resources.

3.2.6.2 Critical Analysis of the Research Item

This is one of the few studies that targets demand forecasting for small businesses directly rather than for large retail chains, which makes its findings especially relevant to this project. The combination of clustering and gradient boosting is practical and easy to apply with common tools, and the paper shows that small businesses can benefit from these methods. Its limitations are that it uses a single small-business dataset from one product category, which limits how far the results can be generalized, and that it does not address demand at the level of sizes or store locations. The study also does not discuss how the forecasts would be turned into inventory decisions or how they would be presented to business owners without technical knowledge. These limitations do not reduce the value of the study, but they show which parts of the problem remain open for a platform that serves many different small businesses.

3.2.6.3 Relationship to the Proposed Research Work

Of all the reviewed papers, this one is closest to StockSense in terms of its target users, which are small businesses without a data science team. Its finding that gradient boosting outperforms classical statistical approaches on small-business sales data supports the StockSense choice of a gradient-boosted model, with statistical methods kept as baselines for comparison. StockSense extends the idea in the directions that the paper leaves open, by forecasting at the product, size and location level, by turning the forecasts into redistribution, markdown and reorder recommendations, and by explaining each recommendation in plain language so that owners can act on it with confidence. The paper also shows that a small business can benefit from ML without employing data specialists, provided that the method is packaged in a simple and reliable tool, which is exactly the role that StockSense aims to play.

3.2.7 Retail Forecasting: Research and Practice

3.2.7.1 Summary of the Research Item

Fildes, Ma and Kolassa [9] present an extensive review of academic research on retail forecasting and compare it with the needs of retailers in practice. They organize the review by the level of the decision being supported, from strategic decisions such as store locations, to tactical decisions such as promotions and assortment, to operational decisions such as store replenishment. For each level they summarize the data, methods and evaluation approaches used in the literature. The authors identify several gaps between research and practice, for example in forecasting for new products, in modeling the effects of promotions and in the evaluation of forecasts in terms of their effect on business decisions rather than only on error measures. The review also discusses the data available to retailers, such as POS records and loyalty data, and explains how its quality and level of detail limit what can be forecast.

3.2.7.2 Critical Analysis of the Research Item

As a broad survey, the paper serves as a map of the retail forecasting research landscape and is useful for placing any new project within it. It is particularly valuable for its discussion of how forecasts are actually used in retail organizations and of the practical problems that limit their adoption. Since the paper is a review, it does not propose or test a new model, so it cannot by itself guide the choice of a specific algorithm. Most of the practice it describes also comes from medium-sized and large retailers, and it says relatively little about very small businesses that have no forecasting process at all. The review is nevertheless one of the most complete and recent treatments of the subject, and it is widely used as a starting point for research on retail forecasting. Its organization by decision level also helps to show that StockSense works mainly at the operational level of replenishment and transfers.

3.2.7.3 Relationship to the Proposed Research Work

The review shows that most published retail forecasting research and most practical tools assume a data science team and an established IT infrastructure, which is consistent with the main premise of this project. StockSense aims to close this gap for small retailers by providing forecasting as part of a simple platform that they already use for recording sales. The paper's emphasis on evaluating forecasts by their effect on decisions also influenced the StockSense evaluation plan, which includes a historical backtest that measures dead stock and lost sales in addition to the usual error metrics. The review also points out that forecasts for individual stores and items are among the hardest to produce, which confirms that the level of detail chosen for StockSense is challenging. This is one reason why the project keeps simple baselines and tests the minimum amount of history that a tenant needs before forecasts are shown.

3.2.8 Principles and Algorithms for Forecasting Groups of Time Series: Locality and Globality

3.2.8.1 Summary of the Research Item

Montero-Manso and Hyndman [10] study the conditions under which it is better to forecast a group of related time series with one shared global model rather than with a separate local model for each series. They give theoretical arguments showing that global models can be as expressive as local ones and that their complexity can grow with the size of the dataset without overfitting individual series. They support these arguments with experiments on large collections of time series, which show that global models often outperform local models even when there is no obvious similarity between the series. The authors also discuss how the complexity of global models should be chosen in practice. They also show that global models can be extended with features and nonlinear methods, such as tree-based models and

neural networks, without losing these advantages. This makes their results directly applicable to the gradient-boosted models used in retail forecasting.

3.2.8.2 Critical Analysis of the Research Item

The paper offers a sound theoretical and empirical foundation for global forecasting models, and its results are directly useful for justifying model selection in projects with many short series. A key strength is that it explains why global models work instead of only showing that they do. However, the paper is a general time series study rather than a retail-specific one, so its recommendations need to be adapted to the structure of product, size and store sales data, which includes many zeros, price effects and calendar events. The experiments also do not consider the case where the data of different organizations must be kept separate, which limits how far a global model can be shared. Despite these limitations, the paper is one of the clearest statements of why the global approach works, and it has been widely adopted in recent forecasting practice.

3.2.8.3 Relationship to the Proposed Research Work

This result supports the StockSense decision to use one shared forecasting model across all products, sizes and stores of a tenant instead of training a separate model for each series, which would require far more historical data than a small retailer is likely to have. Because tenant data must never be shared between businesses, StockSense applies the global model idea within each tenant rather than across tenants. Whether this per-tenant global model is more accurate than separate local models is one of the experiments planned in Chapter 9. A global model is also cheaper to maintain, since the platform needs to train and store only one model per tenant rather than one for every product, size and store. This keeps the cost of the forecasting service low enough to be offered to small businesses at an affordable price.

3.2.9 Reducing the Cost of Demand Uncertainty Through Accurate Response to Early Sales

3.2.9.1 Summary of the Research Item

Fisher and Raman [11] show that fashion companies can reduce the cost of demand uncertainty by shortening lead times and by reacting to early-season sales instead of committing all production and ordering decisions months in advance. They develop a model that uses early sales data to update demand forecasts and to decide how much of each product to produce once the first sales are known. The approach was applied at a major ski-wear company, and the authors estimate that it could increase profits by roughly sixty percent compared with the company's existing informal response process. The paper shows that much of the uncertainty in fashion demand can be removed by observing a small amount of

early sales. The approach is known as accurate response, and it has influenced much of the later research on quick response in the fashion industry.

3.2.9.2 Critical Analysis of the Research Item

This is a classic paper that clearly shows the value of responding early to demand signals instead of relying on fixed, instinctive stocking decisions. Its strength lies in combining a practical decision model with a real application and a clear estimate of the financial benefit. However, it deals with production and ordering decisions for a single season at a single manufacturer and retailer, which is quite different from the day-to-day inventory decisions of a small multi-branch shop. The model also assumes that production capacity can be reserved and adjusted during the season, which is not relevant for retailers who only buy finished goods from suppliers. The paper is also older than the modern ML methods now used for forecasting, so its forecasting component is simpler than current practice. Its main contribution is therefore the decision principle rather than the forecasting method itself.

3.2.9.3 Relationship to the Proposed Research Work

The paper provides an early foundation for the idea on which StockSense relies, that reacting quickly to demand signals reduces both stockouts and overstocking. It supports treating recent sales velocity as an important signal for both the recommendation engine and the markdown optimizer. In StockSense this idea appears in the lag and rolling features of the forecasting model, which give recent weeks a strong influence on the forecast, and in the markdown optimizer, which flags an item as soon as its weeks of cover exceed the remaining selling time instead of waiting until the end of the season. Both parts of the system therefore act on new information as soon as it appears, rather than waiting for a fixed planning date. This is especially important for small retailers, who cannot afford to hold large safety stocks and must instead respond quickly to the signals in their sales.

3.2.10 Sales Forecasts in Clothing Industry: The Key Success Factor of the Supply Chain Management

3.2.10.1 Summary of the Research Item

Thomassey [12] reviews the forecasting methods used in the clothing industry and discusses the characteristics that make clothing sales hard to forecast. These include short product life cycles, strong seasonality, a very large number of items in different sizes and colours, and the lack of historical data for new items. The paper describes forecasting systems suited to different decision horizons, from long-term planning of collections to short-term replenishment of stores, and explains how methods such as

statistical models and soft computing techniques can be adapted to these conditions. It argues that accurate sales forecasts are a key factor in the success of supply chain management in the clothing sector. The review also stresses that different decisions need different forecasts, since the information needed to plan a collection months in advance differs from the information needed to refill a store next week.

3.2.10.2 Critical Analysis of the Research Item

The paper explicitly addresses data scarcity and seasonality, which make apparel demand harder to forecast than the demand for staple products, and this makes it highly relevant to the problem of StockSense. It provides a useful overview of how the forecasting task changes with the decision horizon. However, the methods discussed in the paper are mostly designed for manufacturers and large distributors that plan whole collections, and the paper predates the recent evidence in favour of gradient-boosted global models. It also does not address how forecasts can be made available to small retailers that lack forecasting expertise. As a result, the paper is most useful as a description of the problem, while the choice of method has to be informed by more recent work such as the M5 competition. It remains one of the most cited reviews of forecasting in the clothing sector.

3.2.10.3 Relationship to the Proposed Research Work

The paper supports the understanding that demand for specific sizes and variants of clothing is more volatile and seasonal than demand at the product level, which is why StockSense forecasts at the level of each product, size and location. Its discussion of different decision horizons also supports the StockSense focus on weekly and monthly forecasts, which match the replenishment, transfer and markdown decisions of small retailers. The seasonality it describes is reflected in the calendar and holiday features included in the forecasting model. The paper's discussion of the lack of history for new items also explains why StockSense uses product attributes such as category, size and colour as features, so that new variants can borrow information from similar ones. In this way the review informs both the level of detail and the features of the forecasting model.

3.2.11 Predictive Big Data Analytics for Supply Chain Demand Forecasting

3.2.11.1 Summary of the Research Item

Seyedan and Mafakheri [13] survey the methods and applications of big data predictive analytics for demand forecasting in supply chains. They classify the methods into groups such as time series forecasting, regression, clustering and ML techniques including neural networks and support vector machines, and they review how these methods have been applied in different parts of the supply chain. The paper also discusses the data sources that are available to supply chains today and identifies open research

questions, including the need for methods that can be applied by organizations with limited analytical capacity. The survey also highlights the growing role of data from sources such as POS systems and online sales, and it discusses the challenges of storing and processing such data at scale. It concludes that predictive analytics can improve supply chain performance when the right methods are matched to the available data.

3.2.11.2 Critical Analysis of the Research Item

As a general survey, the paper gives a good overview of current data-driven forecasting methods and shows clearly which methods are used in which situations. It also highlights that most practical applications come from large, centrally managed supply chains with strong analytical teams. Because it is a survey, the paper does not present or test any particular model, so it cannot guide detailed design decisions on its own. Its focus on big data also means that it gives less attention to the opposite situation, in which each business has only a small amount of data. The survey also does not compare the methods it lists on a common dataset, so it cannot show which of them works best for retail sales. For such comparisons, competition results such as M5 are more informative.

3.2.11.3 Relationship to the Proposed Research Work

The gap identified by Seyedan and Mafakheri between advanced forecasting models and organizations that lack an analytics infrastructure forms a strong basis for the StockSense idea. StockSense addresses this gap by packaging the forecasting pipeline into a hosted platform, so that small retailers can benefit from data-driven forecasting without building their own analytical capacity. The survey also confirms that ML methods are an appropriate choice for demand forecasting when enough data is available, which StockSense ensures by asking each new tenant to provide a minimum amount of sales history before showing model-based forecasts. The survey's emphasis on matching methods to data also supports the StockSense practice of measuring how much history each tenant needs, rather than assuming that the same model suits every business.

3.2.12 Forecasting: Theory and Practice

3.2.12.1 Summary of the Research Item

Petropoulos and a large group of co-authors [14] provide an encyclopedic review of forecasting theory, methods and practice. The review covers statistical and ML forecasting methods, the evaluation of forecasts, the combination of forecasts and the treatment of uncertainty, as well as applications in many fields including retail, energy, finance and supply chains. Each section is written by specialists in the area and refers to the relevant literature, so the paper serves as a reference guide to the state of the art

in forecasting. Because it brings together the views of many experts, the review is also a useful guide to which practices are widely agreed upon and which remain under debate. It has quickly become a standard reference for both researchers and practitioners who need an overview of the field.

3.2.12.2 Critical Analysis of the Research Item

The breadth of the review makes it a valuable reference for forecast evaluation practices and for guidelines on model selection, and its sections on evaluation are particularly useful for avoiding common mistakes such as evaluating forecasts on data that the model has already seen. However, because it is an encyclopedic review, it lacks a specific focus on retail at the size and store level and on small businesses. Readers must therefore select and adapt the relevant guidance themselves. Some sections are also necessarily brief, so the review often points to other sources rather than giving complete guidance. Its value lies in its breadth and in the reliability of its recommendations rather than in any single new result. For a project such as StockSense, it works best as a checklist of good evaluation practice.

3.2.12.3 Relationship to the Proposed Research Work

The review provides guidance on forecast evaluation and model selection that shapes the comparison between the gradient-boosted model and the statistical benchmarks in StockSense. In particular, it supports the use of chronological data splits, rolling-origin validation and error measures that remain valid for sparse series, such as the weighted absolute percentage error (WAPE) and the mean absolute error (MAE), alongside the root mean squared error (RMSE) and the mean absolute percentage error (MAPE). These choices are described in the forecast evaluation methodology in Chapter 5 and in the experiments planned in Chapter 9. It also supports reporting results separately for different types of series, since an average over all series can hide poor performance on the series that matter most. These practices help ensure that the comparison between LightGBM and the baselines in StockSense is fair and that its conclusions can be trusted.

3.2.13 From Predictive to Prescriptive Analytics

3.2.13.1 Summary of the Research Item

Bertsimas and Kallus [15] present a framework for turning predictive ML models into prescriptions, that is, into decisions that minimize an expected cost. Instead of first forecasting a single value and then optimizing as if the forecast were certain, their approach uses methods such as nearest neighbours and random forests to weight past observations that are similar to the current situation and then optimizes the decision directly against this weighted data. The authors demonstrate the framework with an inventory management problem at a large media distribution company and show that it gives better decisions than

approaches that ignore the link between prediction and decision. The framework also defines a measure of how much the auxiliary data improves the decisions, which allows the value of the data itself to be judged. The paper is an important bridge between ML and operations research.

3.2.13.2 Critical Analysis of the Research Item

The paper provides a strong theoretical basis for going beyond prediction to the decision stage, which is central to the design of StockSense. Its main contribution is to show that the quality of a decision depends on how the prediction is used, not only on how accurate it is. However, the case study concerns a large distribution network with many locations and rich auxiliary data, whereas StockSense serves retailers with a few branches and limited data. The methods are also more complex to implement and explain than the simple decision rules that small business owners can easily follow. There is therefore a trade-off between the quality of the decisions and their transparency, which StockSense resolves in favour of transparency because small business owners need to understand the advice they receive. The paper's main idea can still be applied in this simpler setting.

3.2.13.3 Relationship to the Proposed Research Work

The paper supports the StockSense design decision to turn forecasts into specific prescriptions, namely redistribution, markdown and reorder recommendations, rather than simply presenting demand predictions to the user. StockSense follows the same principle in a simpler form, by accounting for forecast uncertainty through an upper quantile forecast and a safety stock, and by choosing transfers and discounts that maximize the expected margin. This keeps the decision logic transparent enough to be explained to the owner while still linking each decision to the forecast. The historical backtest planned in Chapter 9 also follows the spirit of the paper, because it judges the system by the quality of its decisions, measured in dead stock and lost sales, rather than by the accuracy of its forecasts alone. In future work, the prescriptive methods of the paper could be used to refine the decision rules once enough data has been collected.

3.2.14 Stop Explaining Black Box Machine Learning Models for High Stakes Decisions

3.2.14.1 Summary of the Research Item

Rudin [16] argues that in high-stakes decisions it is better to use models that are interpretable by design than to use black box models and explain them afterwards. The paper points out that explanations produced for black box models may be unreliable or misleading, because they only approximate what the model actually does. Using examples from areas such as criminal justice and healthcare, it shows that interpretable models can often reach accuracy similar to black box models, and it calls on researchers

and practitioners to prefer such models whenever decisions have serious consequences for people. She also argues that the belief that accuracy must always be traded for interpretability is often mistaken, especially for structured data with meaningful features. The paper has become one of the most influential statements in the debate on explainable artificial intelligence (AI).

3.2.14.2 Critical Analysis of the Research Item

The argument is clear, well supported and widely cited, and it has influenced how the research community approaches explainability. Its examples are convincing for decisions that affect people's lives. However, the paper is framed around high-stakes domains such as justice and medicine, where the cost of a wrong decision is very high, rather than around retail inventory decisions, where the stakes of an individual recommendation are comparatively low and some loss of interpretability may be acceptable in exchange for accuracy. The paper also does not address situations in which the users of a model are not specialists, such as small business owners, for whom even an interpretable model may need to be translated into plain language. Its message nevertheless applies to any system whose outputs people are expected to act on.

3.2.14.3 Relationship to the Proposed Research Work

Although StockSense recommendations are not high-stakes in the sense that Rudin describes, the same issue arises because a business owner is unlikely to follow a recommendation that he or she does not understand. This supports the StockSense approach of attaching an explanation to every recommendation. To reduce the risk raised by Rudin, the main part of each StockSense explanation is built from the actual quantities used by the decision rule, such as forecast demand, current stock and transfer cost, and only the description of the forecast drivers comes from the model's feature contributions. The decision rules themselves, such as the transportation problem and the markdown grid, are simple and transparent, so they can be explained exactly rather than approximately.

3.2.15 Desperately Seeking Shelf Availability

3.2.15.1 Summary of the Research Item

Corsten and Gruen [17] examine the extent, causes and consequences of out-of-stocks in retail based on a large set of studies conducted worldwide. They report that out-of-stock rates had not improved over a long period, with an average of about eight percent of items unavailable at any time. Most out-of-stocks were found to be caused by problems in store ordering and replenishment rather than by shortages at suppliers. The paper also studies how consumers respond to an out-of-stock and shows that a substantial share of them buy the item at another store, delay the purchase or do not buy at all, instead of choosing

a substitute in the same store. The authors therefore argue that retailers should treat out-of-stocks as a strategic problem rather than as an occasional operational issue, since lost customers may not return.

3.2.15.2 Critical Analysis of the Research Item

This is a major and widely cited study that shows that out-of-stocks are a persistent and costly problem for retailers, which provides practical rather than purely theoretical motivation for this project. Its findings on the causes of out-of-stocks are especially useful, because they show that better ordering decisions at the store level can address most of the problem. The main limitation for StockSense is that the study concerns fast-moving consumer goods sold in supermarkets rather than apparel and footwear, where demand is more seasonal and sizes add another level of detail. The study is also more than twenty years old, and retail practices have changed since then, although later studies have reported out-of-stock rates of a similar level. Its main conclusions about the causes and consequences of out-of-stocks are therefore still considered valid.

3.2.15.3 Relationship to the Proposed Research Work

The paper strengthens the business case for giving priority to redistribution and timely reordering. Since many customers facing a stockout leave the store entirely rather than buying a different item, preventing a stockout has a larger business impact than the lost sale of a single item suggests. Its finding that most stockouts come from store-level ordering problems also supports the StockSense approach of forecasting demand separately for each store, so that each branch receives stock according to its own demand rather than an average across all branches. It also shows why lost sales are one of the measures used in the StockSense backtest, alongside dead stock and sell-through, since reducing stockouts is as important to retailers as reducing excess stock. The paper thus links the technical design of StockSense to a well-documented business problem.

3.3 Literature Review Summary Table

Table 3.2 summarizes the fifteen reviewed papers, their year of publication, their main finding and their relevance to StockSense.

Table 3.2: Summary of Literature Review

No.	Paper Title	Year	Summary	Relevance to StockSense
1	Exploring the Use of Deep Neural Networks for Sales Forecasting in Fashion Retail	2018	Deep learning outperforms shallow models for new fashion-product sales forecasting	Supports learning-based forecasting of fashion product sales
2	Clearance Pricing Optimization for a Fast-Fashion Retailer	2012	Formal markdown model increased clearance revenue at Zara by 6%	Validates preventive markdown optimization as a business lever
3	Inventory Models with Lateral Transshipments: A Review	2011	Reviews proactive and reactive stock-transfer models between locations	Basis for treating branch-to-branch redistribution as a first-class action
4	Forecasting Across Time Series Databases Using RNNs on Groups of Similar Series	2020	Clustering similar series and forecasting them with one shared model improves accuracy	Basis for shared/clustered forecasting models
5	M5 Accuracy Competition: Results, Findings, and Conclusions	2022	Gradient-boosted tree models outperformed statistical baselines on large-scale retail data	Informs choice of forecasting model family

Continued on next page

No.	Paper Title	Year	Summary	Relevance to StockSense
6	Demand Forecasting for Improved Inventory Management in SMBs	2023	ML outperforms classic methods for small-business demand forecasting	Directly matches StockSense's target user segment
7	Retail Forecasting: Research and Practice	2022	Survey of retail-forecasting research and the gap with practitioner adoption	Confirms the small-retailer research and tooling gap
8	Principles and Algorithms for Forecasting Groups of Time Series	2021	Formalizes when a shared global model outperforms per-series local models	Theoretical basis for shared forecasting models
9	Reducing the Cost of Demand Uncertainty Through Accurate Response to Early Sales	1996	Quick Response to early sales data increased a fashion retailer's profits by $\sim$60%	Validates acting on recent sales velocity for stocking decisions
10	Sales Forecasts in Clothing Industry: The Key Success Factor of Supply Chain Management	2010	Reviews forecasting models suited to apparel's short life cycles and seasonality	Supports forecasting at the product-size-location level

Continued on next page

No.	Paper Title	Year	Summary	Relevance to StockSense
11	Predictive Big Data Analytics for Supply Chain Demand Forecasting	2020	Survey of data-driven forecasting methods and research gaps across supply chains	Confirms the gap for organizations without analytics infrastructure
12	Forecasting: Theory and Practice	2022	Encyclopedic review of forecasting theory, methods, and evaluation practice	Informs forecast evaluation and model-selection approach
13	From Predictive to Prescriptive Analytics	2020	Framework for converting predictive models into actionable decisions	Basis for converting forecasts into recommendations, not just numbers
14	Stop Explaining Black Box Machine Learning Models for High Stakes Decisions and Use Interpretable Models Instead	2019	Argues interpretable models are more trustworthy than explained black boxes	Supports explainable, plain-language recommendations
15	Desperately Seeking Shelf Availability	2003	Worldwide study finds stockouts persistent; many customers switch stores entirely	Strengthens business case for prioritizing stockout prevention

3.4 Related Work

This section reviews six commercial platforms that address demand planning or multi-location inventory management. They range from enterprise planning suites used by large chains to POS systems aimed at small businesses. For each platform the review describes its main capabilities and explains how it differs from StockSense, especially in terms of cost, the level of detail of its forecasts and the priority

it gives to redistribution between stores. The information is based on the public product pages of each vendor. The selection covers the main types of solution available to retailers today, so that the position of StockSense in the market can be seen clearly. For each platform, the features that matter most for small multi-branch retailers are emphasized, namely the cost of entry, the level of detail of the forecasts and the treatment of transfers between stores.

3.4.1 SAP Integrated Business Planning

SAP Integrated Business Planning (IBP) is an enterprise demand sensing and supply chain planning platform [18]. It takes in near real-time signals, including POS data, to fine-tune short-term demand forecasts and to help large companies avoid stockouts and overstocking across their networks. It also supports sales and operations planning, inventory optimization and supply planning in a single integrated environment that connects to the company's ERP system. The platform is designed for large organizations with dedicated planning teams, existing ERP systems and enterprise licensing budgets. For a small retailer with two or three stores, the cost of licensing, implementation and specialist staff makes it out of reach, and its scope is far wider than such a business needs. StockSense addresses the same core problem of matching stock to demand, but in a form that a small business can adopt for free through its POS and extend with an affordable subscription.

3.4.2 Blue Yonder

Blue Yonder, a Panasonic company, provides demand planning and retail planning software based on ML and is used by large retailers [19]. Its forecasting goes beyond historical averages by modeling the causal effects of factors such as prices, promotions, weather and events on demand, and the company presents its forecasts in a way that helps planners see why a forecast was made. These are strong capabilities, and the explanation of forecasts in particular is close in spirit to the StockSense approach. However, like SAP, Blue Yonder is an enterprise product that requires considerable implementation effort, integration with existing systems and a technical infrastructure that small businesses do not have. Its pricing and onboarding process are aimed at large retail chains. StockSense takes the idea of explained, causally informed forecasts and offers it in a much simpler package suited to small retailers.

3.4.3 Nextail

Nextail is a merchandising platform based on AI that is designed specifically for fashion, footwear and apparel retailers [20]. It supports in-season allocation, replenishment and rebalancing of inventory between stores, with the aim of selling more products at full price and reducing markdowns at the end of the season. Of all the platforms reviewed, Nextail is the closest to the redistribution focus of StockSense,

because it explicitly moves stock between stores according to demand. It also works at a detailed level, which is necessary for apparel with many sizes. However, it targets mid-size and large fashion chains with many stores, costs considerably more than a small retailer can pay and offers its recommendations as part of a larger merchandising suite. It does not provide a free entry point such as a POS, so a small shop would have to pay for and integrate the full platform before seeing any benefit.

3.4.4 Shopify POS (with Stocky)

Shopify POS offers multi-location inventory management, stock transfers between locations and low-stock alerts [21]. Through its Stocky app and its Sidekick AI assistant, it also provides demand-based reorder suggestions and recommendations for rebalancing stock between stores. It is the easiest of the reviewed tools for a small retailer to adopt, particularly for a business that already sells online through Shopify. However, the advanced inventory features are not included in the basic subscription and are available only on higher paid plans, so the full set of features requires a significant monthly fee. In addition, its forecasting and rebalancing work mainly at the product level rather than at the level of sizes and variants, which is where most dead stock in apparel and footwear actually builds up. StockSense differs by forecasting at the size and location level and by giving redistribution priority over new orders.

3.4.5 Lightspeed Retail

Lightspeed Retail is a POS and inventory system designed for retailers with several locations and a large number of SKUs [22]. It supports stock transfers between stores, custom reorder points for each location and reports that highlight slow-moving items, which are useful tools for managing inventory across branches. However, the reorder points are rules that the user has to set manually, and the slow-mover reports show items only after they have already sold poorly. The platform does not provide demand forecasting at the size or variant level, explanations for its suggestions or a separate tool for choosing the timing of markdowns. It also has no free tier, so retailers have to pay for the POS even if they do not need advanced inventory analytics. StockSense differs by offering the POS for free and by replacing manual reorder points and backward-looking reports with forecasts and early recommendations.

3.4.6 Odoo

Odoo is an open-source business platform whose free Community edition includes a POS application and an Inventory application with reordering rules and transfers between warehouses [23]. It is the existing tool closest to the free POS model of StockSense, since the core POS and inventory tracking cost nothing. However, the free edition has no built-in demand forecasting, no analysis at the size or variant level and no explained recommendations. Such capabilities are available only as separately

priced third-party marketplace applications or in the paid Enterprise edition, and even these add-ons generally forecast at the product level and do not give priority to redistribution between branches. Setting up and maintaining Odoo also requires some technical effort, which many small shop owners cannot provide themselves. StockSense combines the free POS idea of Odoo with built-in forecasting and recommendations designed specifically for small multi-branch retailers.

3.5 Related Work Summary Table

Table 3.3 summarizes the six platforms, their target users and their main gaps relative to StockSense. The comparison shows that the platforms with strong forecasting are designed for large enterprises, while the platforms that are affordable for small businesses offer at most product-level forecasting and do not give priority to redistribution.

Table 3.3: Summary of Related Work

No.	Platform	Target Users	Gap Relative to StockSense
1	SAP Integrated Business Planning	Large enterprises with ERP	Requires ERP infrastructure and enterprise budget; no free tier
2	Blue Yonder	Large retail chains	Enterprise-only; requires dedicated planning/IT teams
3	Nextail	Mid-size to large fashion chains	Closest in focus to redistribution, but priced and scaled for many-store chains, not a free small-retailer entry point
4	Shopify POS (Stocky/Sidekick)	Shopify merchants of any size	Product-level, not size/variant-level, forecasting; advanced features gated behind paid plans
5	Lightspeed Retail	Multi-location, high-SKU retailers	No demand forecasting or explainable recommendations; no free tier
6	Odoo (Community)	Small businesses wanting free, self-hosted POS/Inventory	Free tier has no demand forecasting or explainable recommendations; forecasting only via paid add-ons, product-level not size/variant-level

3.6 Conclusion

The literature shows that ML forecasting and formal markdown and redistribution methods can improve inventory decisions and increase revenue. Studies of the M5 competition and of small-business data indicate that gradient-boosted models trained globally on many related series forecast retail demand more accurately than statistical methods, while work on transshipments and markdowns shows the value of moving stock proactively and discounting it in a planned way. Research on prescriptive analytics and ex-

plainability adds that forecasts are most useful when they are turned into decisions that users understand. However, most of this research, and most of the commercial solutions reviewed, are designed for large retailers that have the infrastructure and data teams to use them. The platforms that small businesses can afford offer at most product-level forecasting and treat transfers between stores as one option among many. None of the reviewed solutions combines a free POS with explained, size- and location-level forecasting and recommendations that put redistribution first for small multi-branch retailers. StockSense is designed to fill this gap.

3.7 Selected Model

Based on this review, the StockSense forecasting engine will use the gradient-boosted tree family, implemented with LightGBM, as its main model, because this family performed best on hierarchical retail sales data in the M5 competition and also outperformed classical methods on small-business data. A single global model will be trained for each tenant on all of its product, size and store series, following the work of Montero-Manso and Hyndman and of Bandara et al., while simple statistical methods will be kept as baselines to check whether the more complex model is really needed. Redistribution recommendations will follow the proactive transshipment framing of Paterson et al., with moving stock between branches given priority over discounting and reordering. The markdown optimizer will follow the forecast-then-optimize approach of Caro and Gallien, but it will be applied early in the season so that items are discounted before they become unsellable. Every recommendation will carry an explanation built from the quantities behind it, in line with the concerns raised by Rudin.

Chapter 4 Software Requirement Specifications

This chapter describes the functional and non-functional requirements of StockSense, which are derived from the goals and scope presented in Chapter 2 and from the model choices made in Chapter 3. It starts with the list of features that the platform will offer and then states the functional requirements of each type of user and of the system itself. These are followed by the quality attributes and non-functional requirements, which describe how well the system must perform rather than what it must do, and by the assumptions on which the requirements are based. The chapter then presents the use cases of the system in a structured table format, the hardware and software requirements, the graphical user interface designed for the main screens and the database design, including the entity relationship diagram and the data dictionary. It closes with an analysis of the main risks to the project. Together these sections define what will be built in the second phase of the project and how it will be verified.

4.1 List of Features

The features of StockSense follow from the freemium model described in Chapter 2. The free tier covers everything a small business needs to run its daily sales, including registration, the POS, the upload of historical data and the management of stores, products and inventory. The paid tier adds the analytical features that turn this data into decisions, which are forecasting, redistribution, reordering and markdown recommendations, each with an explanation and an estimated financial impact. The dashboard is shared by both tiers but shows the analytical sections only to paying tenants. The full list of features is given below. This division means that a business can start using StockSense at no cost and upgrade only once it has enough history to benefit from forecasting.

- Multi-tenant business registration and secure login.
- A free built-in POS for recording sales and tracking inventory.
- Upload of historical sales reports in CSV and Excel format for businesses that use an external POS.
- Validation, cleaning and normalization of uploaded sales data.
- Management of products, sizes and variants, and stores.
- Demand forecasting for each product, size and location.
- Recommendations for redistributing inventory between branches.
- Reorder recommendations.
- A markdown optimizer for slow-moving stock.
- Explanations and estimated financial impact for every recommendation.
- A web dashboard showing sales, inventory, forecasts and recommendations.

- Freemium subscription management with a free POS tier and a paid analytics tier.
- Benchmarking across tenants, if enough tenants and data are available.

4.2 Functional Requirements

The functional requirements are grouped by the actor that they serve. Each requirement has an identifier that is used in the use cases and test cases later in the report, so that every requirement can be traced to the way it will be tested. The business owner has the widest set of functions, because the owner manages the whole business and makes the decisions on recommendations and subscriptions. The store manager works within a single branch, mainly through the POS and the recommendations that concern that branch. The system requirements describe the behaviour that the platform must show without direct user action, such as keeping tenant data separate, cleaning uploaded files, updating inventory after a sale and producing forecasts and recommendations.

4.2.1 Functional Requirements of the Business Owner

Table 4.1 lists the requirements of the business owner, who registers the business as a tenant and manages its stores, users, recommendations and subscription.

Table 4.1: Functional Requirements of the Business Owner

ID	Requirement
FR-1	The business owner shall be able to register the business as a tenant and open a business account.
FR-2	The business owner shall be able to add, edit and remove the stores of the business.
FR-3	The business owner shall have consolidated access to the sales, inventory and forecast data of all stores.
FR-4	The business owner shall be able to view and act on redistribution, reorder and markdown recommendations.
FR-5	The business owner shall be able to upgrade or downgrade the subscription tier.
FR-6	The business owner shall be able to invite and manage store manager accounts.

4.2.2 Functional Requirements of the Store Manager

Table 4.2 lists the requirements of the store manager, whose access is limited to the store that the owner has assigned.

Table 4.2: Functional Requirements of the Store Manager

ID	Requirement
FR-7	The store manager shall be able to record sales through the POS interface.
FR-8	The store manager shall be able to view the current inventory of the assigned store.
FR-9	The store manager shall be able to view the demand forecasts and recommendations relevant to the assigned store.
FR-10	The store manager shall be able to confirm or reject a redistribution recommendation that involves the assigned store.

4.2.3 Functional Requirements of the System

Table 4.3 lists the requirements that the system must fulfil on its own, covering tenant isolation, data ingestion, inventory updates, forecasting, recommendations and access control.

Table 4.3: Functional Requirements of the System

ID	Requirement
FR-11	The system shall keep the data of each tenant separate from the data of all other tenants.
FR-12	The system shall accept and validate sales report uploads in CSV and Excel format.
FR-13	The system shall normalize uploaded data into a common schema of product, size, store, date and quantity.
FR-14	The system shall update the inventory automatically after each recorded sale.
FR-15	The system shall produce demand forecasts at the product, size and location level.
FR-16	The system shall determine whether each store has a shortage or a surplus of inventory compared with the forecast demand.
FR-17	The system shall recommend a redistribution when one or more branches have a surplus of an item and size for which another branch has a forecast shortage.
FR-18	The system shall recommend a reorder when no other branch has enough surplus to cover a shortage.
FR-19	The system shall identify inventory at risk of becoming dead stock and recommend a markdown before it becomes unsellable.
FR-20	The system shall accompany every recommendation with a clear explanation and an estimate of its financial impact.
FR-21	The system shall restrict access to paid features according to the subscription tier of the tenant.
FR-22	The system shall evaluate forecast accuracy with standard error metrics such as MAPE and RMSE.

4.3 Quality Attributes

The quality attributes below describe the characteristics that the design of StockSense must have in order to be dependable, easy to change and easy to use. They complement the functional requirements and

guide the design decisions in Chapter 6.

4.3.1 Reliability

The system shall produce consistent forecasts and recommendations from the same input data, so that two runs on the same sales history give the same results. This is important because owners will compare recommendations over time and will lose trust in the system if the advice changes without a change in the data. To achieve this, model training uses fixed random seeds and every forecast is stored together with the name of the model that produced it. The system shall also handle malformed uploads safely. A file with missing columns, invalid dates or damaged content must never crash the server or corrupt the data that a tenant has already saved. Uploaded rows are therefore validated before anything is written to the database, and the import of valid rows takes place in a single transaction that is rolled back completely if any step fails. If a forecasting run fails, the previous valid forecasts remain available on the dashboard until a new run succeeds.

4.3.2 Maintainability

The forecasting, recommendation, markdown and POS logic shall be built as separate modules with clear interfaces, so that any one of them can be changed, retrained or replaced without affecting the others. For example, it must be possible to replace the forecasting model with a different algorithm by changing only the forecasting module, as long as the new model writes its results to the same forecast table. This separation also allows the team members to work on different modules in parallel during the second phase of the project. The code will follow agreed naming conventions, will be organized into one package per module and will be reviewed by another team member before it is merged. Errors such as failed uploads, rejected rows and failed forecasting runs shall be logged with enough detail to find their cause quickly, which keeps the effort of maintaining the system low after deployment.

4.3.3 Usability

The dashboard shall present forecasts and recommendations in a form that business owners and store managers can understand without any knowledge of data science or ML. Forecasts will be shown as charts next to past sales, and every recommendation will be written as a short sentence in plain business language that states what to do, why, and what the expected financial impact is. Statistical terms such as quantiles or error metrics will not be shown to ordinary users. The POS screen must allow a sale to be recorded in a few clicks, because it is used during busy business hours. Error messages, for instance when an uploaded file cannot be read, must explain in simple terms what went wrong and how to fix it. The interface will be tested with first-time users to check that they can register, record a sale and act on

a recommendation without help or a user manual.

4.3.4 Correctness

The inventory quantities shown to users shall always reflect the latest sales, adjustments and completed transfers. Every sale recorded through the POS decrements the stock of the sold variant in the same database transaction that stores the sale, so the inventory and the sales record can never disagree. Manual adjustments after a physical count are logged with a reason, and accepted redistribution recommendations update the inventory of both stores when the transfer is completed. Recommendations themselves must be based on the stock position at the time they are generated, and if the stock of the source store has changed before a transfer is approved, the system must warn the user rather than create a transfer that is no longer valid. Forecasts and recommendations also depend on correct input data, which is why uploaded data is validated against the products, sizes and stores of the tenant before it is accepted.

4.3.5 Efficiency

The system shall produce forecasts and recommendations for the entire product catalog of a tenant within a reasonable time that depends on the size of the uploaded dataset. To keep the user interface fast, the heavy work of training models, generating forecasts and processing large uploads runs in background jobs rather than during user requests, and the results are stored in the database so that they do not have to be recomputed each time the dashboard is opened. Uploaded CSV files are read in chunks and Excel files in batches, which keeps memory use low even for files with up to 100,000 rows. The recommendation problems are small, because each one involves only the few branches of a single business, so they can be solved quickly for every variant. Database tables will be indexed on the tenant, store and variant identifiers that are used in almost every query.

4.3.6 Flexibility

The system shall allow new forecasting models and new recommendation rules to be added without redesigning the data ingestion or dashboard layers. Every forecast is stored together with the name of the model that produced it, so several models can be run side by side and compared before one of them is chosen for a tenant. The recommendation engine works on forecasts and stock positions rather than on a particular model, so new rules, for example for seasonal promotions, can be added as further steps in its pipeline. The data ingestion layer maps uploaded columns to a common schema, so support for the file formats of further POS systems can be added by defining new mappings rather than changing the rest of the system. The forecasting engine is also general-purpose, so it can serve any business that stocks products with variants, sizes or several locations, not only clothing and footwear stores.

4.4 Non-Functional Requirements

The non-functional requirements below set measurable targets for performance, reliability, security and compatibility. These targets are checked by the non-functional test cases in Chapter 7.

4.4.1 Performance

The system should process a typical small retailer's monthly sales upload, which contains a few thousand rows, and produce updated forecasts within a couple of minutes. Larger historical uploads of up to 100,000 rows should be processed in less than five minutes without the system running out of memory. A sale recorded through the POS should be saved, together with the inventory update, in less than two seconds, because the POS is used while customers are waiting at the counter. Dashboard pages should load quickly because they read forecasts and recommendations that have already been computed and stored, rather than computing them on request. Forecasting and model training run as background jobs, so a long training run for one tenant must not slow down the POS or dashboard for other tenants. These targets will be measured during testing with realistic sample data for tenants with two or three branches.

4.4.2 Reliability

The POS and the dashboard should be highly available, since store managers rely on the POS to record sales throughout business hours and any downtime means sales that are not recorded. The system will therefore be deployed so that a failure in the forecasting or upload components cannot stop the POS from working, and the forecasting jobs will be retried automatically if they fail. The database will be backed up regularly so that business data can be restored after a failure. A failed upload or forecasting run must leave the existing data and forecasts unchanged, which is achieved through database transactions and by keeping the previous forecasts until new ones have been created successfully. Errors will be logged and monitored so that problems can be noticed and fixed before they affect many users.

4.4.3 Security

Tenant information will be isolated at the database level, with every business table carrying a tenant identifier that the backend checks before any read or write operation. Access within a tenant will be based on roles, so a store manager can see and change only the data of the assigned store, and the system administrator can manage tenants and plans without viewing their detailed sales data unless this is needed for support. Passwords will be stored only as hashes produced by a secure hashing algorithm, never in plain text. All communication between the browser and the server will be encrypted with Hypertext Transfer Protocol Secure (HTTPS) over Transport Layer Security (TLS), and the server will

not accept plain HTTP connections. Database queries will be parameterized to prevent injection attacks, and secret keys will be kept out of the source code. Tenant isolation and role-based access will be tested specifically to make sure that no data can leak between businesses or between stores.

4.4.4 Compatibility

StockSense will be accessed through a modern web browser such as Chrome, Edge, Firefox or Safari, so users will not need to install a native application on their computers. This matters for small businesses, which often use a mix of older laptops, desktops and tablets at the counter. The interface will use a responsive layout so that the dashboard and the POS can be used on screens of different sizes, including tablets. On the data side, the system will accept sales reports in CSV and Excel format, which almost every POS system and spreadsheet program can export, and it will let the owner map the columns of the file to the required fields and choose the date format, so that files from different sources can be imported without manual editing.

4.5 Assumptions

The requirements above are based on several assumptions about the businesses that will use StockSense and the environment in which they work. If any of these assumptions does not hold for a particular business, some features, especially forecasting, may be less effective for that business. The assumptions are listed below. The assumptions will be reviewed during testing, and the requirements will be adjusted if any of them proves unrealistic for the target businesses.

- Businesses have, or are willing to record, sales data at the product, size and store level.
- Users have basic computer skills and a stable internet connection.
- Uploaded CSV and Excel files follow, or can be mapped to, a reasonably consistent column structure.
- Each store belongs to exactly one tenant at a time.
- Forecast accuracy for a tenant will improve as more historical data accumulates, which will be measured rather than assumed.

4.6 Use Cases

The use cases below describe how each actor interacts with StockSense to achieve a goal. Each use case is presented in a table that gives its actors, summary, pre-conditions, post-conditions, basic flow and alternative flow. Use cases UC-01 to UC-08 cover the free tier, including registration, login, the management of stores, managers, products and inventory, the POS and the upload of historical data.

UC-09 to UC-13 cover the dashboard, forecasts and the three types of recommendation, which are available on the paid tier, while UC-14 and UC-15 cover the management of subscriptions by the owner and of tenants by the system administrator. Each use case has a matching sequence diagram in Chapter 6 and at least one test case in Chapter 7.

Table 4.4: UC-01 Register Business

Name	Register Business
Actors	Business Owner
Summary	A new business signs up as a tenant on StockSense.
Pre-Conditions	The business is not already registered.
Post-Conditions	A new tenant account is created on the free POS tier.
Basic Flow	1. The owner opens the registration page. 2. The owner enters business and account details. 3. The system creates the tenant and sends a verification email.
Alternative Flow	If the email is already registered, the system displays an error and asks for a different email.

Table 4.5: UC-02 Login

Name	Login
Actors	Business Owner, Store Manager, System Administrator
Summary	A registered user logs in using an email and password.
Pre-Conditions	The user has an account and is not already logged in.
Post-Conditions	A session is established and the user is redirected to the dashboard.
Basic Flow	1. The user opens the login page. 2. The user enters the email and password. 3. The system verifies the credentials and opens the dashboard.
Alternative Flow	If the credentials are invalid, the system displays “Incorrect email or password” and stays on the login page.

Table 4.6: UC-03 Manage Stores

Name	Manage Stores
Actors	Business Owner
Summary	The owner adds, edits, or removes the stores (branches) of the business.
Pre-Conditions	The owner is logged in.
Post-Conditions	The list of stores of the tenant is updated.
Basic Flow	1. The owner opens the Stores page. 2. The owner chooses to add, edit, or remove a store and enters the details. 3. The system validates and saves the change.
Alternative Flow	If a store being removed still has inventory or sales history, the system archives it instead of deleting it.

Table 4.7: UC-04 Manage Store Managers

Name	Manage Store Managers
Actors	Business Owner
Summary	The owner invites store managers, assigns them to a store, or removes their access.
Pre-Conditions	The owner is logged in and at least one store exists.
Post-Conditions	The store manager accounts of the tenant are updated.
Basic Flow	1. The owner opens the Users page. 2. The owner invites a manager by email and selects the store to assign. 3. The system creates the account and sends an invitation.
Alternative Flow	If the email already belongs to another account, the system rejects the invitation and shows an error.

Table 4.8: UC-05 Manage Products and Variants

Name	Manage Products and Variants
Actors	Business Owner
Summary	The owner adds, edits, or removes products and their sizes/variants.
Pre-Conditions	The owner is logged in.
Post-Conditions	The product catalog of the tenant is updated.
Basic Flow	1. The owner opens the Products page. 2. The owner creates or edits a product and its sizes/variants. 3. The system validates and saves the product catalog.
Alternative Flow	If a variant with the same SKU already exists, the system rejects the entry and asks for a different SKU.

Table 4.9: UC-06 Adjust Inventory

Name	Adjust Inventory
Actors	Business Owner, Store Manager
Summary	A user corrects the stock quantity of a variant at a store, for example after a physical count.
Pre-Conditions	The user is logged in and the variant exists.
Post-Conditions	The stock quantity is updated and the adjustment is logged.
Basic Flow	1. The user selects a store and a variant. 2. The user enters the new quantity and a reason. 3. The system updates the inventory and records the adjustment.
Alternative Flow	If the entered quantity is negative, the system rejects the adjustment.

Table 4.10: UC-07 Record Sale (POS)

Name	Record Sale (POS)
Actors	Store Manager
Summary	A store manager records a sale through the built-in POS.
Pre-Conditions	The manager is logged in; the product and size exist in inventory.
Post-Conditions	A sales record is created and inventory is decremented.
Basic Flow	1. The manager selects the product and size sold. 2. The manager confirms the quantity and completes the sale. 3. The system logs the sale and updates inventory.
Alternative Flow	If inventory is insufficient, the system warns the manager before confirming.

Table 4.11: UC-08 Upload Historical Sales Data

Name	Upload Historical Sales Data
Actors	Business Owner
Summary	The owner uploads a CSV or Excel file of historical sales from an external POS or spreadsheet.
Pre-Conditions	The owner is logged in and belongs to a registered tenant.
Post-Conditions	Valid rows are cleaned, normalized, and stored; invalid rows are listed in an upload report.
Basic Flow	1. The owner selects a CSV/Excel file. 2. The system shows the detected columns and the owner maps them to the required fields. 3. The system validates and normalizes the data and shows an upload report. 4. The owner approves the report and the system imports the rows.
Alternative Flow	If the file format is unsupported or unreadable, the system rejects the upload and explains why. Rejected rows are listed in the report with reasons.

Table 4.12: UC-09 View Dashboard and Alerts

Name	View Dashboard and Alerts
Actors	Business Owner, Store Manager
Summary	The user views a summary of sales, inventory, forecasts, recommendations, and alerts.
Pre-Conditions	The user is logged in.
Post-Conditions	The dashboard displays current, tenant-scoped data.
Basic Flow	1. The user logs in and is directed to the dashboard. 2. The system displays sales trends, current inventory, open recommendations, and alerts.
Alternative Flow	If the tenant has no sales data yet, the dashboard asks the user to record or upload sales.

Table 4.13: UC-10 View Demand Forecast

Name	View Demand Forecast
Actors	Business Owner, Store Manager
Summary	The user views the forecasted weekly or monthly demand of a product-size-store combination.
Pre-Conditions	The user is logged in and the tenant has a paid subscription.
Post-Conditions	The forecast is displayed next to the past sales.
Basic Flow	1. The user opens the Forecasts page. 2. The user selects a product, size, and store. 3. The system displays the stored forecast with the past sales.
Alternative Flow	If the tenant has not yet provided enough sales history to train a model, no forecast is shown and the user is asked to record or upload more data. On the free tier, the system asks the user to upgrade.

Table 4.14: UC-11 View and Act on Redistribution Recommendation

Name	View and Act on Redistribution Recommendation
Actors	Business Owner, Store Manager
Summary	The user reviews a suggestion to move stock from one branch to another and approves or dismisses it.
Pre-Conditions	A forecasted shortage at one store coincides with a forecasted surplus of the same item at another.
Post-Conditions	An approved recommendation creates a stock transfer; a dismissed one is closed.
Basic Flow	1. The user opens the Recommendations page and selects a redistribution recommendation. 2. The system displays source and destination stores, quantity, explanation, and estimated financial impact. 3. The user approves it and the system creates the stock transfer and notifies both stores.
Alternative Flow	The user dismisses the recommendation and the system marks it as dismissed.

Table 4.15: UC-12 View Markdown Recommendation

Name	View Markdown Recommendation
Actors	Business Owner, Store Manager
Summary	The user views a suggestion to discount a slow-moving item before it becomes dead stock.
Pre-Conditions	The markdown optimizer has flagged an item as at risk of becoming dead stock.
Post-Conditions	The user sees the recommended discount, explanation, and estimated impact.
Basic Flow	1. The user opens the Recommendations page. 2. The user selects a markdown recommendation. 3. The system displays the item, suggested discount, explanation, and estimated financial impact.
Alternative Flow	The user can dismiss the recommendation if a markdown is not desired.

Table 4.16: UC-13 View Reorder Recommendation

Name	View Reorder Recommendation
Actors	Business Owner
Summary	The user views a suggestion to reorder an item when no surplus exists at other branches.
Pre-Conditions	A forecasted shortage remains after redistribution has been considered.
Post-Conditions	The user sees the suggested reorder quantity with its explanation.
Basic Flow	1. The user opens the Recommendations page. 2. The user selects a reorder recommendation. 3. The system displays the item, store, suggested quantity, and explanation.
Alternative Flow	The user marks the recommendation as actioned or dismisses it.

Table 4.17: UC-14 Manage Subscription

Name	Manage Subscription
Actors	Business Owner
Summary	The owner views the current plan and upgrades or downgrades between the free and paid tiers.
Pre-Conditions	The owner is logged in.
Post-Conditions	The subscription tier is updated and paid features are enabled or disabled accordingly.
Basic Flow	1. The owner opens the Subscription page. 2. The owner selects a plan and confirms the change. 3. The system updates the tier and the available features.
Alternative Flow	If the payment fails, the system keeps the current tier and informs the owner.

Table 4.18: UC-15 Manage Tenants

Name	Manage Tenants
Actors	System Administrator
Summary	The administrator views tenants and manages their status and plans.
Pre-Conditions	The administrator is logged in.
Post-Conditions	The status or plan of the selected tenant is updated.
Basic Flow	1. The administrator opens the Tenants page. 2. The administrator selects a tenant and chooses an action (suspend, reactivate, or change plan). 3. The system applies the change and records it.
Alternative Flow	Users of a suspended tenant cannot log in until the tenant is reactivated.

4.7 Hardware and Software Requirements

This section lists the hardware needed to host and use StockSense and the software stack with which it will be built. The stack follows the model choice made in Chapter 3 and the architecture described in Chapter 6.

4.7.1 Hardware Requirements

StockSense runs on a server, while users only need a device with a browser. The server will need at least 8 GB of RAM and a quad-core processor, which is enough to run the web application, the database and the background jobs that train forecasting models and process uploads for a small number of tenants during development and early deployment. The server can be hosted in the cloud or on premises, and its resources can be increased as the number of tenants and the volume of sales history grow. On the client side, any modern computer, tablet or smartphone with a web browser and an internet connection is sufficient, because all processing happens on the server. No special hardware such as barcode scanners or receipt printers is required for the first version, although the POS interface is designed so that such devices could be supported later.

4.7.2 Software Requirements

The backend of StockSense will be written in Python with the FastAPI framework, which provides the application programming interface (API) used by the web dashboard and the POS screens, as well as the forecasting and recommendation services. The web dashboard and the POS interface will be built with React, so that users can work entirely in a browser. All business data will be stored in a PostgreSQL relational database. Forecasting will use scikit-learn for data preparation and evaluation and LightGBM for the gradient-boosted trees, following the model choice informed by the M5 competition in Chapter 3, while statsmodels will provide the statistical baselines. The pandas and openpyxl libraries will read and clean the uploaded CSV and Excel files. The server will run in a Linux environment, and Git with GitHub will be used for version control.

4.8 Graphical User Interface

This section presents mockups of the main screens of StockSense, which were designed to match the use cases above. The screens are kept simple because the main users are business owners and store managers without technical training. A user first reaches the login page, or the signup page when registering a new business. After logging in, the user is taken to the dashboard, which gives an overview of the business

and links to the other screens. Store managers spend most of their time on the POS screen, while owners mainly use the dashboard, the inventory page and the recommendations page. Owners who already have sales data in another system use the upload page to bring it into StockSense. The paid analytical sections, such as forecasts and recommendations, are shown only to tenants on the paid plan, and free tenants see an upgrade notice in their place. Each screen is described below together with the users and the use cases it supports.

4.8.1 Login Page

Any registered user, whether a business owner, a store manager or the system administrator, logs in from this page with an email address and a password, as described in UC-02. If the credentials are wrong, the page shows a short error message and the user stays on the page. Users of a suspended tenant cannot log in until the tenant is reactivated by the administrator.

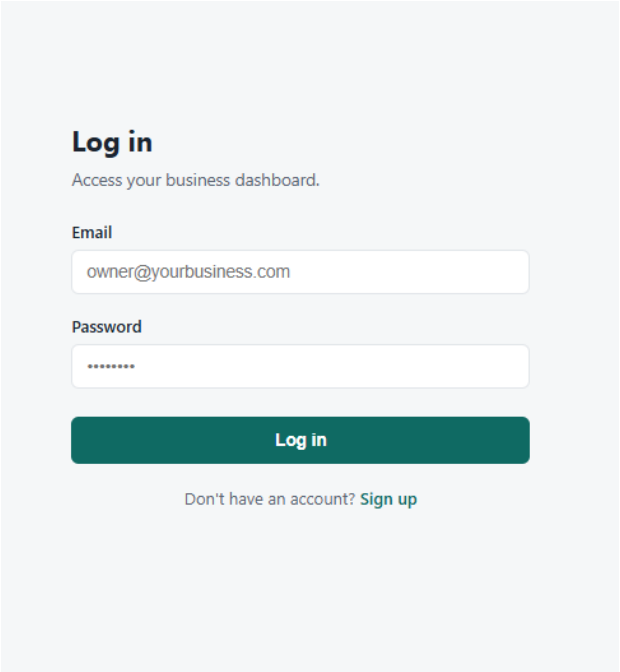


Figure 4.1: Login Page

4.8.2 Signup Page

A new business owner registers the business as a tenant from this page by entering the business name, the owner's name, an email address and a password, as described in UC-01. The new tenant starts on the free POS tier, and a verification email is sent to the owner before the account is used.

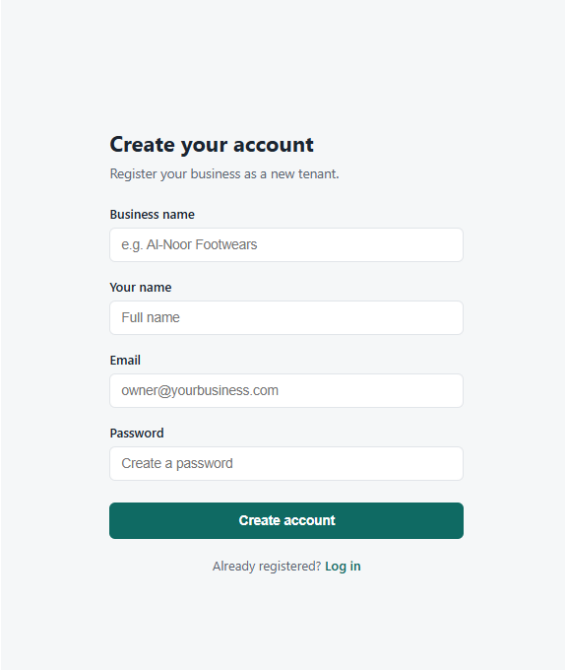


Figure 4.2: Signup Page

4.8.3 Dashboard

The dashboard is the first screen after login and supports UC-09. It shows the monthly sales, the items at risk of stockout or of becoming dead stock, the recent sales activity and the inventory health of each branch. Owners see all branches, while store managers see only their own store. On the paid plan, the dashboard also lists open recommendations and alerts.

4.8.4 POS Page

A store manager records a sale on this page by selecting the branch, the product and the variant sold and confirming the quantity, as described in UC-07. The inventory is updated automatically once the sale is confirmed, and the manager is warned before confirming a sale for which there is not enough stock.

4.8.5 Upload Sales Report

Businesses that already use an external POS or a spreadsheet upload their historical sales as a CSV or Excel file on this page, as described in UC-08. The owner maps the detected columns to the required fields, reviews the upload report with the accepted and rejected rows, and approves the import.

4.8.6 Inventory

This page shows the current stock level of every product variant in every branch and can be filtered by store. A status indicator marks each item as healthy, at risk of stockout or at risk of becoming dead

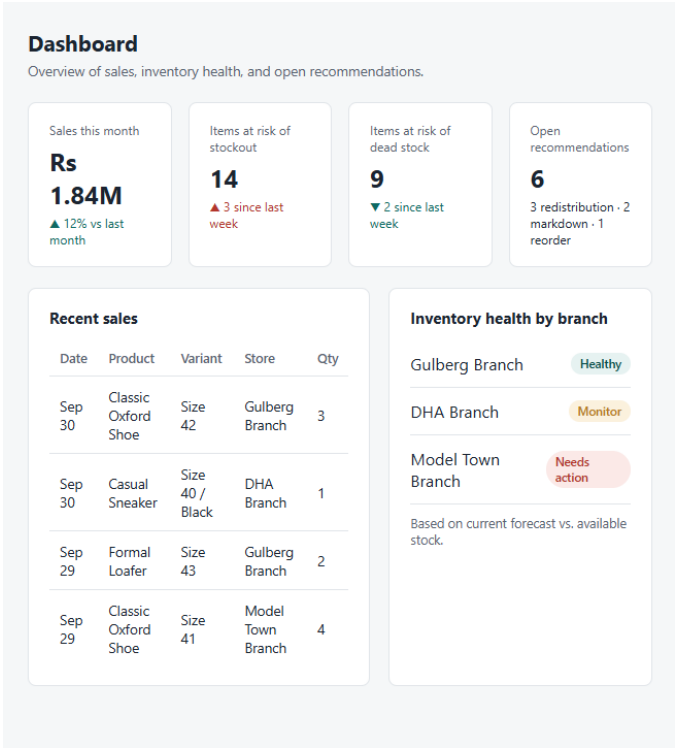


Figure 4.3: Dashboard

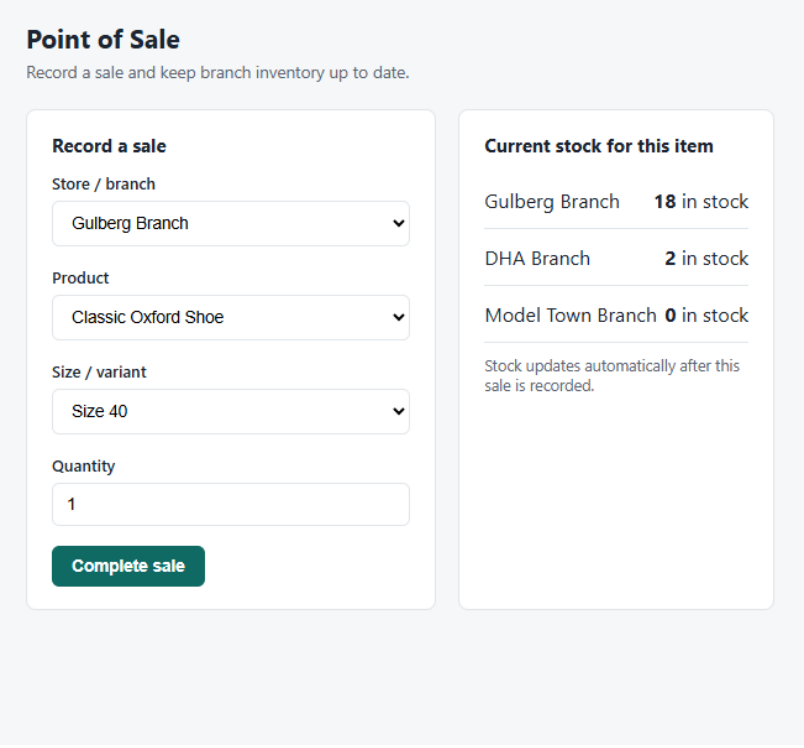


Figure 4.4: POS Page

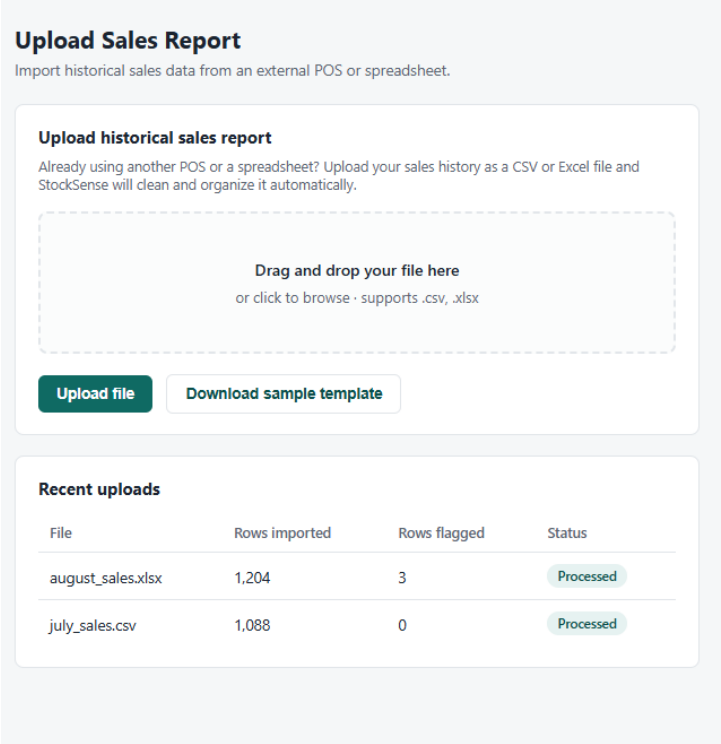


Figure 4.5: Upload Sales Report

stock, and users can correct a quantity after a physical count, as described in UC-06.

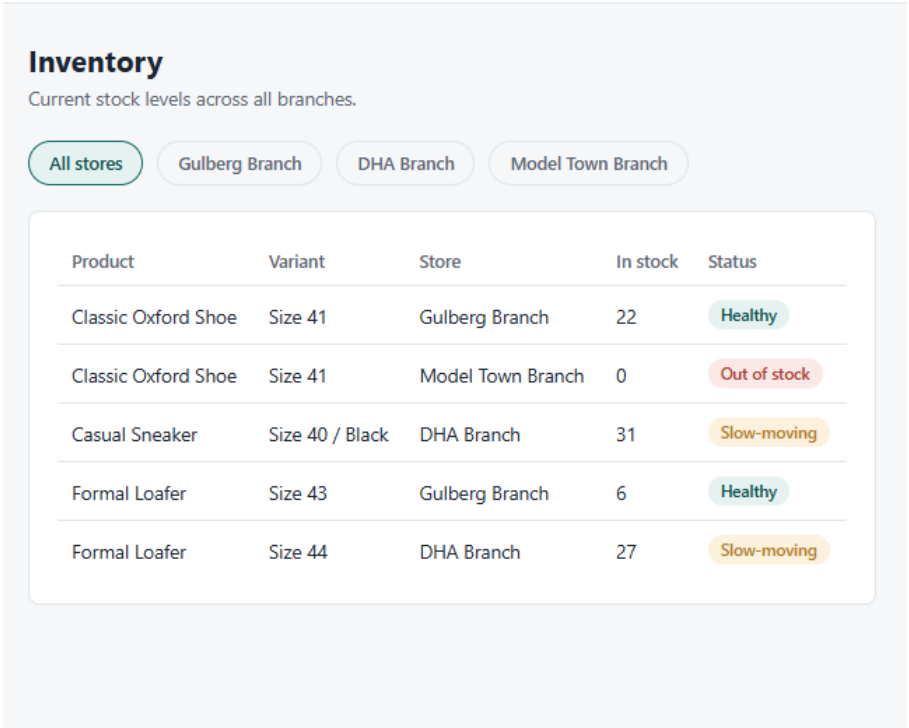


Figure 4.6: Inventory

4.8.7 Recommendations

This page lists the redistribution, markdown and reorder recommendations of the tenant, as described in UC-11 to UC-13. Each recommendation shows a plain-language explanation and an estimated financial impact, and the user can approve or dismiss it. Approving a redistribution creates a stock transfer between the two stores.

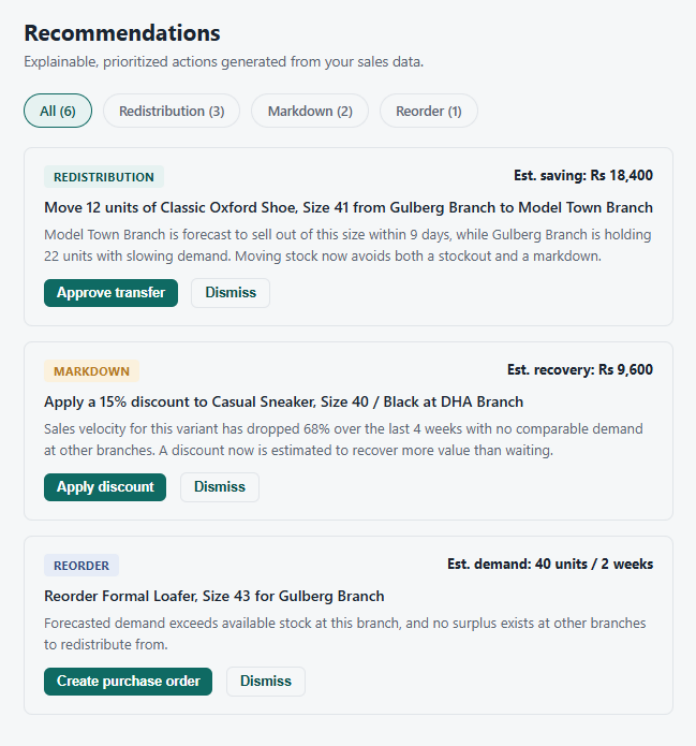


Figure 4.7: Recommendations

4.9 Database Design

StockSense stores all business data in a single PostgreSQL database shared by all tenants, in which every business table carries a tenant identifier. This design keeps the data of each business logically separate while allowing the platform to serve many small tenants on the same infrastructure at low cost. The database is designed around the variant and the store, because sales, inventory, forecasts and recommendations are all recorded at this level of detail. The design also keeps a complete history of sales, adjustments and transfers, so that forecasts can be retrained and decisions can be audited later.

4.9.1 ER Diagram

Figure 4.8 shows the entity relationship diagram of StockSense. Each tenant has its own users, stores and products, so all business data belongs to exactly one tenant. A product has many variants, such as sizes,

and the inventory entity records how many units of each variant a store holds. Sales are recorded for a variant at a store on a given date with a quantity, whether they come from the POS or from an uploaded file. The historical sales feed the forecast entity, which stores the predicted demand of each variant at each store for a weekly or monthly period. Forecasts are in turn used by the recommendation entity, which holds redistribution, reorder and markdown recommendations together with their explanation, estimated impact and status, and which refers to the store or stores involved. A redistribution recommendation refers to both a source store with surplus stock and a target store with a forecast shortage, while reorder and markdown recommendations refer to a single store. This structure lets every query start from the tenant and narrow down to the store and variant, which supports both tenant isolation and fast access to the data that the dashboard needs.

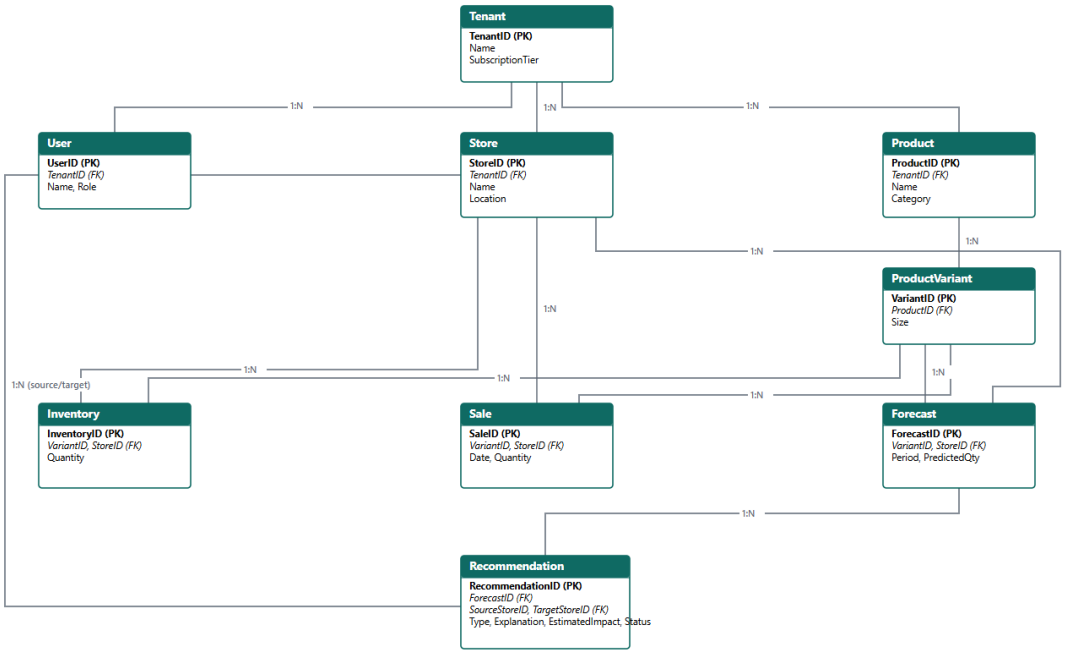


Figure 4.8: ER Diagram

4.9.2 Data Dictionary

The data dictionary below describes the main tables of the database, including the type of each field and its meaning. Identifiers are integers and act as primary or foreign keys, so that for example the store and variant of each sale can be traced back to their tenant. Only the most important tables are shown here, while supporting tables such as users, subscriptions, upload jobs, stock transfers and notifications are described in the class diagram in Chapter 6. Each table also includes the identifiers needed to enforce tenant isolation, either directly or through the store or product it belongs to.

Table 4.19: Data Dictionary for Tenant Table

Field	Type	Description
TenantID	Integer	Unique identifier for each business
Name	String	Business name
SubscriptionTier	String	Free or Paid
CreatedDate	Date	Date the tenant registered

Table 4.20: Data Dictionary for Store Table

Field	Type	Description
StoreID	Integer	Unique identifier for each store
TenantID	Integer	Owning tenant
Name	String	Store/branch name
Location	String	Store address or region

Table 4.21: Data Dictionary for Product Variant Table

Field	Type	Description
VariantID	Integer	Unique identifier for a product-size combination
ProductID	Integer	Parent product
Size	String	Size or variant label
Category	String	Product category

Table 4.22: Data Dictionary for Sale Table

Field	Type	Description
SaleID	Integer	Unique identifier for each sale
VariantID	Integer	Product variant sold
StoreID	Integer	Store where the sale occurred
Date	Date	Date of sale
Quantity	Integer	Units sold

Table 4.23: Data Dictionary for Recommendation Table

Field	Type	Description
RecommendationID	Integer	Unique identifier for each recommendation
Type	String	Redistribution, Reorder, or Markdown
VariantID	Integer	Product variant involved
SourceStoreID	Integer	Store with surplus (redistribution only)
TargetStoreID	Integer	Store with forecasted need
Explanation	String	Plain-language reasoning for the recommendation
EstimatedImpact	Float	Estimated financial impact
Status	String	Pending, Actioned, or Dismissed

4.10 Risk Analysis

Several risks could affect the success of StockSense, and each of them has been considered in the design. The main technical risk is that forecasts may be inaccurate for tenants with very limited sales history. This is reduced by training one global model for all series of a tenant, by showing model-based forecasts only after a minimum amount of history is available and by keeping simple baselines for comparison. A related data quality risk is that uploaded sales reports may be inconsistent or incomplete, which could require considerable cleaning effort and could lead to wrong forecasts. The validation and upload report described in Chapter 5 address this by rejecting invalid rows with clear reasons before any data is imported. There is also an adoption risk, because small business owners may be reluctant to trust automated recommendations without a visible track record, which is why every recommendation is explained and its estimated impact is shown. A scalability risk arises as more tenants and more historical data accumulate, since forecasting jobs must remain fast enough, so training runs in background jobs and results are cached. Finally, there are project risks such as changing requirements and limited access to real business data for testing, which the team will manage through short iterations, regular reviews with the supervisor and the use of public retail datasets.

4.11 Conclusion

This chapter presented the functional and non-functional requirements, quality attributes, assumptions, use cases, user interface and database design of StockSense. The requirements are built around three pillars identified in the project vision and the literature review. The first is affordable multi-tenant data collection through a free POS and the upload of existing sales reports. The second is demand forecasting at the product, size and location level, based on the model choice made in Chapter 3. The third is explained recommendations that give priority to redistribution between branches, followed by markdowns and reorders. Each requirement has an identifier that is traced to the use cases, the sequence diagrams and the test cases, and the non-functional requirements set measurable targets for performance, reliability and security. The database design keeps the data of each tenant separate and records sales, forecasts and recommendations at the level of the variant and the store. The next chapter describes the approach and methodology that will be used to meet these requirements.

Chapter 5 Proposed Approach and Methodology

This chapter presents the approach and methodology of StockSense and explains how the system turns raw sales data into inventory recommendations. It follows the data through every stage of the pipeline, starting with ingestion from the POS or from uploaded files and continuing with validation and cleaning, the construction of demand series, feature engineering, forecasting, the recommendation engine, markdown optimization and the explanation of each recommendation. It then describes how the forecasts and recommendations will be evaluated and how the work will be developed in increments. Each design decision in this chapter is based on the literature reviewed in Chapter 3, in particular the evidence in favour of global gradient-boosted models, the proactive approach to transshipment and the forecast-then-optimize approach to markdowns. The methods are described at a level of detail that allows them to be implemented directly in the second phase of the project and tested with the experiments planned in Chapter 9.

5.1 Overall Pipeline

StockSense is organized as a sequential pipeline in which each stage reads its input from the PostgreSQL database and writes its output back to it, through the modules described in Chapter 6. Sales enter the system either through the built-in POS or through file uploads and are stored in a single canonical schema, so the rest of the pipeline does not depend on where the data came from. A demand series is built for each product, size and store combination, features are computed from the history of each series, and the forecasting model produces weekly and monthly demand forecasts. These forecasts are compared with the current inventory to produce redistribution, markdown and reorder recommendations, each with an explanation and an estimated financial impact. Forecasting and recommendation run as background jobs, and their results are stored in the forecast and recommendation tables. Every forecast records the name of the model that produced it, so different models can be compared or replaced without changing any other part of the system. To respect tenant isolation, models are trained separately for each tenant, so one model is shared by all products, sizes and stores of a business but never across businesses.

5.2 Data Ingestion

Sales enter StockSense in two ways. Sales recorded through the built-in POS are stored as `Sale` and `SaleItem` records with the source set to POS, while sales imported from CSV or Excel files exported by an external system are stored in the same records with the source set to Upload. Because both sources share the same schema, all later steps of the pipeline work in the same way for both. CSV files are read

in chunks with the pandas library, and Excel files are read in batches with pandas and `openpyxl`, which keeps memory use low for files of up to 100,000 rows. After a file is uploaded, the system shows the columns it has detected, and the owner maps them to the canonical fields of product, size, store, date and quantity. Validation and import run as a background task, so the owner can continue working while a large file is processed. Valid rows are held in a buffer until the owner approves the upload report, and they are then imported in a single database transaction that is rolled back completely if any part of the import fails, so a failed upload never leaves partial data behind.

5.3 Data Validation, Cleaning, and Normalization

Every uploaded record passes through several checks before it is accepted. Structural checks confirm that all mandatory fields are present and have the correct data type. Value checks confirm that the date is valid, that the quantity is numeric and that the price is greater than or equal to zero. Because a date such as 03/04/2025 can be read in two ways, the owner chooses the date format while mapping the columns. Referential checks normalize the names of products, sizes and stores by trimming spaces, ignoring case and standardizing sizes, for example by turning “Medium” into “M”, and then require each record to match a variant and a store of the same tenant. Duplicate records are detected through a composite key made of the store, variant, date, quantity and price, but they are only flagged and not deleted, since identical sales can genuinely occur. Negative quantities are treated as returns, and extreme quantities are flagged as outliers and capped only in the copy of the data used for training, so the original records remain unchanged. Accepted records are saved in the standard format, while rejected and flagged records are listed in the upload report together with the reason for each decision, so the owner can correct the source file and upload it again.

5.4 Demand Series Construction

After cleaning, the sales are aggregated into weekly and monthly series for each variant and store. Weekly forecasts are made from weekly series and monthly forecasts from monthly series, so that the forecast horizon and the evaluation period always have the same time resolution. The demand of variant v in store s during period t is

$$y(v,s,t) = \sum_{i \in \mathscr{S}(v,s,t)} q_i, \tag{5.1}$$

where $\mathscr{S}(v,s,t)$ is the set of sale items of variant v in store s during period t, and q_i is the quantity of sale item i. A period with no sales is treated as zero demand only from the date on which the variant first appeared in that store, because before that date the demand is unknown rather than zero.

Series at the size level are often sparse and intermittent, with many periods of zero sales. Each series is therefore classified as smooth, erratic, intermittent or lumpy according to the Syntetos–Boylan classification, which uses the average inter-demand interval (ADI) and the squared coefficient of variation of the non-zero demands (CV^2), with the commonly used thresholds $ADI = 1.32$ and $CV^2 = 0.49$. This classification helps in choosing suitable baseline methods, such as Croston's method for intermittent series, and allows the forecast accuracy to be reported separately for each type of demand. A further problem is that observed sales underestimate the real demand when an item is out of stock, because customers who wanted it could not buy it. When the stock history is known, weeks in which the variant was unavailable for most of the time are therefore treated as censored and excluded from the training loss, so that the model does not learn that stockout periods represent low demand. Together these steps produce clean series that reflect demand as closely as the available data allows.

5.5 Feature Engineering

Each training sample describes one variant in one store at a forecast origin t, and its target is the demand at a later period $t + h$. The features are organized in several groups, each of which captures a different aspect of demand. Lag features contain the demand of the previous one to eight weeks, and of the same week one year earlier when at least a year of history is available, which captures the current level and yearly seasonality. Rolling statistics contain the average, the standard deviation and the share of weeks with zero sales over the last 4, 8 and 12 weeks, which capture level, volatility and intermittency. Calendar features include the week number, the month and flags for holidays and festivals, which capture seasonality and special events. Price features include the selling price, the depth of any discount and the price relative to the category average, which capture the effect of pricing on demand. Product and store attributes such as category, size, colour, store and city are used as categorical features, which lets the global model share patterns between similar series. Finally, lifecycle features record the number of weeks since the first and the last sale, which describe the age of the product. To prevent data leakage, features at origin t use only data that was available up to t, and only features that are known in advance, such as calendar events, may refer to future periods.

5.6 Demand Forecasting Model

This section describes the forecasting model, the loss function and forecast horizon, the training and tuning procedure, the statistical baselines, the handling of new tenants and the treatment of forecast uncertainty.

5.6.1 Global Gradient-Boosted Model

Individual series at the size level are too short and sparse to support a separate model for each series. Following the global models of Montero-Manso and Hyndman and the approach of Bandara et al. for groups of similar series, StockSense trains one global model per tenant on all of the tenant's series, with product, size and store included as categorical features. The selected model is LightGBM, an implementation of gradient-boosted decision trees. Gradient-boosted trees outperformed statistical methods in the M5 competition, and LightGBM handles missing values and categorical variables directly and trains quickly on tabular data, which keeps training costs low for a platform that has to serve many tenants. Training a single model per tenant also means that a newly added variant can benefit immediately from the patterns learned on similar products and stores, which is important for small businesses that frequently introduce new items with no sales history of their own.

5.6.2 Loss Function and Forecast Horizon

Demand is never negative and contains many zeros, so squared error is not a suitable loss function. The model therefore uses the Tweedie objective with a power parameter between 1 and 2, which describes a distribution with a point mass at zero and a continuous positive part, matching the shape of sparse sales data. The Poisson objective and squared error are kept as alternatives and will be compared in the experiments. Multi-step forecasts are made with a direct approach, in which a separate model is trained for each step $h = 1, \ldots, H$ of the forecast horizon, so that errors do not accumulate as they would with recursive predictions. The forecast for step h is

$$\hat{y}(v, s, t+h) = f_h\big(\mathbf{x}_{v,s,t}\big), \tag{5.2}$$

where $\mathbf{x}_{v,s,t}$ denotes the features of variant v in store s at time t.

5.6.3 Training and Tuning

The hyperparameters of the model, which include the number of leaves, the learning rate, the minimum amount of data per leaf, the feature and bagging fractions and the number of boosting rounds, are tuned with rolling-origin validation. In this procedure the training window expands step by step through time, the model is validated on the period immediately after each window, and early stopping is applied in every fold. Random shuffling of the data is never used, because it would let the model see future data during training and would give an overly optimistic picture of its accuracy. Sample weights are used to exclude censored stockout periods from the loss, as described in Section 5.4. Fixed random seeds are used for training so that results are reproducible, and the tuned settings are stored together with each

trained model so that every forecast can be traced to the model and settings that produced it.

5.6.4 Statistical Baselines

To check whether the more complex model is really necessary, several simple baselines are computed on exactly the same data splits. The naive forecast repeats the last observed value, while the seasonal naive forecast repeats the value of the same period one year earlier when at least one year of history is available. The moving average forecasts the mean of recent periods, and Holt–Winters exponential smoothing, computed with the `statsmodels` library, captures level, trend and seasonality. Croston's method is included for intermittent series, since it is designed for demand that is often zero. These baselines are cheap to compute and easy to explain, so they also serve as a fallback for tenants that do not yet have enough data for the global model. If LightGBM does not clearly beat the best baseline for a group of series, the simpler method will be preferred for that group.

5.6.5 Initial Training Period

StockSense does not ship with a pre-trained model, and the data of one tenant is never shared with another, so a new tenant starts without a forecasting model. The owner first records sales through the POS or uploads historical sales reports. Until a minimum amount of history is available, for example several weeks of sales for the tenant's products, no model-based forecast is displayed and the user is told that more data is needed, as described in UC-10. During this period, simple baseline forecasts may be shown if enough observations exist. Once the minimum is reached, the first global model is trained on the tenant's own data and model-based forecasting and recommendations become available. The model is then retrained on a schedule as new sales arrive, and its accuracy is measured rather than assumed to improve as the history grows. The minimum amount of history is determined experimentally, as described in Section 5.10.

5.6.6 Forecast Uncertainty

In addition to the point forecast, an upper quantile forecast is trained, for example the 0.9 quantile using the quantile objective of LightGBM. This forecast estimates a level of demand that will be exceeded only in about one period out of ten. The recommendation engine uses it to set a safety stock, so that its decisions take the uncertainty of demand into account and not only the expected value. Without a safety stock, the engine would treat any stock above the expected demand as surplus and could recommend transfers that leave the source store unable to cope with a normal rise in sales. The quantile forecast is trained with the same features and validation procedure as the point forecast, and its coverage will be checked on the test period to make sure that it is neither too cautious nor too optimistic.

5.7 Recommendation Engine and Stock Redistribution

For every variant v and store s, the recommendation engine compares the forecast with the stock position over a planning horizon of H weeks. Let I be the stock on hand plus any accepted incoming transfers that have not yet arrived, which are included so that they are not recommended again. Let D be the forecast demand over the next H weeks and $Q_{0.9}$ the upper quantile forecast over the same horizon, so that the safety stock above the expected demand is $SS = \max(0, Q_{0.9} - D)$. The net position of the store is then

$$N = I - (D + SS). \tag{5.3}$$

If $N > 0$, the store has a surplus of N units, and if $N < 0$, it has a shortage of $-N$ units.

Following the proactive transshipment approach of Paterson et al., transfers are planned before a stockout or dead stock occurs. For each variant, the transfer quantities x_{ij} from a surplus store i to a shortage store j are found by solving a transportation problem,

$$\max_{x} \quad \sum_{i}\sum_{j} \left(m_j - c_{ij}\right) x_{ij} \tag{5.4}$$

$$\text{s.t.} \quad \sum_{j} x_{ij} \leq \text{surplus}_i \quad \forall i, \tag{5.5}$$

$$\sum_{i} x_{ij} \leq \text{shortage}_j \quad \forall j, \tag{5.6}$$

$$x_{ij} \geq 0, \tag{5.7}$$

where m_j is the unit margin at the destination store j and c_{ij} is the unit transfer cost between stores i and j. A transfer is recommended only when $m_j - c_{ij} > 0$, that is, when the expected margin of the unit at the destination is larger than the cost of moving it. Transfer quantities must be whole units, and because the constraint matrix of a transportation problem and its supply and demand values are integers, the linear program has an integral optimal solution. Since small retailers have only a few branches, each problem is small and cheap to solve. Transfer lead times and capacity limits are not part of the model at present and are handled as assumptions in the backtest.

If a shortage remains after redistribution, because no other branch has enough surplus, it becomes a reorder recommendation with a suggested order quantity equal to the remaining shortage. Recommendations are always handled in a fixed order of priority, with redistribution first, markdowns second and reorders last, because using existing inventory is usually cheaper and faster than discounting it or buying new stock. This order is the main way in which StockSense differs from tools that consider each store on its own and treat transfers as only one option among many.

5.8 Markdown Optimization

Any surplus stock that is not removed by redistribution is passed to the markdown optimizer. Like the method of Caro and Gallien, the optimizer follows a forecast-then-optimize approach, but it is meant to act as an early warning during the season rather than as a clearance process at its end. A variant in a store is flagged as at risk when its weeks of cover are greater than the selling time that remains. When the forecast demand is positive, the weeks of cover are

$$\mathrm{WOC} = \frac{I}{\bar{d}} > T, \tag{5.8}$$

where $\bar{d}$ is the average weekly demand and T is the number of weeks within which the product needs to be sold. If $\bar{d} = 0$ and $I > 0$, the weeks of cover are undefined, so the item is treated as at risk and handled separately.

For each candidate discount d from a grid, for example from 5% to 50% in steps of 5%, the expected demand is adjusted with a constant price elasticity model,

$$D(d) = D \cdot (1-d)^{-\varepsilon}, \qquad \mathrm{Sold}(d) = \min\big(I,\, D(d)\big), \tag{5.9}$$

where $D(d)$ is the total adjusted demand over the horizon and ε is the price elasticity. The system recommends the smallest discount for which the projected sell-through reaches the target, subject to the margin floor $p\,(1-d) \geq c$, where p is the selling price and c is the cost price. A discount is recommended only if it does not lead to a lower expected margin than not discounting at all. Its impact is measured as the difference in margin between the two cases,

$$\mathrm{Impact} = \mathrm{Sold}(d)\,\big(p(1-d) - c\big) - \mathrm{Sold}(0)\,\big(p - c\big). \tag{5.10}$$

The elasticity ε is estimated with a log-log regression on the tenant's own price history when the prices of the product have varied enough. Otherwise a cautious estimate for the product category is used, so that the optimizer does not overestimate the effect of a discount for a business with little pricing history.

5.9 Explainable Recommendations

Each recommendation is stored together with an explanation made of two parts. The first part contains the numbers that describe the situation, which are the forecast demand, the stock level, the net position, the weeks of cover, the source and destination stores, the margins, the transfer costs and the estimated impact of the recommendation. The second part describes the main drivers of the forecast, which are

taken from the feature contributions computed by LightGBM through its `pred_contrib` option and translated into plain phrases such as “recent weekly demand” or “seasonality”. Both parts are inserted into text templates that produce short sentences such as “Size M of this item is forecast to sell 12 units at Store B over the next 4 weeks, but only 3 are in stock, while Store A holds 15 units and is forecast to sell only 4”. The explanation is created and saved at the same moment as the recommendation, so it always matches the situation that led to it, even if the stock or the forecast changes later. Because the main part of the explanation comes from the actual quantities used by the decision rule rather than from an approximation of the model, it accurately describes why the recommendation was made, which addresses the concern raised by Rudin about explanations of black box models.

5.10 Forecast and Recommendation Evaluation

This section describes the data, protocol, metrics and experiments used to evaluate the forecasts, and the historical backtest used to evaluate the recommendations. The detailed experiment design and the result tables are given in Chapter 9.

5.10.1 Data

Public retail datasets will be used for the first development and validation of the models. A dataset such as M5 contains sales and prices for items in individual stores, which allows evaluation at the store level and at higher levels of the product hierarchy, but it does not contain sales by size. Behaviour at the size level will therefore be evaluated on real sales data from a small retailer if access to such data can be secured. Sample tenants with two or three branches will also be created to test the full pipeline from upload to recommendation. The findings from public data will be interpreted with the limitation in mind that a large retailer’s data may not reflect the behaviour of small businesses, as noted among the constraints in Chapter 2. This combination of public, real and sample data allows each part of the system to be tested on the most suitable data available.

5.10.2 Protocol

All data splits are chronological, so the models are always trained on earlier data and tested on later data, just as they will be used in practice. Hyperparameters are tuned with rolling-origin validation on an expanding window, and the final comparison between models is made on the most recent period, which none of the models sees during training or tuning. Every model and baseline is evaluated on exactly the same splits, so the differences in their results come from the models themselves and not from differences in the data. Random seeds are fixed so that each experiment can be repeated, and the results

are reported for weekly and monthly horizons separately. This protocol avoids the common mistake of evaluating forecasts on data that leaked into training, which would make the models appear more accurate than they really are.

5.10.3 Metrics

The primary accuracy metrics are

$$\text{RMSE} = \sqrt{\frac{1}{n}\sum_{k=1}^{n}\left(a_k - f_k\right)^2}, \tag{5.11}$$

$$\text{MAPE} = \frac{100}{|P|}\sum_{k\in P}\frac{|a_k - f_k|}{a_k}, \qquad P = \{k \mid a_k > 0\}, \tag{5.12}$$

where a_k is the actual demand and f_k is the forecast.

Series at the size level contain many zeros, and MAPE is undefined when the actual demand is zero. MAPE is therefore computed only on periods with positive actual demand, and the share of periods that had to be excluded is reported alongside it. WAPE, which divides the total absolute error by the total actual sales, and MAE are used as additional measures because they remain valid for sparse series. Results are reported separately for each demand class and for fast-moving and slow-moving items, because an average over all series can hide poor accuracy on exactly the sparse series that matter most for dead stock.

5.10.4 Experiments

Four experiments are planned to test the main modeling choices. The first compares LightGBM with the statistical baselines, which tests the main claim drawn from the literature in Chapter 3. The second compares the global model with separate models for each series, to check whether sharing information across series improves accuracy for short histories. The third measures forecast accuracy against the amount of training history available, for example 4, 8 and 12 weeks, to estimate how much data a new tenant must provide before forecasts become reliable, which sets the minimum history used in the initial training period. The fourth compares the Tweedie and Poisson objectives with squared error, to confirm the choice of loss function for sparse and non-negative demand. Together these experiments test each of the main decisions in the forecasting design rather than only the final model.

5.10.5 Recommendation Backtest

The value of the recommendations is estimated by replaying past sales. At each cut-off date, the engine produces recommendations using only the information available up to that date. The recommended transfers and markdowns are applied to a simulated stock record, and the following weeks are replayed

with the actual sales as demand. The simulation includes a transfer lead time, assumes that stock is unavailable at both stores while it is in transit and applies an accepted markdown from its effective date. Transfers are assumed not to change demand, while markdowns change demand according to the elasticity model. The outcome is compared with a run in which no action is taken, using the number of dead-stock units whose cover exceeds T weeks, the units of sales lost to stockouts and the sell-through rate. This backtest measures the effect of the recommendations on business results, which is what matters to a retailer, rather than only the accuracy of the forecasts behind them.

5.11 Iterative Development

StockSense is developed in increments, and each increment ends with the testing activities described in Chapter 7 and a review by the supervisor. The first increment builds the multi-tenant infrastructure, which covers tenancy, users, products, inventory, the POS and the ingestion and validation pipeline. The second increment consists of offline forecasting experiments in Jupyter Notebook, in which the features, baselines and LightGBM model are developed and compared on historical data. The third increment integrates the selected model into the platform as a batch forecasting job and adds the recommendation engine and redistribution. The fourth increment adds markdown optimization, explanations, notifications and the dashboard views for forecasts and recommendations. Developing the forecasting models offline first makes it possible to choose the model on the basis of measured results before it is built into the system, and it means that the platform can be tested end to end with simple baselines while the models are still being developed. Each increment produces a working version of the system that can be demonstrated and tested on its own.

5.12 Conclusion

This chapter described how StockSense turns each recorded or uploaded sale into an inventory decision. The data is first validated and standardized and then aggregated into demand series for each product, size and store. A global LightGBM model trained on each tenant's own data forecasts weekly and monthly demand and is compared with simple statistical baselines, while new tenants first provide enough sales history before model-based forecasts are shown. The forecasts are then turned into actions in a fixed order, with redistribution solved as a small transportation problem, early markdowns chosen through a sell-through model with a margin floor, and reorders covering any remaining shortage. Every action carries an explanation and an estimated financial impact. The approach will be evaluated with chronological splits and a historical replay of sales, and the next chapter presents the design that implements it.

Chapter 6 High-Level and Low-Level Design

This chapter describes the design of StockSense at two levels. The high-level design presents the overall system, its modules and the architecture that connects them, while the low-level design presents the domain model, the sequence of interactions for each use case and the policies and tactics that will guide the implementation. The chapter also lists the assumptions, constraints and goals that shaped the design, so that the reasons behind each decision are clear before implementation starts.

6.1 System Overview

StockSense is a multi-tenant SaaS solution that helps small and medium-sized retailers reduce dead stock and stockouts by making use of their own sales data. Every business is an independent tenant with its own users, stores, products, inventory and sales. Sales are recorded through the built-in POS or imported from uploaded CSV or Excel files. The system forecasts demand at the product, size and store level and recommends actions, each with a clear explanation, on a single web dashboard. The system is divided into the modules described in the following subsections. Each module has one clear responsibility and communicates with the others through the database and well-defined interfaces, which allows the modules to be developed, tested and replaced independently.

6.1.1 Tenant and User Management

This module handles the registration of businesses, user login and the roles of users. When a business registers, the module creates a tenant, a free subscription and an owner account in a single transaction. It then makes sure that an owner, a store manager or an administrator can access only the data that belongs to their own tenant and that their role allows. Every request that reaches the backend carries the tenant identifier and the role of the user, and this module checks them before any data is read or changed. It also handles the invitation of store managers by the owner and the suspension and reactivation of tenants by the administrator, so that users of a suspended tenant cannot log in until the tenant is reactivated.

6.1.2 POS and Inventory Management

This module forms the free part of StockSense and is the main way in which sales data reaches the platform. It records sales and keeps the current stock of every product, size and store up to date. A sale and the resulting decrease in stock are saved in the same database transaction, so the inventory always matches the recorded sales. The module also supports manual adjustments after a physical count, which must be given a reason and are logged, and it completes the stock transfers created when a redistribution recommendation is accepted. Because store managers use it during business hours, it is designed to work

quickly and to remain available even when other modules, such as forecasting, are busy or temporarily unavailable.

6.1.3 Data Ingestion and Cleaning

This module imports historical sales from CSV or Excel files, validates the data, identifies invalid records and converts the data into the standard format used inside the system. It allows businesses that already use another POS or a spreadsheet to join StockSense without changing their current system, and it gives them a forecasting model sooner because their history is available from the first day. The module shows the detected columns so that the owner can map them to the required fields and choose the date format, runs the checks described in Chapter 5 in the background and presents an upload report with the accepted and rejected rows before anything is imported. Approved rows are imported in a single transaction, so a failed upload never leaves partial data in the database.

6.1.4 Demand Forecasting

This module forecasts the weekly and monthly demand of each product, size and store. It trains a global LightGBM model on each tenant's own sales history, together with the statistical baselines used for comparison, and stores the forecasts with the name of the model that produced them. A new tenant must first record or upload enough sales data before model-based forecasts are shown, and during this period the module reports that more data is needed. The module runs as a scheduled background job, so retraining and forecasting do not slow down the rest of the system. If a forecasting run fails, the previous valid forecasts are kept so that the dashboard and the recommendations can continue to work.

6.1.5 Stock Redistribution and Reorder Recommendations

This module compares the forecast demand with the current stock of each branch and calculates whether each store has a surplus or a shortage of each variant, taking a safety stock based on the upper quantile forecast into account. It first recommends moving stock from branches with surplus to branches with a shortage, choosing the transfers by solving a small transportation problem that maximizes the expected margin after transfer costs. Only when the surplus at other branches is not enough to cover a shortage does it recommend a reorder for the remaining quantity. Each recommendation is saved with its explanation and estimated impact, and when a redistribution is approved the module creates a stock transfer and notifies both stores.

6.1.6 Markdown Optimization

This module identifies slow-moving stock and suggests discounting it early enough for it to sell, before it becomes dead stock. It works on the surplus that remains after redistribution, flags items whose weeks of cover are greater than the selling time left, and evaluates a grid of discounts with a price elasticity model. It recommends the smallest discount that reaches the sell-through target without dropping the price below the cost price and without reducing the expected margin. Items with no forecast demand but with stock on hand are flagged separately. Like the other recommendations, each markdown recommendation carries an explanation and an estimate of its effect on margin, and the owner decides whether to apply it.

6.1.7 Explainable Recommendations

Every recommendation produced by StockSense is accompanied by a short explanation of why it is recommended and an estimate of its financial impact, so that the owner can understand and trust it before acting on it. The explanation is built from two sources, namely the actual quantities used by the decision, such as forecast demand, current stock, transfer costs and margins, and the main drivers of the forecast, which are taken from the feature contributions of the LightGBM model. Both are turned into plain sentences through text templates. The explanation is saved at the same time as the recommendation, so it always describes the situation that led to the recommendation even if the data changes afterwards.

6.1.8 Dashboard and Reporting

This module presents sales, inventory, forecasts and recommendations to owners and store managers in an easy-to-use web interface. Owners see consolidated information for all branches, while store managers see only their own store. The dashboard shows monthly sales, items at risk of stockout or dead stock, recent sales and the inventory health of each branch, and on the paid plan it also shows forecasts next to past sales and the list of open recommendations, which can be approved or dismissed. Free tenants see an upgrade notice in place of the paid sections. Because forecasts and recommendations are computed in the background and stored, the dashboard only has to read them, which keeps it fast even for tenants with large catalogs.

6.2 Design Considerations

This section lists the assumptions, dependencies, constraints and goals that affect the design of StockSense, so that they are understood and handled before implementation starts.

6.2.1 Assumptions and Dependencies

The design depends on assumptions about the environment of the users, their skills and the possibility that the requirements will change during development.

6.2.1.1 Operating System and Browser

StockSense is a web application, so it does not depend on any particular operating system on the user's side. Users need a recent version of Chrome, Edge, Firefox or Safari on a laptop, desktop or tablet. The server side depends on a Linux environment with Python, PostgreSQL and the forecasting libraries described in Chapter 4, and on a hosting provider that offers enough memory and processing power for the background jobs. Supporting only modern browsers allows the interface to use current web standards, while the server-side design keeps all heavy computation away from the user's device, so that even an older machine at the shop counter can run the POS and the dashboard comfortably. These dependencies are standard and widely supported, which reduces the risk that the system will be difficult to deploy or maintain.

6.2.1.2 End User Characteristics

The design assumes that users have a stable internet connection, since the POS and the dashboard run in the browser and store their data on the server. Owners and managers are assumed to know how to use a computer for everyday tasks but not to have advanced technical skills or any training in data analysis. This assumption shapes much of the design, including the plain-language explanations, the hiding of statistical terms and the simple flows for recording a sale or uploading a file. It is also assumed that the retailer either has access to historical sales data, for example from a previous POS or from spreadsheets, or is willing to record new sales through the built-in POS until enough history is available for forecasting.

6.2.1.3 Possible Changes in Functionality

The quality of the forecasts depends on the quality and quantity of the sales data, which cannot be fully known before the models are tested. The functionality of the system may therefore change during development in response to the results of the experiments on the datasets and to the feedback of the supervisor. For example, the forecasting model may be replaced for some types of series if a baseline proves more accurate, the minimum amount of history required from new tenants may change, and the recommendation rules may be adjusted after the backtest. The modular design is intended to make such changes possible without affecting the rest of the system, since each module communicates with the others only through stored data and defined interfaces.

6.2.2 General Constraints

The design must also respect several constraints that come from the environment in which StockSense will run, the data it receives and the security and performance it must provide.

6.2.2.1 Hardware and Software Environment

Forecasting and data processing need considerably more computing power than ordinary page requests, so the design separates these tasks into background jobs that can be scheduled and scaled independently of the web application. The interface must work on a range of devices and browsers, which requires a responsive layout and careful testing on different screen sizes. The use of external libraries such as LightGBM, statsmodels, pandas and openpyxl also places some constraints on the implementation, for example on the supported file formats and on the versions of Python that can be used, and these libraries must be kept up to date to receive security fixes and performance improvements. The background jobs must also be monitored, since a job that fails silently could leave a tenant with outdated forecasts. The design therefore records the status of every job and alerts the team when a job fails repeatedly.

6.2.2.2 End-User Environment

Some shops may have slow or unstable internet connections, so the POS must keep its requests small and must show clearly whether a sale has been saved. Long-running tasks such as uploads and forecasting must continue on the server even if the user's connection drops, and the user must be able to check their status later. Many owners and managers have no training in data analysis, so the interface must avoid technical language and present forecasts and recommendations as simple business statements. These constraints affect both the network design of the application and the wording of every screen and message that users see. The design therefore assumes the least favourable conditions for the interface, so that it works well in every shop rather than only in those with good equipment and connections. Messages and labels are tested with users who have no technical background.

6.2.2.3 Data Format Differences

Files uploaded by different businesses vary in their column names, date formats and product naming. One file may call a column "Qty" and another "Quantity", one may use day-first dates and another month-first dates, and sizes may be written as "Medium", "Med" or "M". All of these must be converted into a single internal format before the data can be used for forecasting. The design handles this with a column mapping step, a date format choice made by the owner and normalization rules for product, size and store names, and it reports every row that cannot be matched so that the owner can correct it.

This keeps the forecasting pipeline independent of the many formats used by external systems. Once a mapping has been defined for a tenant, it can be reused for later uploads in the same format.

6.2.2.4 Data Repository and Tenant Isolation

No business must ever be able to see the data of another business, which is the most important constraint on the data repository. Every record that belongs to a business is therefore associated with a tenant, and every query is restricted to the tenant of the user who makes the request. The database must also be able to grow as more tenants join and as each tenant's sales history becomes longer, without a loss of performance. This is addressed by indexing the tenant, store and variant identifiers that appear in almost every query and by storing computed forecasts so that they do not have to be recalculated on each request. Isolation is enforced in the backend rather than only in the interface, so that a user cannot reach another tenant's data even by sending requests directly to the API. Backups and logs must follow the same rule, so that no copy of the data becomes a way around it.

6.2.2.5 Security Requirements

Each user logs in with a personal account and is responsible for keeping the password private, while the system stores passwords only as secure hashes. Access to data is based on the user's role, so store managers can work only with their assigned store and the system administrator can manage tenants without access to their detailed sales. Data must be protected both in transit, through encrypted connections, and at rest, through access controls and regular backups. The implementation must also follow secure coding practices such as parameterized queries and keeping secret keys out of the source code, and tenant isolation must be tested explicitly for any possible leakage between businesses. Sessions expire after a period of inactivity, which reduces the risk on shared shop computers.

6.2.2.6 Performance Requirements

Forecasting and the processing of large uploads must run in the background so that they never block the POS or the dashboard. Results that are used often, such as the latest forecasts and open recommendations, are stored and read directly instead of being calculated again on every request. Invalid input must produce clear error messages quickly rather than long waits followed by failures. The specific targets for performance, such as saving a POS sale in under two seconds and processing a file of 100,000 rows in under five minutes, are given in Chapter 4 and are checked by the non-functional test cases in Chapter 7. Pages that show long lists, such as the inventory page, will load their data in pages rather than all at once. These measures keep the system responsive as the data of each tenant grows over time.

6.2.2.7 Interoperability Requirements

StockSense must be able to import CSV and Excel files exported by external POS systems and spreadsheet programs, since this is how existing businesses bring their history into the platform. Internally, the modules must communicate through clear and documented interfaces, so that each one can be developed and tested separately and so that further sources of data, such as direct integrations with popular POS systems, can be added in the future without changing the forecasting and recommendation modules. The API that serves the web interface also follows a consistent structure for this reason. Keeping the interfaces stable also means that the web dashboard and any future mobile application can use the same API. This reduces duplicated work and keeps the behaviour of the system consistent across different clients.

6.2.2.8 Verification and Validation Requirements

The upload pipeline must be tested with faulty and damaged files to make sure that bad data is rejected without harming existing data. Tenant separation must be tested specifically for any possible data leakage between businesses or between the stores of one business. Forecasts must be evaluated against historical data that the models have not seen, following the protocol described in Chapter 5, and the recommendations must be evaluated through the historical backtest. Each functional requirement is linked to at least one test case, so that the verification of the system can be traced back to the requirements. These verification activities are planned from the start of development rather than added at the end, so that problems are found while they are still cheap to fix. The results will be recorded in Chapter 7 and Chapter 9 in FYP-2.

6.2.3 Goals and Guidelines

The design follows four goals that guide decisions whenever several options are possible.

6.2.3.1 The KISS Principle

The design must remain as simple as possible, following the principle of keeping things simple, so that the system is easy to maintain for developers and easy to use for non-technical users. Wherever a simple method gives similar results to a complex one, the simple method is preferred, which is why the redistribution problem is solved as a small linear program and the markdown optimizer uses a simple elasticity model. Simplicity also makes the recommendations easier to explain, since a rule that can be stated in one sentence is much easier for an owner to trust than the output of a complex procedure. Simple designs are also easier to test thoroughly, which matters for a small team with limited time.

6.2.3.2 Separation of Tenant Data

Each business must have its own secure space for its data, and this goal takes priority over convenience or performance. Models are trained per tenant, all data carries a tenant identifier, and every request is checked against it. Even features that could benefit from shared data, such as benchmarking across tenants, may use only aggregated and anonymized information, and only if enough tenants are available to prevent any single business from being identified. This goal reflects the fact that small businesses will only share their sales data with a platform they trust. Any feature that would weaken isolation is therefore rejected, even if it could improve the forecasts, and the team will review every new feature against this goal before it is added to the system.

6.2.3.3 Modularity and Extensibility

Forecasting, recommendations, the POS, data ingestion and markdown optimization must be kept separate so that, for example, a new forecasting algorithm can be added without changing the rest of the design. Each module reads its inputs from the database and writes its outputs back, and the forecasts carry the name of the model that produced them. This structure lets new models and recommendation rules be added and compared side by side, and it lets the team develop and test each module independently. It also limits the effect of errors, since a fault in one module, such as a failed forecast, does not stop the others from working. Modularity therefore supports both the maintainability and the reliability of the platform, which are two of the quality attributes defined in Chapter 4.

6.2.3.4 Explainability

Every recommendation must come with the reasoning behind it and an estimate of its financial impact. This goal affects the design of the recommendation engine, which keeps the quantities on which each decision is based, and of the database, which stores each explanation together with its recommendation. It also affects the choice of methods, since decision rules that can be expressed in plain language are preferred over methods whose results cannot be explained to a business owner. Explainability is treated as a requirement of every new feature rather than as an addition at the end, because a recommendation that cannot be explained will not be followed. For the same reason, the dashboard shows the explanation next to each recommendation instead of hiding it behind a separate link.

6.2.4 Development Methods

StockSense will be developed iteratively in phases. The first phase covers the tenant, product and inventory management modules, the POS and data ingestion, which together form the free tier and produce the

data needed by the later modules. The second phase covers forecasting, recommendations, markdown optimization and the dashboard views that present them, with the forecasting models first developed offline and then integrated into the platform. Each phase is divided into short iterations that end with a working version of the system, testing and a review with the supervisor. A phase is evaluated before the next one starts, so that problems found in testing can be fixed early and the plan can be adjusted to the results of the experiments.

6.3 System Architecture

StockSense follows a layered web architecture, shown in Figure 6.1. Users work with the web dashboard or the POS interface, which communicate with the backend through its API. The API verifies the identity of the user and the tenant and passes each request to the appropriate module. Cleaned sales data from the ingestion module and from the POS feeds the forecasting engine, and the forecasts, together with the current stock from the POS and inventory module, feed the recommendation engine. Surplus stock that cannot be redistributed is passed on to the markdown optimizer. All results are stored in the database and shown back to the user on the dashboard. Heavy tasks such as training, forecasting and large uploads run as background jobs outside the request cycle, so they never block the interactive parts of the system.

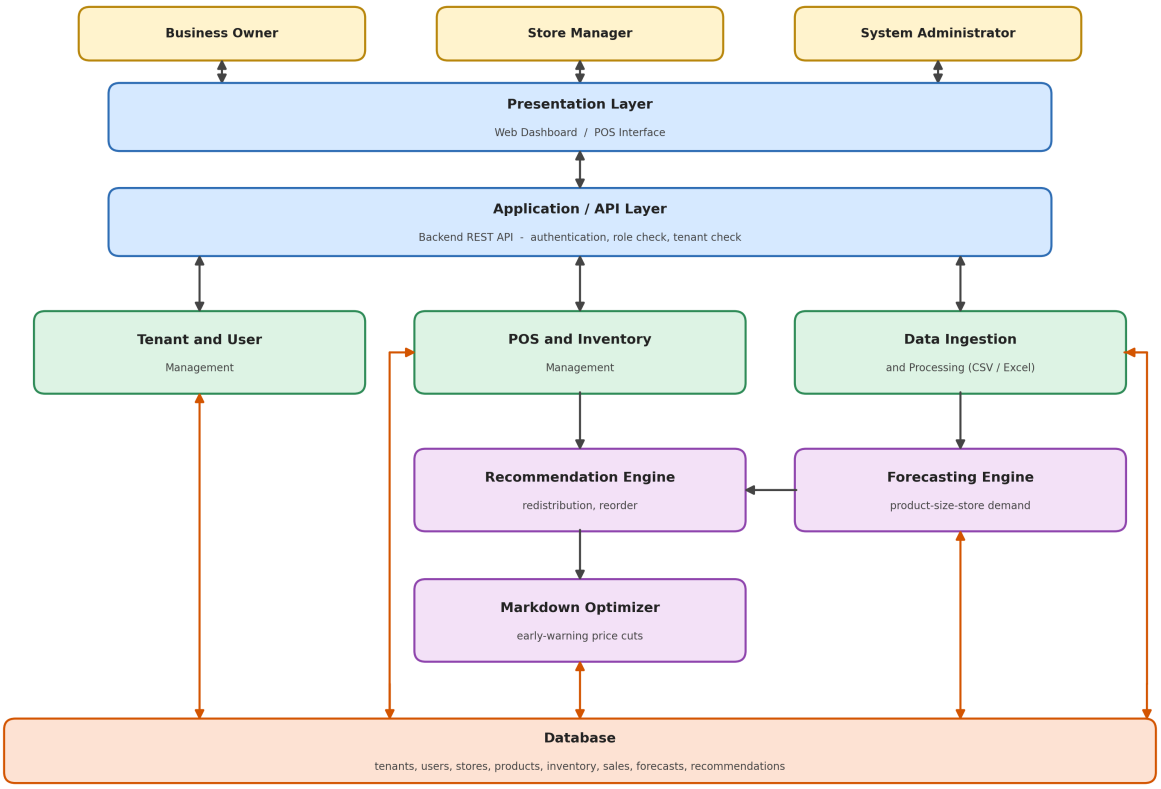


Figure 6.1: System Architecture Diagram
This figure shows the System Architecture Diagram of our Application

6.3.1 Subsystem Architecture

The architecture is made up of six subsystems. The user and tenant management subsystem stores businesses and their users, handles registration, login and roles, and makes sure that every user can access only the data of their own tenant. The POS and inventory subsystem maintains the current count of every item, size and store and records every sale. The data ingestion subsystem accepts CSV and Excel files, validates them, flags invalid records and converts the files into the standard internal format. The forecasting subsystem analyzes the sales history of each tenant and forecasts weekly and monthly demand at the product, size and store level, training a model on the tenant's own data and showing no forecast until enough history is available. The recommendation subsystem compares the forecasts with the stock of each branch, recommends transfers between branches first and reorders only when transfers are not enough, and stores each recommendation with its reason and estimated impact. Finally, the markdown optimization subsystem looks for stock that is not selling fast enough and recommends a price reduction early enough for it to sell. The subsystems share the database but do not depend on each other's internal logic, which keeps them independent.

6.4 Architectural Strategies

The architectural strategies below describe the main technical decisions that implement the design goals.

6.4.1 Multi-Tenant Architecture

Many businesses share the same platform, but their data remains logically separate. Every important record carries a tenant identifier, and every request is checked against the tenant of the logged-in user before any data is read or updated. This shared-database approach keeps the cost per tenant low, which is essential for a platform that offers a free tier to small businesses, while still guaranteeing that no business can see another's data. Forecasting models are also trained per tenant, so isolation applies to the models as well as to the stored data. The approach makes it easy to add new tenants without new infrastructure, and the tenant checks are concentrated in one layer of the backend so that they cannot be forgotten in individual modules.

6.4.2 Modular Architecture

The system is divided into the modules for forecasting, recommendations, the POS, data ingestion and markdown optimization. Each module has a single responsibility and communicates with the others through stored data and defined interfaces. If one of them changes, for example when a new forecasting model is introduced, the others do not need to change significantly. This also makes testing easier, since

each module can be tested on its own with prepared input data, and it allows team members to work on different modules at the same time. The modular structure follows directly from the maintainability and flexibility attributes in Chapter 4. The interfaces between the modules are kept small and documented, which makes it clear what each module expects as input and produces as output. This reduces misunderstandings between team members and makes integration testing more straightforward.

6.4.3 Programming Languages and Frameworks

The web interface will be built with React, which allows a responsive single-page interface for both the dashboard and the POS. The backend Representational State Transfer (REST) API will be developed in Python with FastAPI, which fits well with the Python data science libraries used for forecasting. LightGBM will provide the main forecasting model, statsmodels will provide the statistical baselines, and pandas with openpyxl will process CSV and Excel files. Using Python for both the API and the forecasting code means that the whole backend shares one language, which reduces the effort of integrating the models into the platform after the offline experiments. All of these tools are open source and widely used, which keeps the cost of the platform low and makes it easy to find documentation and support. The choice also matches the skills of the team, which reduces the time needed to learn new technologies during the project.

6.4.4 Database

All business data will be stored in a PostgreSQL relational database. Each table that contains business data has a tenant identifier, which supports the tenant isolation described in Section 6.4.1. A relational database suits StockSense because its data is highly structured and connected, with sales, inventory, forecasts and recommendations all linked to variants, stores and tenants, and because transactions are needed to keep sales and inventory consistent. The detailed design of the database is given in the database design section of Chapter 4, and indexes on the tenant, store and variant identifiers keep the most common queries fast. PostgreSQL is also open source, mature and well supported by hosting providers, which keeps the cost of running the platform low. Its support for transactions, constraints and indexes covers all the needs of the current design.

6.4.5 Background Processing

Forecasting and the cleaning of large files can take a long time. These jobs will run in the background, and the user will be notified when the results are ready, so that the dashboard and the POS remain fast. Background jobs are also scheduled to retrain the models and refresh the forecasts and recommendations regularly as new sales arrive. Running these jobs separately from the web requests means that a heavy

task for one tenant does not slow down the system for other tenants, and the jobs can be given more computing resources as the number of tenants grows. The status of each job is stored, so users can see whether an upload or a forecast is still running, has finished or has failed. This also allows the team to monitor the background work and to spot problems early.

6.4.6 Error Detection and Recovery

The system will log failed uploads, rejected records and failed forecasting runs with enough information to identify the cause. Uploads are imported in a single transaction that is rolled back on failure, so existing data is never left in a partial state. When a forecasting run fails, the previous valid forecasts remain on the dashboard and are not removed, so users can continue to work with the last good results until the problem is fixed and a new run succeeds. Failed background jobs are retried automatically, and repeated failures are reported so that they can be investigated. Logs will not contain sensitive business data or passwords, so that they can be inspected safely during support. Together these measures make failures visible and recoverable without loss of data.

6.5 Domain Model/Class Diagram

The domain model of StockSense is shown in Figure 6.2. It follows the data dictionary of Chapter 4, so every class that belongs to a business belongs to exactly one tenant, and the foreign keys of the database appear as associations between classes. A Tenant represents a business on the platform and has one or more Subscriptions over time, which record its history of free and paid plans. A Tenant owns many Stores, Products and UploadJobs and has many Users, whose role distinguishes a Business Owner, a Store Manager or a System Administrator, and the System Administrator belongs to no tenant. A Store Manager is assigned to one Store, while a Store can have several users assigned to it. A Product has one or more ProductVariants, such as sizes, each with its own SKU, and an Inventory record links one ProductVariant to one Store with a quantity. A Sale takes place at one Store and contains one or more SaleItems, each of which refers to one ProductVariant, and a Sale may come from an UploadJob or, for POS sales, from no upload at all. An UploadJob records an uploaded file with the number of accepted and rejected rows.

On the analytical side, a Forecast stores the predicted demand of one ProductVariant at one Store for a weekly or monthly period. Recommendation is an abstract class with three subclasses, Redistribution-Recommendation, ReorderRecommendation and MarkdownRecommendation. Every recommendation concerns one ProductVariant, refers to a Store, may be based on a Forecast and is accepted or dismissed by a User. A RedistributionRecommendation also has a destination Store and, when it is accepted, cre-

ates one StockTransfer, which records the movement of stock between two Stores. Finally, a Notification is received by one User and may refer to a Recommendation, for example to alert a store manager that a transfer involving the store has been approved. This model gives each part of the system a clear place in the data and makes the relationships needed for tenant isolation and for the recommendation workflow explicit.

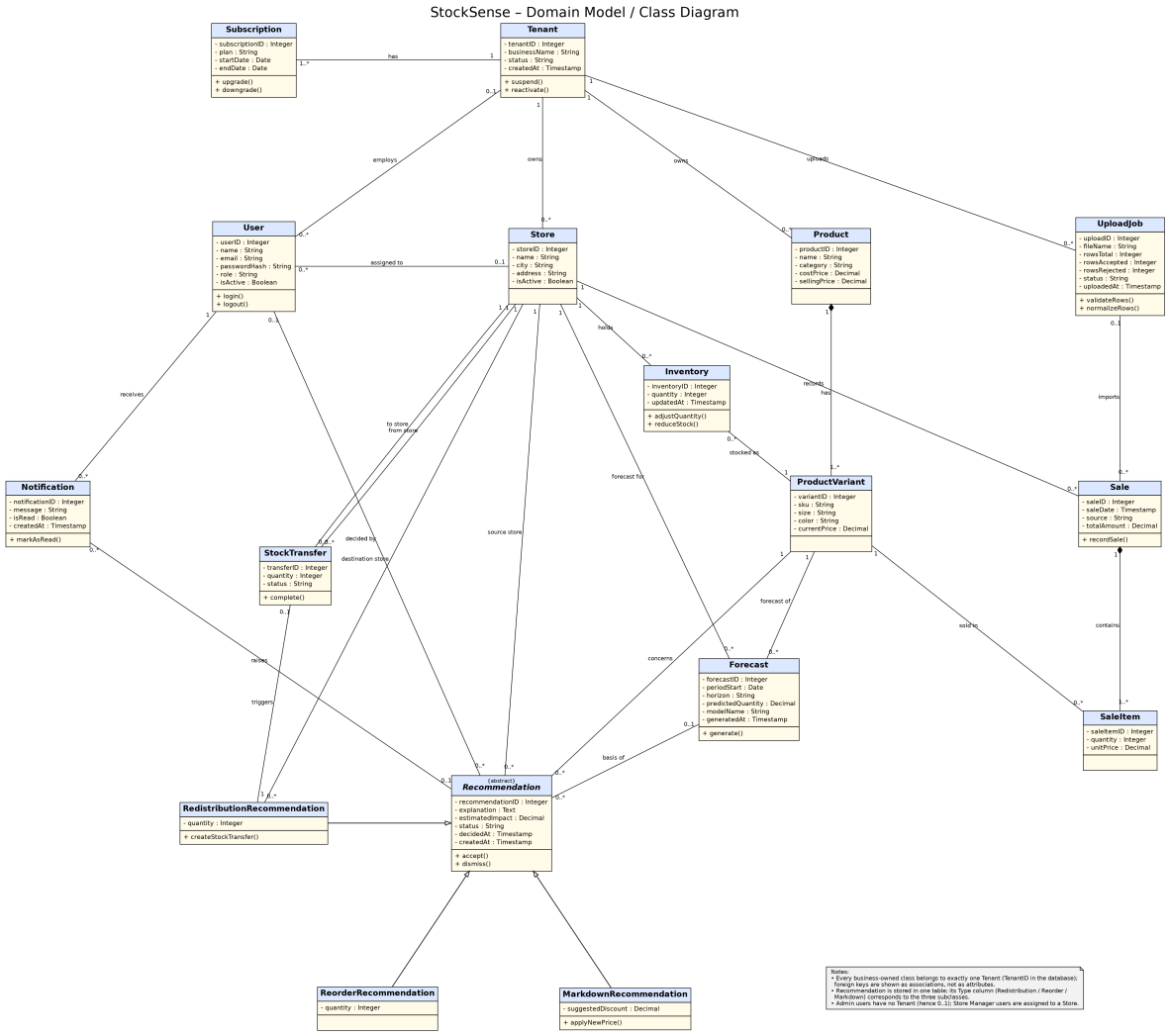


Figure 6.2: Domain Model / Class Diagram
This figure shows the Domain Model / Class Diagram of our Application

6.6 Sequence Diagrams

This section presents one sequence diagram for each use case described in Chapter 4, in Figures 6.3 to 6.17. Each diagram shows the order of the messages exchanged between the actor, the user interface, the application layer, which consists of the API controllers and services, and the database for the basic flow of the use case. Alternative flows are drawn as `alt` fragments, optional behaviour as `opt` fragments

and repeated steps as `loop` fragments, while operations that must succeed or fail together are shown as a single database transaction. Every request is restricted to the data of the user's tenant, as described in the multi-tenant architecture. The diagrams follow the order of the use cases, starting with registration, login and the management of stores, managers, products and inventory, continuing with the POS, the upload of historical data, the dashboard and the forecasts, and ending with the three types of recommendation and the management of subscriptions and tenants.

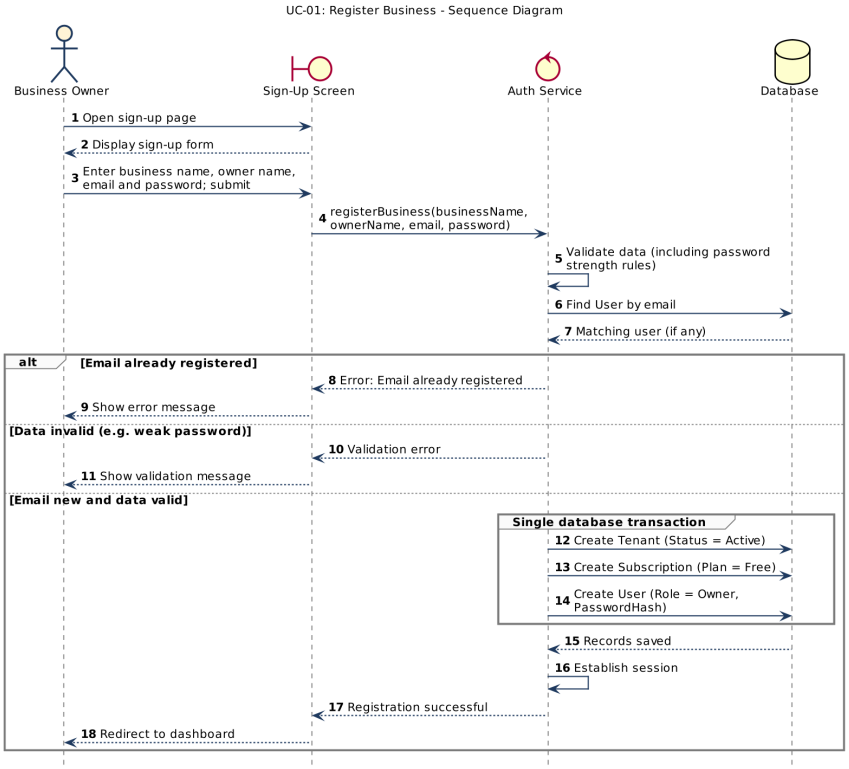


Figure 6.3: Sequence Diagram of UC-01 Register Business
This figure shows the sequence diagram of UC-01 (Register Business)

6.7 Policies and Tactics

The policies and tactics below describe how StockSense will be built, tested and maintained, and which practices the team will follow during implementation.

6.7.1 Tools to be Used

Several tools and libraries will be used to develop StockSense. The web dashboard and the POS screens will be developed with React, and the backend REST API will be implemented in Python with FastAPI or Django, where it will handle authentication, tenant checks and the communication between modules. All business data will be stored in PostgreSQL, with a tenant identifier in every table that holds the

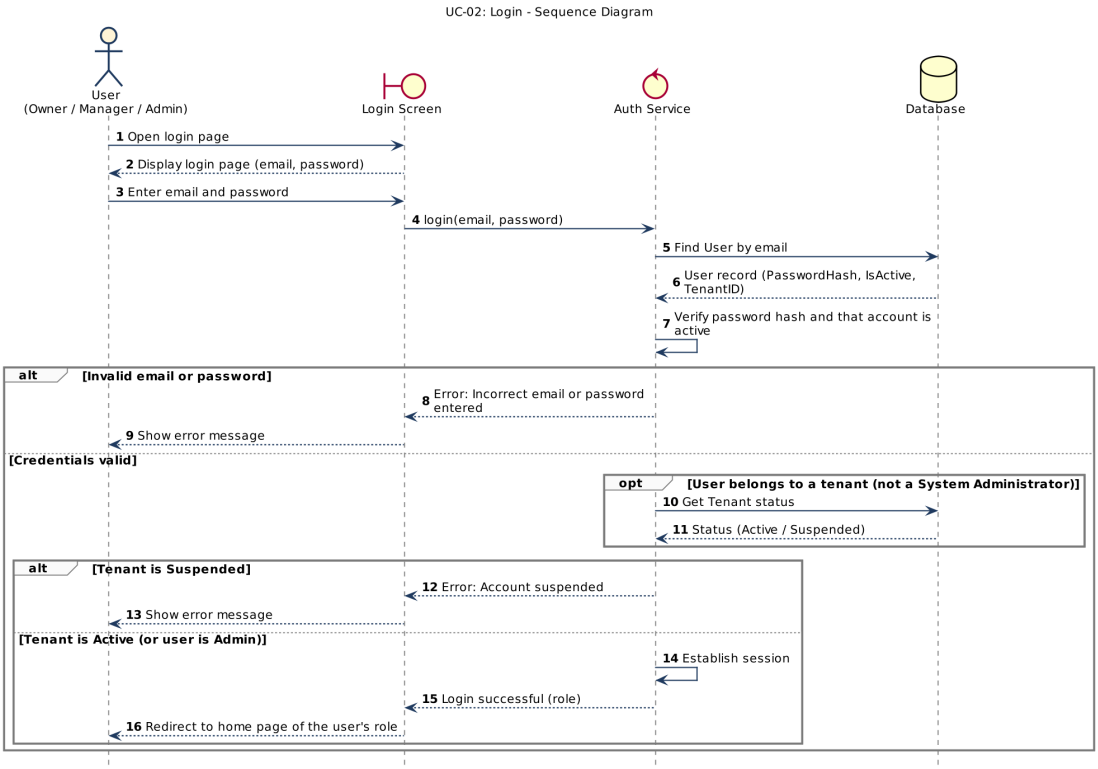


Figure 6.4: Sequence Diagram of UC-02 Login
This figure shows the sequence diagram of UC-02 (Login)

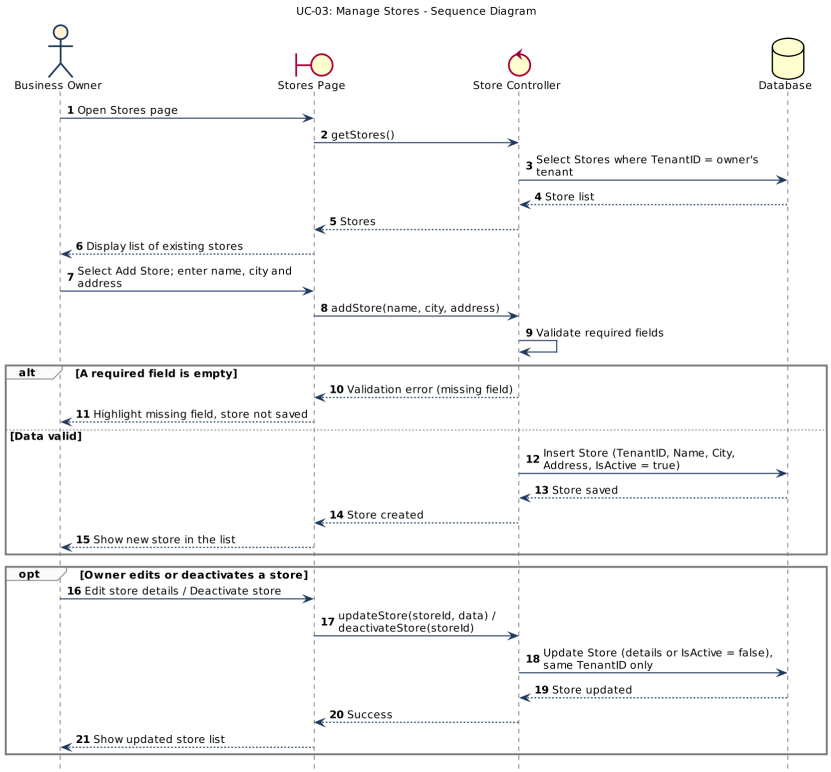


Figure 6.5: Sequence Diagram of UC-03 Manage Stores
This figure shows the sequence diagram of UC-03 (Manage Stores)

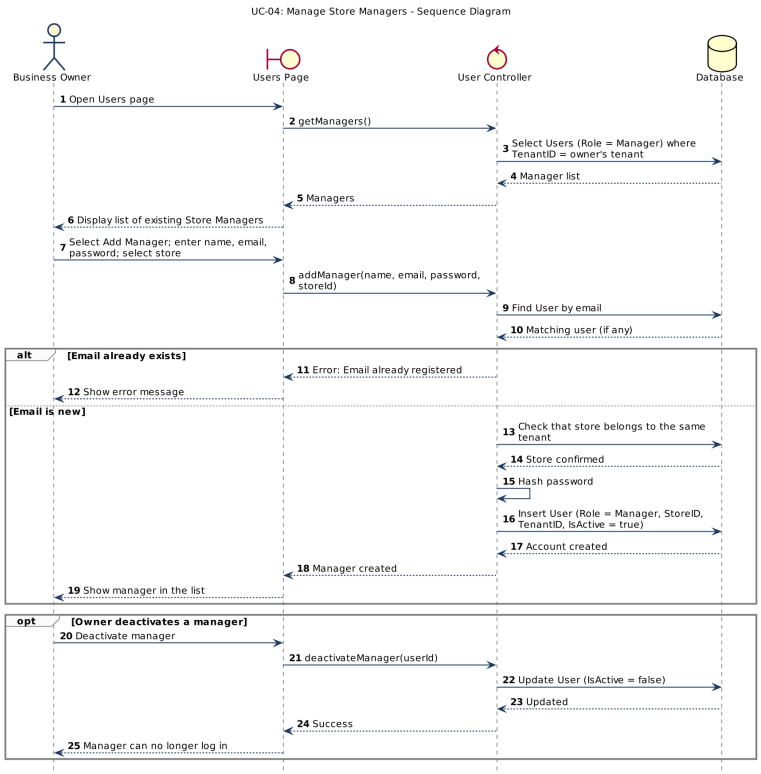


Figure 6.6: Sequence Diagram of UC-04 Manage Store Managers
This figure shows the sequence diagram of UC-04 (Manage Store Managers)

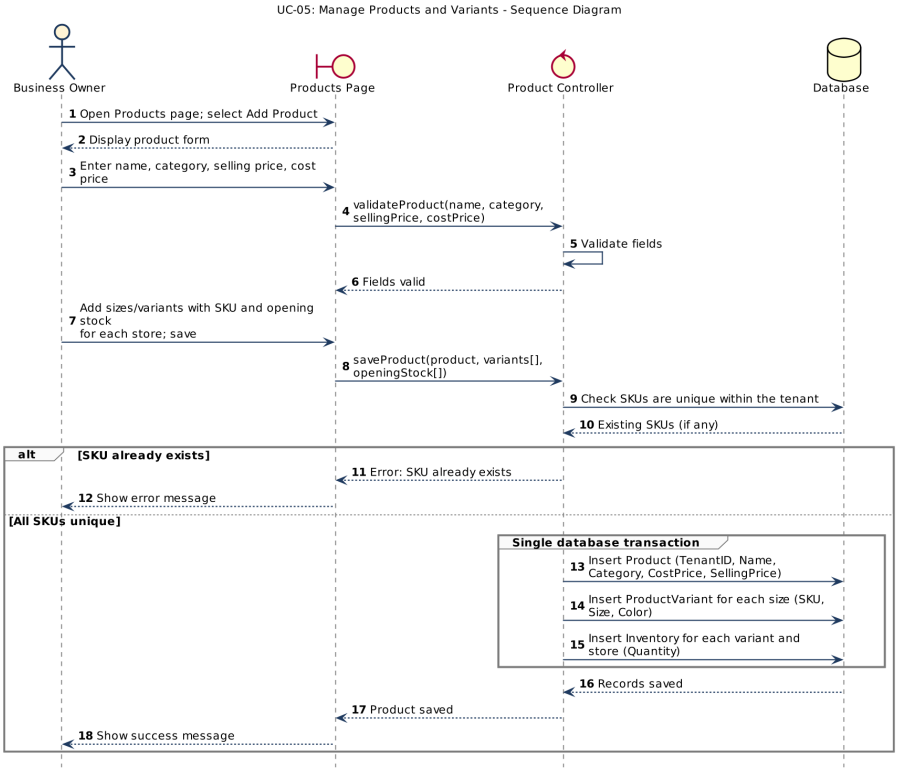


Figure 6.7: Sequence Diagram of UC-05 Manage Products and Variants
This figure shows the sequence diagram of UC-05 (Manage Products and Variants)

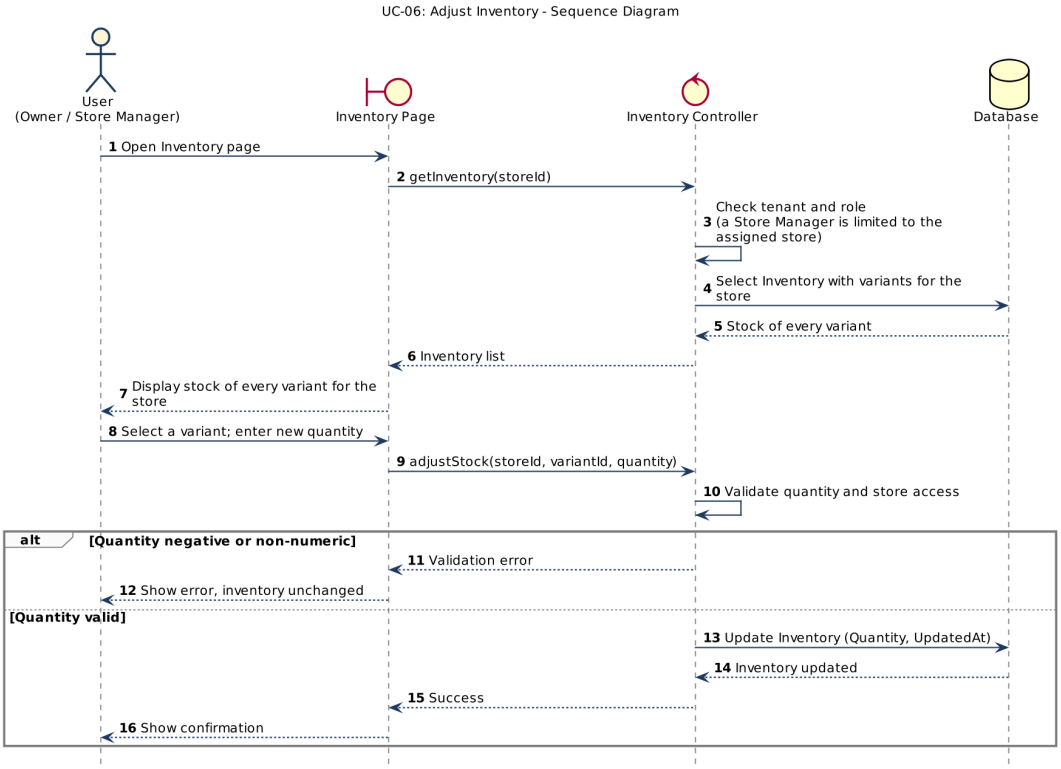


Figure 6.8: Sequence Diagram of UC-06 Adjust Inventory
This figure shows the sequence diagram of UC-06 (Adjust Inventory)

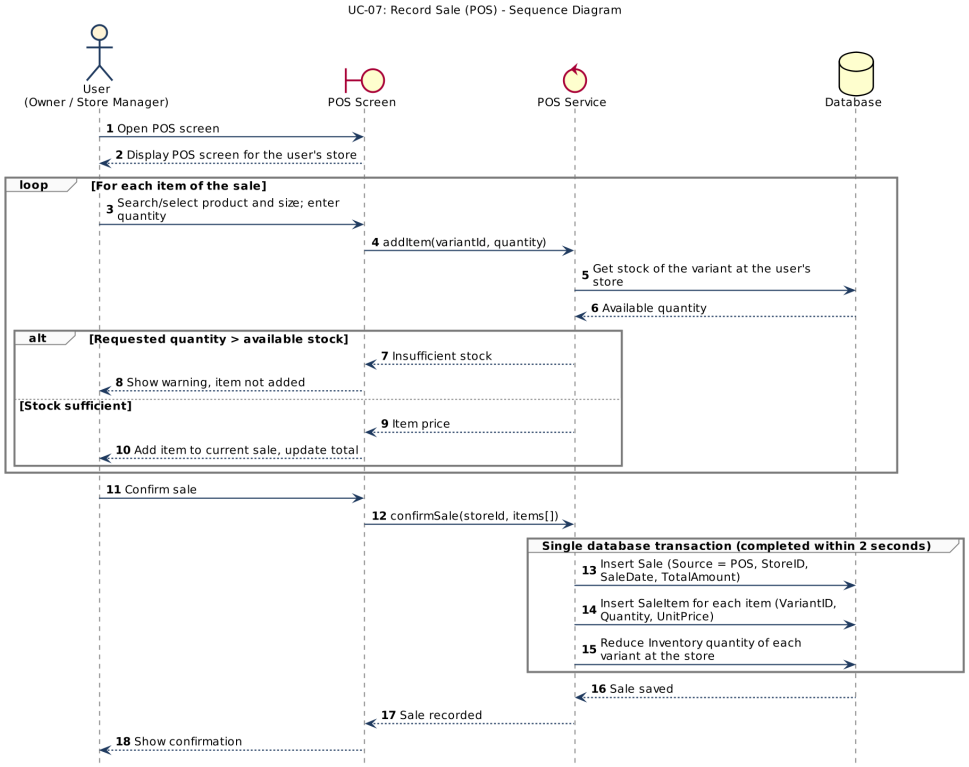


Figure 6.9: Sequence Diagram of UC-07 Record Sale (POS)
This figure shows the sequence diagram of UC-07 (Record Sale (POS))

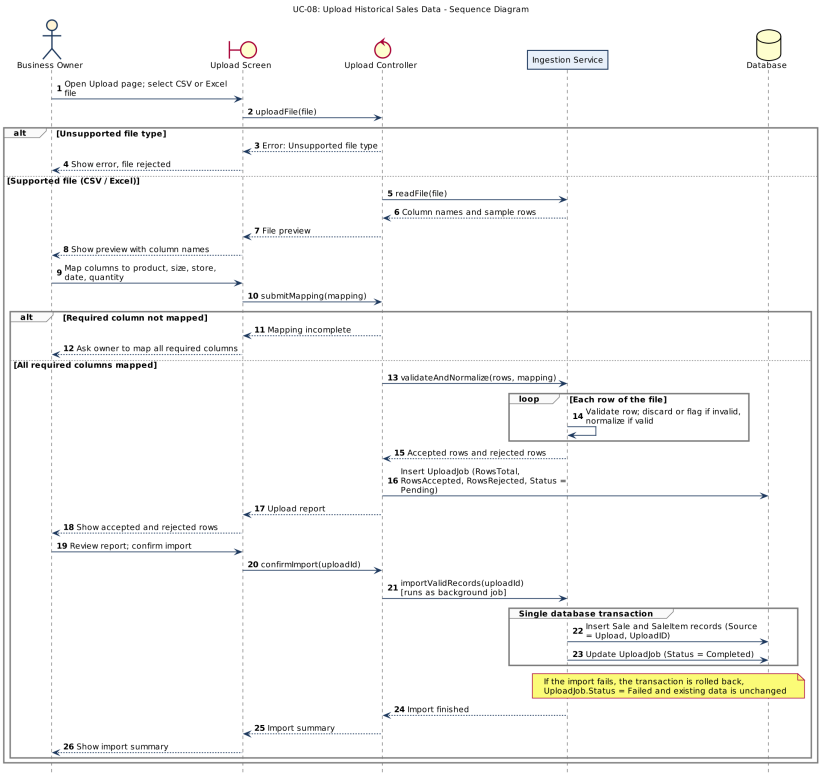


Figure 6.10: Sequence Diagram of UC-08 Upload Historical Sales Data
This figure shows the sequence diagram of UC-08 (Upload Historical Sales Data)

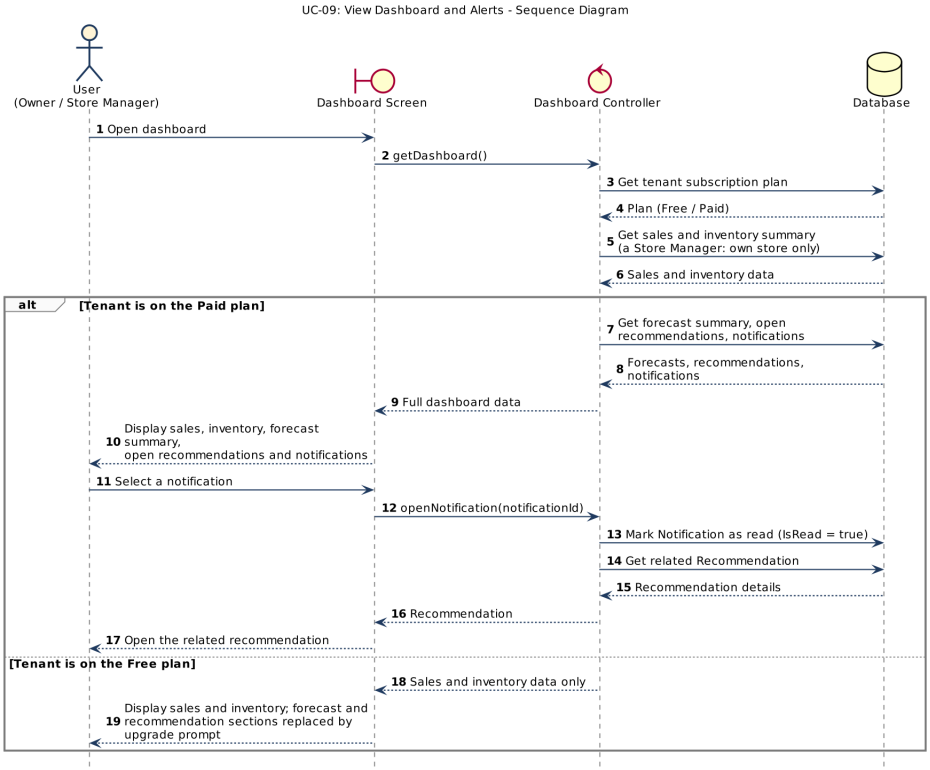


Figure 6.11: Sequence Diagram of UC-09 View Dashboard and Alerts
This figure shows the sequence diagram of UC-09 (View Dashboard and Alerts)

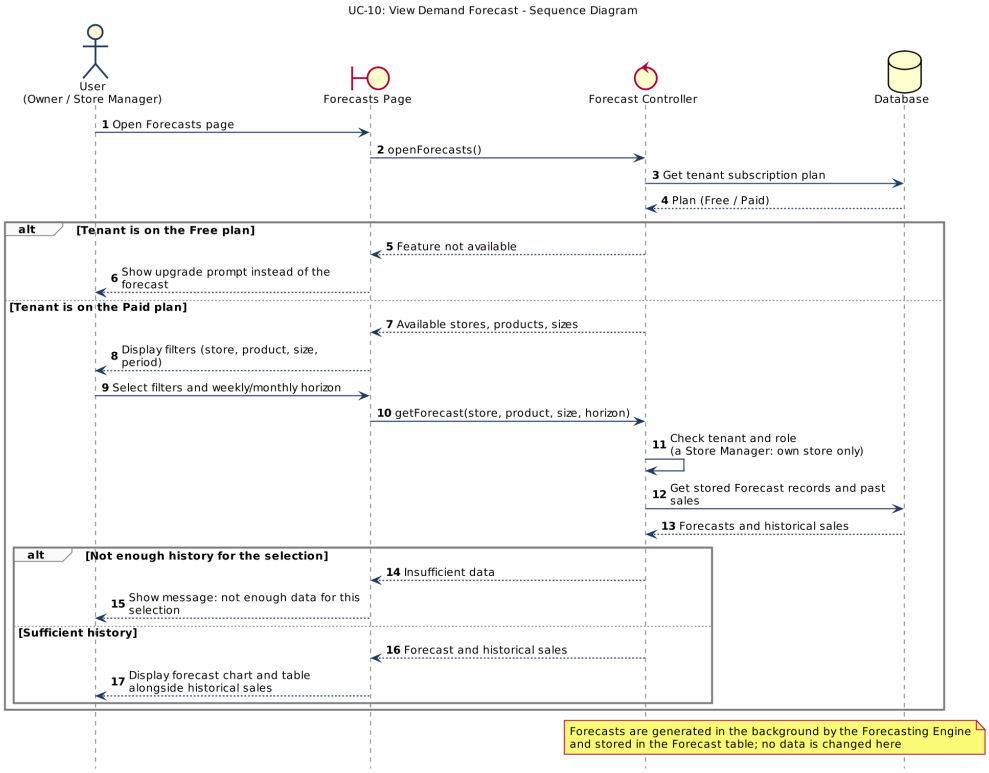


Figure 6.12: Sequence Diagram of UC-10 View Demand Forecast
This figure shows the sequence diagram of UC-10 (View Demand Forecast)

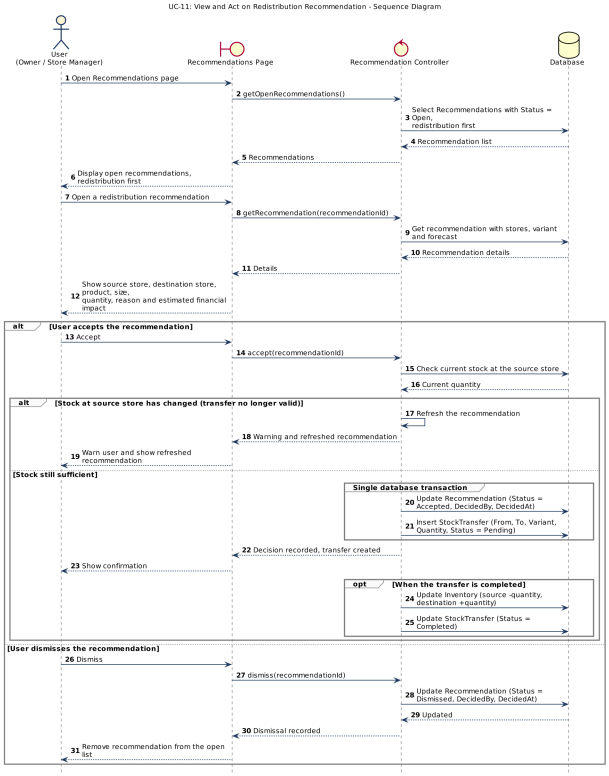


Figure 6.13: Sequence Diagram of UC-11 View and Act on Redistribution Recommendation
This figure shows the sequence diagram of UC-11 (View and Act on Redistribution Recommendation)

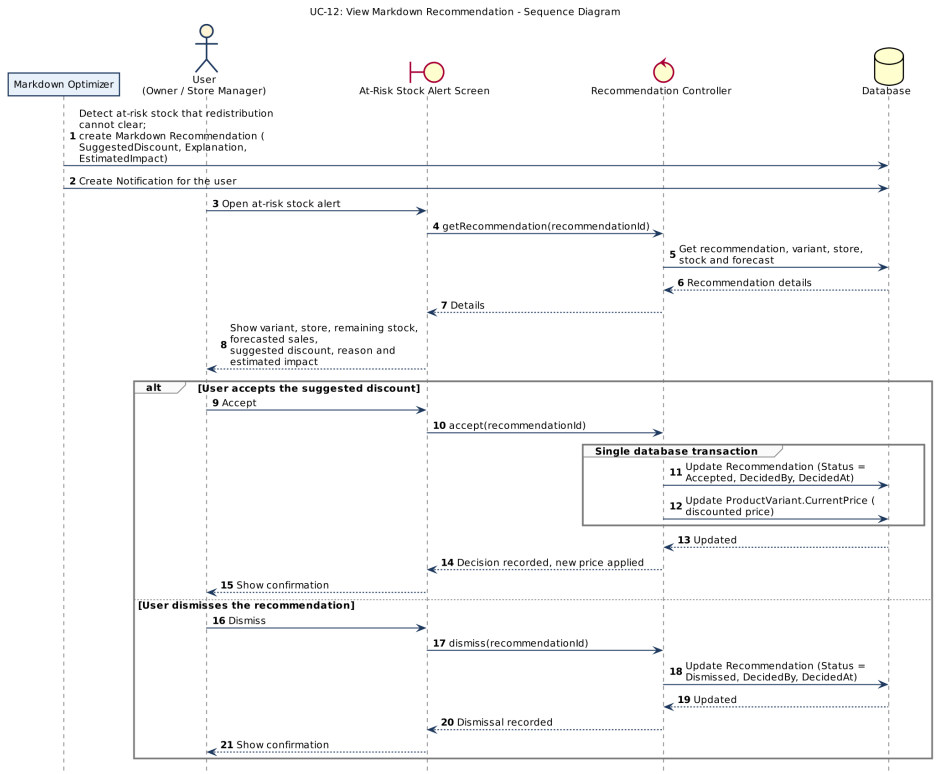


Figure 6.14: Sequence Diagram of UC-12 View Markdown Recommendation
This figure shows the sequence diagram of UC-12 (View Markdown Recommendation)

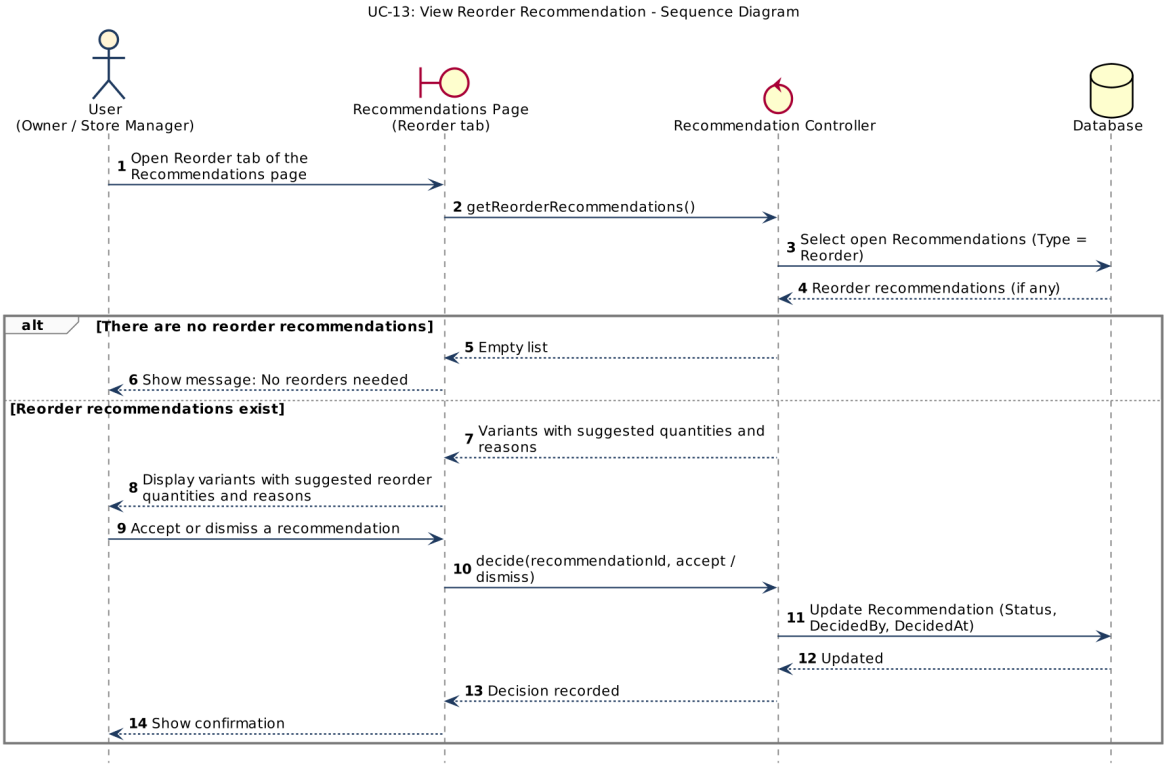


Figure 6.15: Sequence Diagram of UC-13 View Reorder Recommendation
This figure shows the sequence diagram of UC-13 (View Reorder Recommendation)

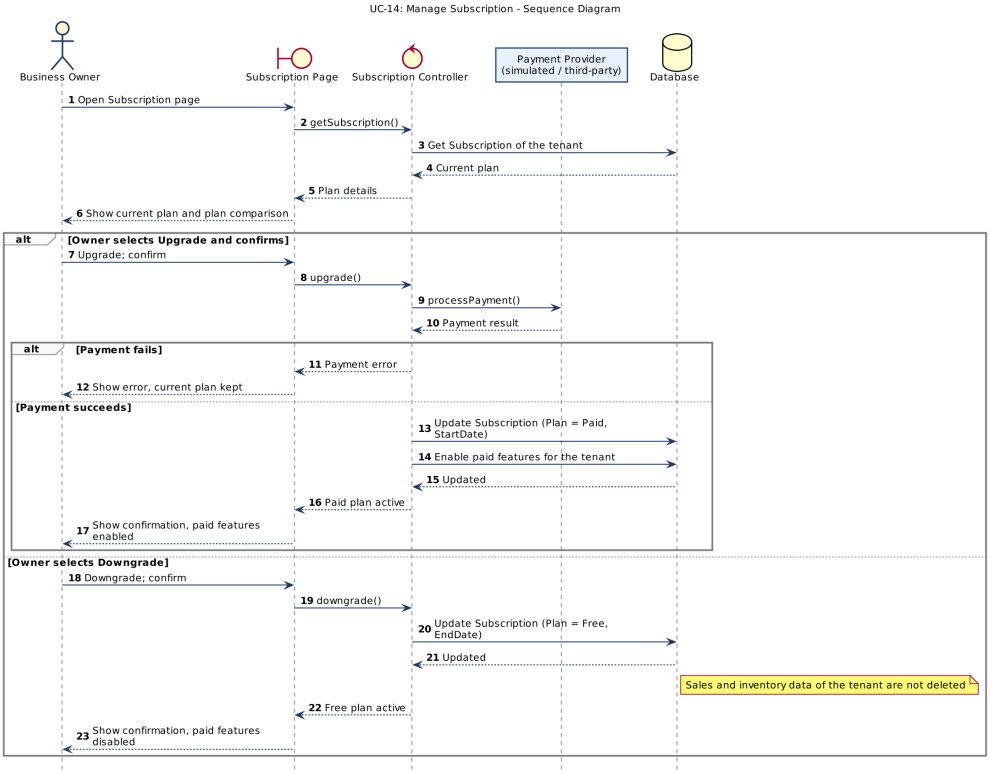


Figure 6.16: Sequence Diagram of UC-14 Manage Subscription
This figure shows the sequence diagram of UC-14 (Manage Subscription)

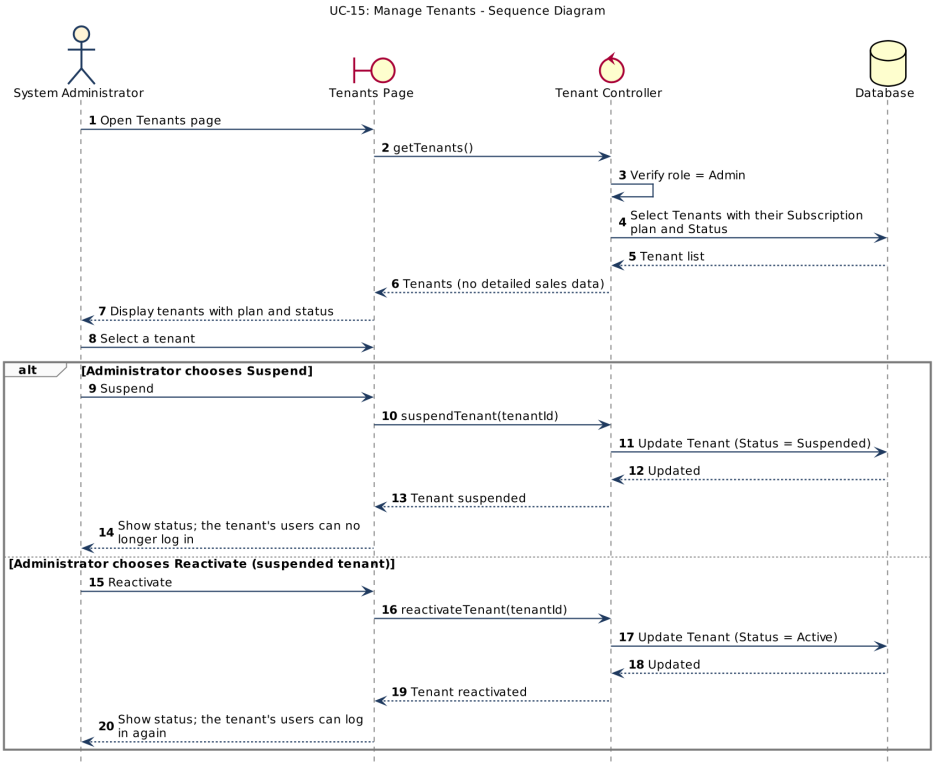


Figure 6.17: Sequence Diagram of UC-15 Manage Tenants
This figure shows the sequence diagram of UC-15 (Manage Tenants)

data of a particular business. Uploaded CSV and Excel files will be parsed, cleaned and normalized with pandas and openpyxl. Data preparation, the grouping of similar time series and the calculation of error metrics will use scikit-learn and NumPy, the main forecasting algorithm will be LightGBM and the statistical baselines will be computed with statsmodels. Experiments with the models will be carried out in Jupyter Notebook before the selected model is integrated into the platform. For development and collaboration, the team will use Git and GitHub for version control, Visual Studio Code as the editor and Postman for testing the API endpoints.

6.7.2 Coding Guidelines and Conventions

The backend and the forecasting code will be written in Python, and the frontend will be written in JavaScript with React. The team will follow agreed naming conventions in both languages, keep functions short and easy to understand, and add comments wherever the logic is not obvious, for example in the forecasting features and in the formulation of the redistribution problem. Each module will be organized in its own package so that its responsibility is clear and its code can be found easily. Every change will be reviewed and approved by another team member before it is merged into the main branch. Secure coding practices will be followed throughout, including hashing passwords, using parameterized database queries and never storing secret keys in the source code. These guidelines keep the code consistent across the three team members and make it easier to maintain after the project ends.

6.7.3 User Interface

StockSense is designed for business owners and store managers rather than technical users, so its interface must be easy to understand and navigate with minimal training. The system should be intuitive enough for first-time users to record their first sale or act on their first recommendation without reading a user manual. Forecasts will be presented as charts next to the actual sales, and every recommendation will be accompanied by an explanation that is easy to understand. All error messages will be written in simple and direct language that explains what went wrong and how to fix it. For example, if there is a problem with an uploaded file, the message will state which rows or columns caused the problem and why, rather than showing a technical error. The interface will also use consistent layouts and colours across all screens, so that users who learn one screen can easily use the others.

6.7.4 Extensibility

The system will be built in a modular way so that each module has its own function, with separate modules for the POS and inventory, data ingestion, forecasting, recommendations and markdown optimization. Because the boundaries between the modules are clear, one module can be changed without

changing the others. The forecasting module in particular will be designed so that several models can be tested side by side without changes to the rest of the application, since every forecast records the name of the model that produced it. The forecasting engine is general-purpose and can serve any business that stocks products with variants, sizes or several locations, and the recommendation engine is designed so that new rules can be added as the platform grows. Future extensions such as direct integrations with external POS systems, seasonal and promotional features, and support for businesses without sales history can therefore be added with limited impact on the existing code.

6.7.5 Tenant Data Isolation and Security

Every record in the system will be assigned to a tenant, and the backend will verify tenant ownership before any request reads or updates data. Features will be available according to the role of the user, so a store manager will be able to view and update only the data of the assigned store. All communication will take place over HTTPS. The system administrator will be able to manage tenants and plans but will not view detailed sales data unless this is required for support. Passwords will be stored only as secure hashes, and sessions will expire after a period of inactivity. Tenant isolation and role-based access will be tested explicitly with dedicated test cases that try to access the data of another tenant or another store, and any such attempt must be refused. These measures ensure that small businesses can trust the platform with their sales data, which is a precondition for its adoption.

6.7.6 Data Validation

Uploaded sales data will be validated before it is passed to the forecasting pipeline. Invalid or incomplete records will either be flagged or rejected, and an upload report will inform the owner about the accepted and rejected data, giving the reason for each rejected row. Data will be imported only after the owner has reviewed and approved the report, and the import will take place in a single transaction, so that an upload that fails part of the way through does not damage the data that has already been saved. Data entered through the POS will also be validated, for example to prevent negative quantities or sales of variants that do not exist. Because the quality of the forecasts depends directly on the quality of the data, validation is treated as a core part of the system rather than as an optional check.

6.7.7 Explainable Recommendations

Whenever a recommendation is created, the main reasons behind it, such as the forecast demand, the current stock, the surplus or shortage and the source branch, will be stored together with it, along with the estimated financial impact. In this way the owner of the business can understand why each suggestion was made and decide whether to follow it. The explanation is saved at the moment the recommendation

is created, so it always reflects the situation that produced the recommendation. Recommendations are also stored with their status, which is pending, actioned or dismissed, so that the owner can review past decisions and so that the team can later measure how often recommendations are accepted and whether they improved the results of the business.

6.7.8 System Testing

The testing policy of StockSense includes unit testing, integration testing and end-to-end testing. Unit tests will check individual parts of the system, such as the validation rules and the update of inventory after a sale. Integration tests will check that the API, the database and the modules work correctly together, and the REST endpoints will be tested with Postman. End-to-end tests will follow complete user journeys, such as registering a business, recording a sale, uploading a file and acting on a recommendation. Project-specific tests will check that no data leaks between tenants, that role-based access works for each type of user, and that the upload pipeline handles incomplete and badly formatted files correctly. The forecasting models will be compared with the statistical baselines on a test set that neither was trained on. Before launch, the performance targets from the requirements chapter will also be tested, such as saving a POS sale in less than two seconds and processing a file of 100,000 rows in less than five minutes.

6.7.9 Software Maintenance

The code will be kept in a version-controlled GitHub repository, and bugs and tasks will be tracked there so that no issue is lost. Failed uploads, rejected records and failed forecasting runs will be logged, which makes problems easier to identify and fix. As sales patterns change over time, forecasts will become less accurate if the models are not updated, so the models will be retrained regularly with the latest data and their accuracy will be monitored. The database will be backed up regularly so that business information can be restored if the database fails. Dependencies such as the Python libraries and the frontend packages will be updated periodically to receive security fixes. Together these practices keep the system reliable and accurate after it has been deployed and allow new team members to understand and continue the work.

6.7.10 Algorithms

The algorithms used in StockSense will be chosen with their time and space efficiency in mind and will be checked using Big-O analysis. LightGBM, an ensemble of gradient-boosted trees, has proven very effective on large retail datasets and performed strongly in the M5 competition. One model will be trained for each tenant on all of its products and sizes, and a new tenant must record or upload enough sales

history before forecasts are shown. Simple statistical models from statsmodels will be kept as baselines to check whether the more complex model is really needed. The recommendation algorithm compares forecast demand with current stock at all branches for each variant, matches branches with surplus to branches with shortages, then examines the stock that remains in surplus for possible markdowns, and finally suggests reorders. The markdown algorithm raises an alert when the sales rate of an item shows that it will not sell before its selling time runs out, so that action can be taken early. Since a business usually has only a few branches, matching branches in this way is computationally cheap. Database tables will be indexed on their primary keys and tenant identifiers, the latest forecasts will be cached instead of recomputed, and uploads and model training will run in the background.

6.8 Conclusion

This chapter explained the high-level and low-level design of StockSense. The proposed architecture defines separate modules for tenant management, the POS and inventory, data ingestion, forecasting, recommendations, markdown optimization and dashboard reporting, which communicate through the database and clear interfaces. Tenant isolation, role-based access control, data validation, background processing and database indexing form the basis of the security and scalability of the system. The design supports the main decision flow of the project, in which demand is forecast first, existing stock is redistributed where possible, markdowns are recommended for remaining slow-moving stock and new stock is ordered only when necessary. The domain model and the sequence diagrams show how each use case is carried out across the layers of the system, while the policies and tactics describe how the system will be built, tested and maintained. The testing and maintenance strategy will be used to make sure that the system is correct, performs well, is secure and remains accurate before and after it is put into use.

Chapter 7 Implementation and Test Cases

This chapter explains how the design of StockSense described in the previous chapters will be implemented and how the system will be tested. It describes each module of the system, the tools used to build it and its current status, and then presents the test cases written for every use case in Chapter 4, followed by additional test cases for the most important non-functional requirements. In FYP-1 the work concentrated on the requirements, methodology and design of the system, so most modules are fully designed but not yet implemented. The descriptions below therefore state how each module will be built in FYP-2, and the test cases are written so that they can be executed as soon as each module is ready. The content of this chapter is not final and will be updated in FYP-2 as the implementation progresses and the test results become available.

7.1 Implementation

The solution will be implemented according to the iterative plan described in Section 5.11. Development starts with the multi-tenant core, which includes tenants, users, stores, products, inventory, the POS and the ingestion pipeline, because every later module depends on the data that these parts provide. The offline forecasting experiments follow, in which the features, baselines and LightGBM model are developed and compared in Jupyter Notebook on historical data. The selected model is then integrated as a background job, and the recommendation and markdown modules are built on top of its forecasts, followed by the explanations and the dashboard views. Each module is developed separately, so that changes to one of them can be made without affecting the others, and each one is tested before the next is started. Table 7.1 summarizes the main tools and the current status of each module.

7.1.1 Development Environment

The backend will be written in Python with FastAPI, and the web dashboard and the POS screens will be written in React. All data will be stored in PostgreSQL, and every business table will have a tenant identifier. The team will use Visual Studio Code as the editor, Git and GitHub for version control, Postman for testing the API and Jupyter Notebook for the forecasting experiments. Each team member will run the full system locally, with the backend, the frontend and a PostgreSQL database, so that changes can be tested before they are merged. The team will work in short iterations, and the supervisor will review the results at the end of each iteration. Code will be merged into the main branch only after review by another team member, which keeps the main branch in a working state and spreads knowledge of each module across the team.

Table 7.1: Implementation Status of StockSense Modules

No.	Module	Main Tools	Status
1	Tenant and User Management	FastAPI, PostgreSQL	Design complete, implementation in FYP-2
2	Store, Product and Inventory Management	FastAPI, PostgreSQL	Design complete, implementation in FYP-2
3	POS	React, FastAPI	Screen designed, implementation in FYP-2
4	Data Ingestion and Cleaning	pandas, openpyxl	Design complete, implementation in FYP-2
5	Demand Forecasting	LightGBM, statsmodels	Offline experiments in FYP-2
6	Recommendation Engine	Python, SciPy	Design complete, implementation in FYP-2
7	Markdown Optimizer	Python, NumPy	Design complete, implementation in FYP-2
8	Explainable Recommendations	LightGBM contributions, text templates	Design complete, implementation in FYP-2
9	Dashboard and Alerts	React	Screens designed, implementation in FYP-2
10	Subscription Management	FastAPI, PostgreSQL	Design complete, implementation in FYP-2

7.1.2 Tenant and User Management

This module will handle business registration, login and the management of roles. When a business signs up, a tenant, a free subscription and an owner account will be created in a single database transaction, so that a failure part of the way through cannot leave an incomplete tenant behind. Only password hashes will be stored in the database. After login, every request will carry the tenant identifier and the role of the user, and the backend will check both on every read and write of business data, so that no request can reach data that belongs to another tenant or, for store managers, to another store. The module will also let the owner invite store managers by email and assign them to a store, and it will let the system administrator suspend and reactivate tenants, with the rule that users of a suspended tenant cannot log in until the tenant is reactivated.

7.1.3 Store, Product and Inventory Management

The owner will be able to create, modify and archive stores, define products together with their sizes and variants, and invite store managers. Each variant will have an SKU that is unique within the tenant, and an attempt to create a duplicate SKU will be rejected with a clear message. A store that still has inventory or sales history will be archived instead of deleted, so that its history remains available for forecasting. Inventory will be stored for each product variant in each store, and a manual correction of

the inventory, for example after a physical count, will require a reason that is logged together with the change. A store manager will be able to view and change only the data of the assigned store. Together these functions provide the catalog and stock data on which the POS, the forecasting module and the recommendation engine depend.

7.1.4 POS Module

On the POS screen, a store manager will select the product and the size, enter the quantity and complete the sale. The system will first check whether enough stock is available and will warn the manager before confirming a sale that would exceed it. Once the sale is confirmed, the sale, the sold items and the updated stock levels will be stored in one transaction, so that the inventory always matches the recorded sales. The goal is to complete this within two seconds, as required in Chapter 4, because the POS is used while customers wait at the counter. The POS will remain available even when the forecasting or upload modules are busy, since it does not depend on them. As the free part of the platform, the POS is also the main source of the sales data that later feeds the forecasting and recommendation modules.

7.1.5 Data Ingestion and Cleaning

This module will accept CSV and Excel files. CSV files will be parsed with pandas in chunks, while Excel files will be parsed with pandas and openpyxl in batches, so that memory use stays low for files of up to 100,000 rows. The owner will match the detected columns to the standard fields of product, size, store, date and quantity and will choose the date format. The system will then validate the structure, values and references of every row, normalize the names of products and sizes, for example by turning “Medium” into “M”, detect returns and unusual values, and mark possible duplicates without removing them. The upload report will show the accepted and rejected rows together with the reason for each rejection. Only after the owner approves the report will the rows be imported, in one transaction that is rolled back if any part of it fails.

7.1.6 Demand Forecasting

Forecasting will first be developed and tested offline in Jupyter Notebook. The steps are to build weekly and monthly demand series for every variant and store, to compute the lag, rolling, calendar, price and lifecycle features, to train one global LightGBM model per tenant with the Tweedie objective, and to compare its performance with the naive, seasonal naive, moving average, Holt-Winters and Croston baselines. An upper quantile model will also be trained to derive the safety stock used by the recommendation engine. The hyperparameters will be tuned with rolling-origin validation and without any shuffling of the data. The selected model will then be integrated into a background job that forecasts

demand and writes the results, together with the model name, into the forecast table. For a new tenant without enough history, no model forecast will be produced, and the user will be asked to record or upload more data.

7.1.7 Recommendation Engine

For each variant and store, the engine will compare the stock on hand with the forecast demand and the safety stock in order to detect a surplus or a shortage. The surplus stock will then be allocated to the stores with shortages by solving a small transportation problem, and a transfer will be suggested only if the margin at the destination is greater than the cost of the transfer. If a shortage still remains after redistribution, a reorder recommendation will be generated for the remaining quantity. Accepted transfers that have not yet arrived will be counted as incoming stock, so the same transfer is never recommended twice. The engine will be implemented in Python, with SciPy used to solve the transportation problems, and it will run as a background job after each forecasting run so that the recommendations always reflect the latest forecasts.

7.1.8 Markdown Optimizer

The stock that remains in surplus after redistribution will be evaluated by its weeks of cover. If an item is not expected to sell within the time left, the optimizer will try discounts of 5%, 10% and so on up to 50%, adjusting the expected demand with the price elasticity of the product. The smallest discount that reaches the sell-through target without going below the cost price will be chosen, provided that it does not reduce the expected margin compared with not discounting at all. The estimated change in profit will be stored as the impact of the recommendation. Elasticity will be estimated from the tenant's own price history when it is varied enough, and otherwise a cautious estimate for the category will be used, so that the optimizer does not suggest discounts that are deeper than necessary.

7.1.9 Explainable Recommendations

All recommendations will be stored together with the figures that led to them, which are the forecast demand, the stock, the surplus or shortage, the stores involved, the margin, the cost and the impact, and with the main forecast drivers obtained from the LightGBM feature contributions. These values will be inserted into text templates, so that the owner reads a short and simple sentence rather than a score or a table of numbers. Because the explanation is created at the same moment as the recommendation and stored with it, it will always describe the situation that produced the recommendation, even if the stock or the forecast changes later. This part of the system is central to the goal of making ML-based advice understandable and trustworthy for owners who have no background in data analysis.

7.1.10 Dashboard, Alerts and Subscription

The dashboard will show the monthly sales, the items at risk of stockout and dead stock, the recent sales, the status of each branch and the open recommendations. The owner or a store manager will be able to approve or reject a recommendation, and an approved redistribution will create a stock transfer and notify both stores. On the free plan, the forecast and recommendation sections will be replaced by an upgrade notice. The subscription component will switch the tenant between the free and paid plans, and a failed payment will keep the previous plan unchanged. A downgrade will turn the paid features off but will keep all sales and inventory data, so that the tenant loses nothing if it upgrades again later. Alerts will notify users when new recommendations are raised or when an upload or forecasting job has finished.

7.2 Test Case Design and Description

Every use case from Chapter 4 has at least one test case, and the upload use case has two, one for a normal file and one for an unsupported or broken file. The tests will be performed in a Linux development environment, with the React dashboard open in Google Chrome and the backend and PostgreSQL running locally. Sample tenants with two or three branches and sample sales files, including files with deliberate errors, will be used as test data. Each test case table states the engineer who will run the test, the reviewer, the related use case or requirement, the objective, the pre-requisites, the steps and the expected result. Forecast accuracy is not checked by these test cases, since it will be measured separately using the evaluation protocol described in Section 5.10. The Result field of every test case currently reads “Not yet executed” and will be changed to Passed or Failed, with comments, when the tests are performed in FYP-2.

Test Cases 1 to 16 check that each use case behaves as specified, but they do not check the most important non-functional requirements of Chapter 4. Four additional test cases, numbered 17 to 20, therefore cover tenant data isolation, the speed of POS transactions, the processing time for large file uploads, and the storage of passwords together with the use of encrypted connections. These requirements are critical for a multi-tenant platform that small businesses must be able to trust with their sales data and use during busy business hours. They are written in the same format as the functional test cases and refer to the requirement or section they verify instead of a use case. Like the other test cases, they will be executed in FYP-2 once the corresponding modules are complete.

Table 7.2: Test Case 1 (Register Business)

Test Case ID	1
QA Test Engineer	Muhammad Usman
Reviewed By	Muhammad Rizwan Arshad
Test Date	To be filled
Use Case Reference	UC-01 Register Business
Objective	To check that a new business can register and gets a free-tier tenant.
Pre-Requisite	The email is not already registered.
Steps	1. Open the signup page. 2. Enter business name, owner name, email and password. 3. Submit the form.
Expected Result	A tenant, a free subscription and an owner account are created, and a verification email is sent. A second signup with the same email shows an error.
Result	Not yet executed

Table 7.3: Test Case 2 (Login)

Test Case ID	2
QA Test Engineer	Muhammad Usman
Reviewed By	Hayyan Haider
Test Date	To be filled
Use Case Reference	UC-02 Login
Objective	To check login with valid and invalid details and with a suspended tenant.
Pre-Requisite	A registered user exists.
Steps	1. Log in with the correct email and password. 2. Log in with a wrong password. 3. Suspend the tenant and try to log in again.
Expected Result	Valid details open the dashboard. A wrong password shows “Incorrect email or password”. A suspended tenant cannot log in.
Result	Not yet executed

Table 7.4: Test Case 3 (Manage Stores)

Test Case ID	3
QA Test Engineer	Hayyan Haider
Reviewed By	Muhammad Usman
Test Date	To be filled
Use Case Reference	UC-03 Manage Stores
Objective	To check that the owner can add, edit and remove stores.
Pre-Requisite	The owner is logged in.
Steps	1. Add a store. 2. Edit its name. 3. Remove a store that has sales history.
Expected Result	The store is added and updated. A store with inventory or sales is archived and not deleted.
Result	Not yet executed

Table 7.5: Test Case 4 (Manage Store Managers)

Test Case ID	4
QA Test Engineer	Hayyan Haider
Reviewed By	Muhammad Rizwan Arshad
Test Date	To be filled
Use Case Reference	UC-04 Manage Store Managers
Objective	To check that the owner can invite a manager and assign a store.
Pre-Requisite	The owner is logged in and at least one store exists.
Steps	1. Invite a manager by email and choose a store. 2. Invite an email that already has an account.
Expected Result	The manager account is created and an invitation is sent. The second invitation is rejected with an error.
Result	Not yet executed

Table 7.6: Test Case 5 (Manage Products and Variants)

Test Case ID	5
QA Test Engineer	Muhammad Rizwan Arshad
Reviewed By	Muhammad Usman
Test Date	To be filled
Use Case Reference	UC-05 Manage Products and Variants
Objective	To check product creation and the unique SKU rule.
Pre-Requisite	The owner is logged in.
Steps	1. Create a product with sizes 40, 41 and 42. 2. Create another variant with an existing SKU.
Expected Result	The product and its variants are saved. The duplicate SKU is rejected.
Result	Not yet executed

Table 7.7: Test Case 6 (Adjust Inventory)

Test Case ID	6
QA Test Engineer	Muhammad Rizwan Arshad
Reviewed By	Hayyan Haider
Test Date	To be filled
Use Case Reference	UC-06 Adjust Inventory
Objective	To check manual stock correction and that a negative quantity is refused.
Pre-Requisite	The variant exists at the store.
Steps	1. Enter a new quantity with a reason. 2. Enter a negative quantity.
Expected Result	The first change is saved and logged. The negative value is rejected and the stock stays the same.
Result	Not yet executed

Table 7.8: Test Case 7 (Record Sale (POS))

Test Case ID	7
QA Test Engineer	Muhammad Usman
Reviewed By	Hayyan Haider
Test Date	To be filled
Use Case Reference	UC-07 Record Sale (POS)
Objective	To check that a sale reduces inventory, and that low stock gives a warning.
Pre-Requisite	A store manager is logged in and stock is available.
Steps	1. Select a product and size and sell 1 unit. 2. Try to sell more units than are in stock.
Expected Result	The sale is saved and the stock goes down by one. The second sale shows an insufficient stock warning.
Result	Not yet executed

Table 7.9: Test Case 8 (Upload Historical Sales Data)

Test Case ID	8
QA Test Engineer	Hayyan Haider
Reviewed By	Muhammad Usman
Test Date	To be filled
Use Case Reference	UC-08 Upload Historical Sales Data
Objective	To check upload, column mapping, cleaning and the upload report.
Pre-Requisite	The owner is logged in. A CSV file with a few wrong rows is ready.
Steps	1. Upload the file. 2. Map the columns. 3. Check the report. 4. Approve the import.
Expected Result	Good rows are cleaned and imported. Wrong rows are listed with reasons. Nothing is saved before approval.
Result	Not yet executed

Table 7.10: Test Case 9 (Upload of an Unsupported or Broken File)

Test Case ID	9
QA Test Engineer	Hayyan Haider
Reviewed By	Muhammad Rizwan Arshad
Test Date	To be filled
Use Case Reference	UC-08 Upload Historical Sales Data
Objective	To check that bad files do not damage existing data.
Pre-Requisite	The tenant already has saved sales.
Steps	1. Upload a .pdf file. 2. Upload a damaged Excel file. 3. Force a failure in the middle of an import.
Expected Result	The file is rejected with a simple reason, and a failed import is rolled back. The existing data is unchanged.
Result	Not yet executed

Table 7.11: Test Case 10 (View Dashboard and Alerts)

Test Case ID	10
QA Test Engineer	Muhammad Rizwan Arshad
Reviewed By	Muhammad Usman
Test Date	To be filled
Use Case Reference	UC-09 View Dashboard and Alerts
Objective	To check the dashboard for free and paid tenants and for a new tenant.
Pre-Requisite	One free tenant and one paid tenant exist.
Steps	1. Open the dashboard as a paid owner. 2. Open it as a free owner. 3. Open it for a tenant with no sales.
Expected Result	The paid tenant sees sales, inventory, forecasts and alerts. The free tenant sees sales and inventory with an upgrade message. A new tenant is asked to add sales data.
Result	Not yet executed

Table 7.12: Test Case 11 (View Demand Forecast)

Test Case ID	11
QA Test Engineer	Muhammad Usman
Reviewed By	Hayyan Haider
Test Date	To be filled
Use Case Reference	UC-10 View Demand Forecast
Objective	To check the forecast display and the rules for the free tier and short history.
Pre-Requisite	A paid tenant with enough sales history exists.
Steps	1. Select a product, size and store. 2. Choose weekly and then monthly view. 3. Repeat for a tenant with too little history. 4. Repeat for a free tenant.
Expected Result	The forecast is shown next to past sales. A short history shows a “not enough data” message. A free tenant sees an upgrade message.
Result	Not yet executed

Table 7.13: Test Case 12 (Redistribution Recommendation)

Test Case ID	12
QA Test Engineer	Muhammad Rizwan Arshad
Reviewed By	Muhammad Usman
Test Date	To be filled
Use Case Reference	UC-11 View and Act on Redistribution Recommendation
Objective	To check that a surplus store and a shortage store give a transfer suggestion that can be approved or dismissed.
Pre-Requisite	One branch has surplus stock and another has a forecast shortage of the same variant.
Steps	1. Run the recommendation job. 2. Open the recommendation. 3. Approve it. 4. Repeat with another one and dismiss it.
Expected Result	A redistribution is suggested with quantity, explanation and impact. Approval creates a stock transfer and both stores are notified. A dismissed one is closed. If the source stock has changed, a warning is shown.
Result	Not yet executed

Table 7.14: Test Case 13 (Markdown Recommendation)

Test Case ID	13
QA Test Engineer	Hayyan Haider
Reviewed By	Muhammad Rizwan Arshad
Test Date	To be filled
Use Case Reference	UC-12 View Markdown Recommendation
Objective	To check that slow-moving stock that cannot be moved gets a discount suggestion.
Pre-Requisite	A variant has weeks of cover above the selling time and no store needs it.
Steps	1. Run the optimizer. 2. Open the markdown recommendation. 3. Accept it.
Expected Result	A discount is suggested that does not go below the cost price. Accepting it updates the price.
Result	Not yet executed

Table 7.15: Test Case 14 (Reorder Recommendation)

Test Case ID	14
QA Test Engineer	Muhammad Usman
Reviewed By	Hayyan Haider
Test Date	To be filled
Use Case Reference	UC-13 View Reorder Recommendation
Objective	To check that a reorder is suggested only when no other branch has surplus.
Pre-Requisite	A forecast shortage exists and no other branch has surplus.
Steps	1. Run the recommendation job. 2. Open the Reorder tab. 3. Mark one as actioned.
Expected Result	A reorder quantity is shown with its reason. The status changes to Actioned.
Result	Not yet executed

Table 7.16: Test Case 15 (Manage Subscription)

Test Case ID	15
QA Test Engineer	Muhammad Rizwan Arshad
Reviewed By	Muhammad Usman
Test Date	To be filled
Use Case Reference	UC-14 Manage Subscription
Objective	To check upgrade, downgrade and failed payment.
Pre-Requisite	The owner is logged in.
Steps	1. Upgrade to the paid plan. 2. Try an upgrade where the payment fails. 3. Downgrade to free.
Expected Result	Paid features turn on after a successful upgrade. A failed payment keeps the old plan. A downgrade turns paid features off but keeps all sales and inventory data.
Result	Not yet executed

Table 7.17: Test Case 16 (Manage Tenants)

Test Case ID	16
QA Test Engineer	Hayyan Haider
Reviewed By	Muhammad Usman
Test Date	To be filled
Use Case Reference	UC-15 Manage Tenants
Objective	To check that the administrator can suspend and reactivate a tenant.
Pre-Requisite	The administrator is logged in.
Steps	1. Suspend a tenant. 2. Try to log in as that tenant's user. 3. Reactivate the tenant.
Expected Result	Users of a suspended tenant cannot log in. After reactivation they can log in again.
Result	Not yet executed

Table 7.18: Test Case 17 (Tenant Data Isolation)

Test Case ID	17
QA Test Engineer	Muhammad Usman
Reviewed By	Hayyan Haider
Test Date	To be filled
Use Case Reference	FR-11, Security (Section 4.4.3)
Objective	To check that one business can never see the data of another.
Pre-Requisite	Two tenants with different data exist.
Steps	1. Log in as Tenant A. 2. Call the API with the IDs of Tenant B records. 3. Log in as a store manager and open another store's data.
Expected Result	All such requests are refused. No data of Tenant B or of another store is returned.
Result	Not yet executed

Table 7.19: Test Case 18 (POS Speed)

Test Case ID	18
QA Test Engineer	Muhammad Rizwan Arshad
Reviewed By	Muhammad Usman
Test Date	To be filled
Use Case Reference	Performance (Section 4.4.1)
Objective	To check that a POS sale is saved in under 2 seconds.
Pre-Requisite	A tenant with a normal amount of data.
Steps	Record 50 sales and note the time taken for each.
Expected Result	Each sale is saved in under 2 seconds.
Result	Not yet executed

Table 7.20: Test Case 19 (Large File Upload)

Test Case ID	19
QA Test Engineer	Hayyan Haider
Reviewed By	Muhammad Rizwan Arshad
Test Date	To be filled
Use Case Reference	Performance (Section 4.4.1)
Objective	To check that a file with 100,000 rows is processed in under 5 minutes.
Pre-Requisite	A test CSV and a test Excel file with 100,000 rows each.
Steps	Upload both files and note the processing time and memory use.
Expected Result	Both files are processed in under 5 minutes without the system crashing.
Result	Not yet executed

Table 7.21: Test Case 20 (Password and Connection Security)

Test Case ID	20
QA Test Engineer	Muhammad Usman
Reviewed By	Muhammad Rizwan Arshad
Test Date	To be filled
Use Case Reference	Security (Section 4.4.3)
Objective	To check that passwords are hashed and the traffic uses HTTPS.
Pre-Requisite	A test database and a deployed test server.
Steps	1. Look at the saved password field in the database. 2. Open the site over plain HTTP.
Expected Result	Only hashed passwords are stored. Plain HTTP is not accepted.
Result	Not yet executed

7.3 Test Metrics

The test metrics in Tables 7.22 and 7.23 will be filled in after the test cases are executed in FYP-2, and only the planned number of test cases is given at this stage. The functional metrics cover the sixteen test cases written for the use cases, and the non-functional metrics cover the four additional test cases for isolation, speed, large uploads and security. For each group, the number of passed and failed test cases and the defect density will be reported, where the defect density is the number of defects found divided by the number of test cases executed. Any test that fails will be investigated, the defect will be fixed, and the test will be repeated until it passes, so that the final metrics reflect the state of the system at the time of the final demonstration.

Table 7.22: Functional Test Case Metrics

Metric	Value
Number of Test Cases	16
Number of Test Cases Passed	To be filled
Number of Test Cases Failed	To be filled
Test Case Defect Density	To be filled

Table 7.23: Non-Functional Test Case Metrics

Metric	Value
Number of Test Cases	4
Number of Test Cases Passed	To be filled
Number of Test Cases Failed	To be filled
Test Case Defect Density	To be filled

7.4 Conclusion

This chapter described how each module of StockSense will be implemented and how the system will be tested. Each module has its own tools and responsibility, from tenant management and the POS to forecasting, recommendations, markdown optimization and the dashboard, and the order of development follows the dependencies between them, with the data-producing modules built first. Every use case specified in Chapter 4 has at least one test case, and four further test cases cover the requirements that ordinary functional tests do not check, namely tenant isolation, the speed of POS transactions, the processing of large uploads and the security of passwords and connections. All test results will be recorded in FYP-2 once the system has been developed, and any test that fails will be fixed and repeated. The next chapter describes the experiments that will evaluate the forecasting models and the recommendations.

Chapter 9 Experimental Results and Discussion

In FYP-1 our work went mostly into the requirements, methodology and design of StockSense, so the forecasting and recommendation experiments have been designed but not yet run. This chapter describes each planned experiment, including the data it uses, how it is set up and the table into which its results will be entered. The result fields are marked “To be filled” for now and will be completed in FYP-2, following the iterative plan in Section 5.11. Fixing the design of the experiments at this stage has two benefits. It ensures that the experiments test the specific decisions made in Chapter 5 rather than only the final model, and it prevents the evaluation from being adjusted after the results are known, which would make the comparison less fair. The chapter first states the questions that the experiments answer, then describes the datasets, the data preparation and the evaluation protocol, and finally presents each of the five experiments followed by a discussion of how the results will be interpreted and of the main threats to their validity.

9.1 Objectives of the Experiments

The experiments are designed to answer five questions that follow directly from the methodology. The first asks whether a global LightGBM model is more accurate than simple statistical baselines at forecasting demand at the product, size and store level. The second asks whether one global model per tenant performs better than a separate model for each series when the sales history is short. The third asks how much sales history a new tenant needs before forecasts can be trusted, which determines how long a new business must wait before it receives model-based forecasts. The fourth asks which loss function, Tweedie, Poisson or squared error, works best for sparse retail demand. The fifth asks whether the redistribution and markdown recommendations actually reduce dead stock and lost sales compared with taking no action. The first four questions concern the accuracy of the forecasts, while the last concerns their value for the business, which is ultimately the more important measure.

9.2 Experimental Setup

This section describes the data, its preparation, the tools and the evaluation protocol that are shared by all of the experiments.

9.2.1 Datasets

Three sources of data will be used. The first is the public M5 dataset [7], which contains daily item and store sales, prices and calendar events from Walmart stores. It supports evaluation at the store level and

at higher levels of the product hierarchy, so it will be used for the early development of the models, but it does not contain any sales by size. The second source is sales data from a small multi-branch clothing or footwear retailer, which will be used if access to it can be secured. Behaviour at the size level can only be tested on data of this kind, so conclusions about sizes will be drawn only from it. The third source is a set of small sample tenants with two or three branches each, which will be used to test the upload pipeline and the recommendation logic from end to end. Using the three sources together allows the forecasting models to be developed on a large public dataset while the parts of the system that are specific to small retailers are tested on data that resembles their situation.

9.2.2 Data Preparation

The data is cleaned and aggregated into weekly and monthly series for each variant and store, as described in Section 5.4. Each series is labelled as smooth, erratic, intermittent or lumpy using the ADI and CV^2 thresholds of 1.32 and 0.49, so that accuracy can be reported separately for each demand class. Periods in which a variant was out of stock for most of the time are treated as censored and left out of the training loss, because the sales observed in those periods understate the real demand. The features described in Section 5.5 are then computed for every series using only the data available at each forecast origin. The experiments are run offline in Jupyter Notebook with Python, pandas, NumPy, scikit-learn, LightGBM and statsmodels, and fixed random seeds are used so that every result can be reproduced by another member of the team.

9.2.3 Evaluation Protocol

All splits are chronological, so the models are always trained on earlier periods and tested on later ones. Hyperparameters are tuned with rolling-origin validation on an expanding window, and the final comparison uses the most recent period, which no model sees during training or tuning. Every model and baseline is evaluated on the same splits, so that differences in results come from the models rather than from the data. Accuracy is measured with RMSE, MAE, WAPE and non-zero MAPE as defined in Section 5.10, and for MAPE the share of periods that had to be left out because the actual demand was zero is reported as well. Results are given for the weekly horizon in the main tables and for the monthly horizon in the same format where relevant.

9.3 Exploratory Analysis of the Demand Data

Before any model is trained, the demand series are analyzed to understand how sparse they are, and Table 9.1 will record the number and share of series in each demand class. This analysis matters because

the right choice of baseline and the expected level of accuracy both depend on it. If a large share of the series turn out to be intermittent or lumpy, baselines designed for that kind of demand, such as Croston's method, become more important in the comparison, and percentage errors such as MAPE become less reliable. The analysis will also show how much of the data is affected by stockouts and censoring, which indicates how strongly the observed sales may underestimate the real demand. The same analysis will be repeated for each dataset, since the public data and the data of a small retailer are expected to differ strongly in their sparsity.

Table 9.1: Distribution of Demand Classes

Demand Class	Number of Series	Share of Series (%)
Smooth	To be filled	To be filled
Erratic	To be filled	To be filled
Intermittent	To be filled	To be filled
Lumpy	To be filled	To be filled

9.4 Experiment 1 – LightGBM Against Statistical Baselines

This experiment tests the main model selection claim from Chapter 3, that the global LightGBM model is more accurate than simple baselines for retail demand at a detailed level. The LightGBM model, trained with the Tweedie objective, and every baseline are evaluated on the same chronological test period with a weekly horizon, and the results will be entered in Table 9.2. The results will also be broken down by demand class and by fast-moving and slow-moving items, because an overall average can hide poor accuracy on exactly the sparse series that matter most for dead stock. If LightGBM does not clearly beat the best baseline for some class of series, the simpler method will be used for that class in the system. Training time will also be recorded for each model.

Table 9.2: Forecast Accuracy of LightGBM and Baselines (Weekly)

Model	RMSE	MAE	WAPE (%)	MAPE (%)
Naive (last value)	To be filled	To be filled	To be filled	To be filled
Seasonal naive	To be filled	To be filled	To be filled	To be filled
Moving average	To be filled	To be filled	To be filled	To be filled
Holt–Winters	To be filled	To be filled	To be filled	To be filled
Croston's method	To be filled	To be filled	To be filled	To be filled
LightGBM (global)	To be filled	To be filled	To be filled	To be filled

9.5 Experiment 2 – Global Model Against Separate Models

Montero-Manso and Hyndman [10] and Bandara et al. [6] suggest that one shared model can beat separate models for each series when each series is short. This experiment checks whether that finding holds

for the product, size and store series of StockSense. One global model per tenant and one model per series are trained on the same data and evaluated on the same test period, and their accuracy and training time will be recorded in Table 9.3. Training time is included because a platform that serves many tenants must retrain its models regularly, so an approach that is slightly more accurate but much slower may not be practical. The comparison will also be broken down by the length of the series, since the advantage of a global model is expected to be largest for the shortest series. If the separate models turn out to be more accurate for long series, a mixed approach could be considered in which long series receive their own models.

Table 9.3: Global Model Against Separate Models

Approach	RMSE	WAPE (%)	Training Time
One global model per tenant	To be filled	To be filled	To be filled
One model per series	To be filled	To be filled	To be filled

9.6 Experiment 3 – Amount of Training History

A new tenant does not see model forecasts until it has enough sales history, as described in Section 5.6. This experiment finds that minimum by training the model on only the last 4, 8 and 12 weeks of history and evaluating it on the same test period each time, with the results recorded in Table 9.4. The system will use the shortest history at which LightGBM clearly beats the best baseline as its minimum history threshold. Below that threshold, a new tenant will see baseline forecasts or a message asking for more data. This experiment therefore has a direct effect on the onboarding experience of new businesses, since a lower threshold means that useful forecasts become available sooner. The result will be reported for each demand class as well, because sparse series may need more history than smooth ones.

Table 9.4: Forecast Accuracy Against Training History

History Used	RMSE	WAPE (%)	Better than Best Baseline?
4 weeks	To be filled	To be filled	To be filled
8 weeks	To be filled	To be filled	To be filled
12 weeks	To be filled	To be filled	To be filled

9.7 Experiment 4 – Loss Function Comparison

Demand is never negative and contains many zeros, which is why the Tweedie objective was chosen in Chapter 5. This experiment checks that choice by training LightGBM with the Tweedie objective, the Poisson objective and squared error while keeping all other settings the same, and the results will

be recorded in Table 9.5. Squared error is included as a reference because it is the default objective of most regression models, even though it does not match the shape of sparse demand. If the Poisson objective performs as well as Tweedie, it may be preferred because it has one parameter fewer to tune. The comparison will be made for each demand class, since the advantage of Tweedie and Poisson over squared error is expected to be largest for intermittent and lumpy series. The objective that performs best overall will be used in the system.

Table 9.5: Comparison of LightGBM Objective Functions

Objective	RMSE	MAE	WAPE (%)
Tweedie	To be filled	To be filled	To be filled
Poisson	To be filled	To be filled	To be filled
Squared error	To be filled	To be filled	To be filled

9.8 Experiment 5 – Recommendation Backtest

The last experiment estimates what the redistribution, markdown and reorder recommendations are worth by replaying past sales, as described in Section 5.10. At each cut-off date, the engine produces recommendations from the data available up to that date. These are applied to a simulated stock record, and the following weeks are replayed with the actual sales as demand. The same weeks are also replayed without any action, and the two runs are compared in Table 9.6 using the number of dead-stock units, the units of sales lost to stockouts and the sell-through rate, together with the number of transfers and the estimated effect on margin. Unlike the first four experiments, this one measures the effect of StockSense on business results, which is the measure that matters most to a retailer.

Table 9.6: Recommendation Backtest Results

Measure	No Action	With StockSense	Change
Dead-stock units	To be filled	To be filled	To be filled
Lost sales (units)	To be filled	To be filled	To be filled
Sell-through rate (%)	To be filled	To be filled	To be filled
Number of transfers	–	To be filled	–
Estimated margin impact	–	To be filled	–

9.9 Discussion

Once the results are available, the discussion will focus on three points. The first is whether LightGBM is accurate enough compared with the baselines to justify its extra complexity, which matters most for intermittent and lumpy series, where simple methods often do well. The second is how much history a new tenant needs, since this decides how long onboarding takes in UC-10 and therefore how quickly a

new business sees the value of the paid features. The third is whether redistribution reduces dead stock and lost sales more than markdowns alone, which is the main claim behind StockSense and the feature that distinguishes it from the platforms reviewed in Chapter 3. The M5 results [7] and the small-business study by Purnamasari et al. [8] lead us to expect that gradient-boosted models will beat the statistical baselines, but this expectation still has to be confirmed by our own experiments, and the result for short histories in particular cannot be predicted from studies of large retailers.

9.9.1 Threats to Validity

Several factors could limit how far the results apply. The M5 dataset does not record sizes, so any conclusions about demand at the size level will come only from the data of a real retailer, if it becomes available. Sales recorded during stockouts understate the true demand, which can bias both the forecasts and the backtest even after censored periods are excluded, because short or partial stockouts may not be detected. The backtest also assumes fixed transfer lead times and a constant price elasticity for markdowns, and neither assumption may hold in practice, so the estimated benefits should be read as indications rather than exact predictions. Finally, results obtained for one retailer or dataset may not carry over to every tenant, since businesses differ in their products, seasonality and number of branches. These threats will be reported together with the results so that the findings are interpreted with appropriate caution.

9.10 Conclusion

This chapter set out the experiments that will evaluate StockSense, which are an analysis of demand sparsity, a comparison of LightGBM with statistical baselines, a comparison of global and separate models, a test of the minimum training history, a comparison of loss functions and a historical backtest of the recommendations. The datasets, the data preparation, the evaluation protocol and the result tables have been fixed in advance, so that the experiments in FYP-2 can be run and reported in a consistent and fair way. Together the experiments test each of the main decisions made in the methodology, from the choice of model and loss function to the priority given to redistribution, and they measure both the accuracy of the forecasts and their effect on dead stock and lost sales. All results will be entered in the tables above and discussed in FYP-2, together with the threats to validity identified in this chapter.

Chapter 10 Conclusion and Future Work

This chapter summarizes the work completed in FYP-1 and presents the plan for the work to be completed in FYP-2.

10.1 Conclusion

StockSense is proposed as a multi-tenant SaaS platform that helps small and medium-sized retailers manage their products, variants, inventory and stores in a single system. The platform combines a free POS and inventory layer with the upload of historical sales, demand forecasting at the product, size and location level, stock redistribution, markdown optimization, reorder recommendations and explanations for every recommended action. The proposed design addresses the main inventory problems of the target retailers. It uses sales data to reveal differences in demand between products, variants and branches, and it gives priority to moving available stock from a branch with a surplus to a branch with a shortage before suggesting markdowns or new orders. It also stores the reasons and estimated financial impact of each recommendation, so that business owners can understand and review the decisions of the system rather than follow a black box. The report has presented the project vision, the business opportunity, the related research and commercial products, the requirements, the methodology, the system architecture, the database design, the security controls, the testing strategy, the maintenance approach and the planned experiments.

The work completed in FYP-1 therefore covers the definition of the system scope, the review of competing platforms and previous research, the functional and non-functional requirements, the use cases, the data ingestion process, the forecasting and recommendation workflow, the evaluation protocol and the high-level and low-level design. The plan for FYP-2 follows the iterative approach described in Chapter 5. The backend, database, authentication, tenant isolation, dashboard, POS workflows and CSV and Excel upload pipeline will be implemented and integrated first, using a React dashboard, a Python REST backend, PostgreSQL, and pandas with openpyxl for data processing. The forecasting experiments of Chapter 9 will then be run, and the selected LightGBM model will be integrated together with its statistical baselines. After that, the redistribution recommendations, markdown optimization, reorder suggestions and explanation features will be developed. The completed system will be tested through unit, integration and end-to-end testing, evaluated with forecasting metrics and the historical backtest, and prepared for documentation, deployment and the final demonstration.

10.2 Future Work

Several extensions are planned beyond the scope of FYP-2. The first is to address the cold-start problem for businesses that have no past sales data at all, so that they can receive useful insights from the first day instead of waiting until enough history has been recorded. This could be done by using aggregated and anonymized patterns from similar businesses, with strict safeguards to preserve tenant isolation. The second is to make the platform compatible with a wide range of external POS systems through direct integrations, so that data can flow into StockSense automatically instead of being uploaded manually. The third is to add seasonal and promotional features that take into account changes in demand during holidays, festivals and different seasons, as well as the effect of discounts and special offers on sales. The fourth is to extend the redistribution model with transfer lead times and capacity limits, which are currently handled only as assumptions in the backtest. Together these extensions will help the system provide more accurate forecasts and more useful recommendations to a wider range of businesses.

Bibliography

[1] P. K. Singh, Y. Gupta, N. Jha, and A. Rajan, "Fashion retail: Forecasting demand for new items," in *Proceedings of the KDD 2019 Workshop on AI for Fashion*, Anchorage, AK, USA, Aug. 2019.

[2] K. J. Ferreira, B. H. A. Lee, and D. Simchi-Levi, "Analytics for an online retailer: Demand forecasting and price optimization," *Manufacturing & Service Operations Management*, vol. 18, no. 1, pp. 69–88, 2016.

[3] A. L. D. Loureiro, V. L. Miguéis, and L. F. M. da Silva, "Exploring the use of deep neural networks for sales forecasting in fashion retail," *Decision Support Systems*, vol. 114, pp. 81–93, 2018.

[4] F. Caro and J. Gallien, "Clearance pricing optimization for a fast-fashion retailer," *Operations Research*, vol. 60, no. 6, pp. 1404–1422, 2012.

[5] C. Paterson, G. Kiesmüller, R. Teunter, and K. Glazebrook, "Inventory models with lateral transshipments: A review," *European Journal of Operational Research*, vol. 210, no. 2, pp. 125–136, 2011.

[6] K. Bandara, C. Bergmeir, and S. Smyl, "Forecasting across time series databases using recurrent neural networks on groups of similar series: A clustering approach," *Expert Systems with Applications*, vol. 140, p. 112896, 2020.

[7] S. Makridakis, E. Spiliotis, and V. Assimakopoulos, "M5 accuracy competition: Results, findings, and conclusions," *International Journal of Forecasting*, vol. 38, no. 4, pp. 1346–1364, 2022.

[8] D. I. Purnamasari, V. A. Permadi, A. Saepudin, and R. P. Agusdin, "Demand forecasting for improved inventory management in small and medium-sized businesses," *Jurnal Nasional Pendidikan Teknik Informatika (JANAPATI)*, vol. 12, no. 1, pp. 56–66, 2023.

[9] R. Fildes, S. Ma, and S. Kolassa, "Retail forecasting: Research and practice," *International Journal of Forecasting*, vol. 38, no. 4, pp. 1283–1318, 2022.

[10] P. Montero-Manso and R. J. Hyndman, "Principles and algorithms for forecasting groups of time

series: Locality and globality," *International Journal of Forecasting*, vol. 37, no. 4, pp. 1632–1653, 2021.

[11] M. L. Fisher and A. Raman, "Reducing the cost of demand uncertainty through accurate response to early sales," *Operations Research*, vol. 44, no. 1, pp. 87–99, 1996.

[12] S. Thomassey, "Sales forecasts in clothing industry: The key success factor of the supply chain management," *International Journal of Production Economics*, vol. 128, no. 2, pp. 470–483, 2010.

[13] M. Seyedan and F. Mafakheri, "Predictive big data analytics for supply chain demand forecasting: methods, applications, and research opportunities," *Journal of Big Data*, vol. 7, p. 53, 2020.

[14] F. Petropoulos *et al.*, "Forecasting: theory and practice," *International Journal of Forecasting*, vol. 38, no. 3, pp. 705–871, 2022.

[15] D. Bertsimas and N. Kallus, "From predictive to prescriptive analytics," *Management Science*, vol. 66, no. 3, pp. 1025–1044, 2020.

[16] C. Rudin, "Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead," *Nature Machine Intelligence*, vol. 1, no. 5, pp. 206–215, 2019.

[17] D. Corsten and T. Gruen, "Desperately seeking shelf availability: an examination of the extent, the causes, and the efforts to address retail out-of-stocks," *International Journal of Retail & Distribution Management*, vol. 31, no. 12, pp. 605–617, 2003.

[18] SAP SE, "SAP integrated business planning for supply chain," https://www.sap.com/products/scm/integrated-business-planning.html, 2026, accessed: Nov. 12, 2026.

[19] Blue Yonder Group, Inc., "Blue yonder demand planning," https://blueyonder.com/solutions/demand-planning, 2026, accessed: Nov. 12, 2026.

[20] Nextail Labs, "Nextail: AI-powered merchandising platform for fashion retail," https://nextail.co, 2026, accessed: Nov. 12, 2026.

[21] Shopify Inc., "Shopify POS: Point of sale system," https://www.shopify.com/pos, 2026, accessed: Nov. 12, 2026.

[22] Lightspeed Commerce Inc., "Lightspeed retail POS," https://www.lightspeedhq.com/pos/retail/, 2026, accessed: Nov. 12, 2026.

[23] Odoo S.A., "Odoo point of sale," https://www.odoo.com/app/point-of-sale, 2026, accessed: Nov. 12, 2026.